A Postmortem of a Withdrawn Result

The Complete Record behind "Super-Precise Latency: How CPU Threading Distorts Benchmarks, an Industry-Wide Audit, and How to Fix It"

Gustavo Pedro Ricou

This document is the complete record behind the paper named above. It is archived with the data and is not part of the submission: the paper's supplementary material is a separate, shorter document organized by the paper's sections, which draws on this one and points into it as "postmortem, Sn". It is written as a postmortem by the lead author. It was written against the paper as it stood before 5 October 2026, when the paper was rebuilt around its four bottom lines. Its pointers to the paper's sections, tables and equations refer to that version, archived with this record in `docs/archive/2026-10-05-before-cleanup/`. Part I — *what we set out to measure, and what broke* — retells the work in the order it happened, with everything withdrawn along the way. Part II — *the mechanism, and the law it produces* — is the argument. Part III — *apparatus, data and tables* — is reference material, meant to be looked things up in rather than read through. Part IV — *the literature, and what it already requires* — places the result. Part V — *the pre-registered law campaign* — reports the campaign that tested the scheduling failure's law in advance, with every run it made drawn and every aberrant one marked. The paper's own references of the form "Supplement Sn" resolve to the supplementary material, not to this document, and the artifact's `docs/supplement_index.md` records where each section here came from and which section of the supplementary material draws on it.

CONTENTS

Part I. What we set out to measure, and what broke 5

S1. How these results were reached: the chronology, and everything withdrawn 5
S1.1. The phase excluded from every result . . . 6
S1.2. The answer we obtained first, and why it was wrong . . . 6
S1.3. The claim about the benchmark that we withdrew . . . 6
S1.4. The causality framing, withdrawn in this version . . . 6
S1.5. The M/G/1 form, withdrawn . . . 7
S1.6. The traced tail-index cross-check, withdrawn . . . 7
S1.7. Pre-registration outcomes . . . 7
S1.8. The achieved publish rate was taken from a flag, not measured . . . 7
S1.9. A source we read wrong, and how we found out . . . 7
S1.10. The distributed-mode gap . . . 7
S1.11. The first result set . . . 7
S1.12. The first corpus's prior corrections . . . 7
S1.13. The result . . . 8
S1.14. The checks it had already passed . . . 8
S1.15. Open questions from the comprehensive campaign . . . 8

S2. What the broker comparison rests on, and what it does not 9

S3. The twentyfold gap in event-time latency was publish delay, and we withdrew the claim 10
S3.1. The delay sweep failed its manipulation check . . . 12

S4. Our own replay-rate record did not survive the audit 12
S4.1. The co-location campaign we withhold . . . 15

S5. E1's shift is confined to the first seven events 15
S5.1. Where the knee sweep placed the load conditions . . . 16

S6. The even-denominator exclusion was wrong, and what replaced it 16

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie. This document accompanies the paper by G. P. Ricou (Trinity College Dublin) and R. Duvignau (Chalmers University of Technology, Gothenburg, Sweden); the retelling and its judgments are the lead author's.

Part II. The mechanism, and the law it produces 17

S7. Priority and geometry each move the rate on their own 17
S7.1. The overnight campaign, prediction by prediction . . . 17

S8. What the payload sweep fits, and what it does not support 18
S8.1. Tail recovery without a reference clock . . . 20

S9. The two-state model commentary and the tracer checks 20

S10. Retention sits on the grid, and the spread rule says when 22

S11. Grid membership, configuration by configuration 24

S12. Every real-time-priority pair, the ladder they make, and the exposure curve 24

S13. What the brokers actually do 27
S13.1. Equivalence on gated artifacts . . . 27
S13.2. The condition that reverses the ordering . . . 28
S13.3. The falsifier, registered in advance . . . 28
S13.4. Why the standard testbed cannot see it . . . 28

S14. The classical statistics of a discarded zero, and what the guard deletes 28
S14.1. The two floored clocks behind the $-1000\,\mu$s signature . . . 29
S14.2. The continuum null, constructed . . . 29

S15. Which clocksource, bounded from a measurement 30

S16. Why the acknowledgment timestamp is late, and why the broker is not the reason 31
S16.1. Where each timestamp is actually taken . . . 31
S16.2. Why one span cannot invert and the other can . . . 32
S16.3. The second thread is not the mechanism . . . 32
S16.4. Two rivals: the scheduler itself, and the hypervisor . . . 32
S16.5. Why averaging does not repair either mode . . . 33
S16.6. The redesign that does not help . . . 34
S16.7. Where the repairs belong . . . 34
S16.8. The scheduler constants, derived . . . 34
S16.9. The displacement recovery, distribution by distribution . . . 35
S16.10. How late: the rest of a slice, timed acknowledgment by acknowledgment . . . 36
S16.11. Two faults in the law campaign's judges, and the runs a stalled disk held . . . 38

S17. The 1970 counter note, line by line, and the grid geometry it explains 39

Part III. Apparatus, data and tables 40

S18. Each campaign moves one variable, and the audit threshold changes no conclusion 40
S18.1. What the audit gate selects, bounded . . . 40

S19. Threats to the scheduling attribution, and what rules them out 41

S20. The workload is a live football feed, described as far as the results require 45

S21. Every setting that can move a latency number 45

S22. This harness has discarded latency data before 46
S22.1. Distributed mode never completed a run in eleven attempts . . . 47

S23. Where the check transfers, and what it costs to run 48

S24. The campaign ledger, and what distributed mode did instead 49
S24.1. OpenMessaging distributed mode, failure diagnostics . . . 49
S24.2. The statistical inventory . . . 49
S24.3. The metric map . . . 50

S25. The evidence behind the count of tools that dispose silently 50
S25.1. Published results, audited against a signature fixed in advance . . . 53
S25.2. Every tool read at source, note by note . . . 53
S25.3. Every tool run, test by test . . . 55

S26. Tables 58
S26.1. The mechanism campaigns' tables . . . 59
S26.2. The audit, timestamping-mode and injected-delay tables . . . 59

Part IV. The literature, and what it already requires 60

S27. Where our irreproducibility sits in the variability literature 60

S28. The resolution literature knew the problem; the deletion failure is new 62
S28.1. The payload sweep's underpowered rows, and the comprehensive campaign's duration and plateau paragraphs (moved from the main text) . . . 63

S29. Two neighboring literatures, and what they do about the second clock 63
S29.1. "One-way latency" that is half a round trip . . . 64
S29.2. Drift-aware synchronization in MPI benchmarking . . . 64

S30. Where the timestamp is written, in other runtimes 65

S31. Neighboring rules, and what they already require 66

S32. What standardized benchmarks require about resolution 66

S33. Where our reading parts from the concurrent report 67
S33.1. The survey behind the two related-work sentences . . . 68
S33.2. Production tracing reached the conclusion without the mechanism . . . 68
S33.3. A framework that does not have the problem . . . 68
S33.4. How large the literature is that misses this . . . 69
S33.5. Where scheduling delay comes from, and what the queueing account gets right . . . 71

Part V. The pre-registered law campaign 71

S34. The campaign as registered, and as it ran 71

S35. Every prediction, as its frozen judge returned it 72

S36. Every aberrant run, marked, and why 74

S37. What the runs showed after the verdicts 75

References 75

List of Figures

S1 Once-per-run cost, diluted by the window . . . 13
S2 Withdrawn: a per-run start-up cost, read as a per-event gap . . . 14
S3 Longer end-to-end latencies invert less often . . . 18
S4 Two-state model, drawn . . . 21
S5 Campaign's confirmed prediction . . . 23
S6 Grid membership, as a set . . . 25
S7 All 8 pairs, as a ladder . . . 26
S8 Every manipulation separates; the broker does not . . . 27

S9 Where a path sits on the exposure curve . . . 28
S10 What the benchmark deletes, beside what it then reports . . . 30
S11 Within each broker, no shift between the verdicts can be detected . . . 36
S12 Each acknowledgment waits out the rest of a slice, not a whole one . . . 38
S13 Why a sample disappears, and why the survivors are not a sample . . . 39
S14 One variable per campaign, one claim per row . . . 41
S15 No natural threshold, and the choice moves no conclusion . . . 43
S16 System measured, and where its clocks are read . . . 50
S17 Every slice run, aberrant runs marked . . . 76
S18 Every tick, core-count and priority run, aberrant runs marked . . . 77
S19 Every load and language run, aberrant runs marked . . . 78
S20 Every L9 run, aberrant runs marked . . . 79

LIST OF TABLES

S1 First result set, later rejected . . . 8
S2 Historical corpus; the publish delay columns are withdrawn . . . 10
S3 Powered transport replication at a verified true-real-time rate, over a median of 125 events per run (not the seven of Table S2), audit-gated (Section S18). Medians in ms; the Hodges–Lehmann shift is $Kafka - Redis$ with a percentile-bootstrap 90% interval. The brokers are equivalent within 1 ms at every N (all three estimators) yet cleanly distinguishable: Redis is ≈ 0.41 ms faster, and the shift does not grow with concurrency. . . . 10
S4 Observation-window sweep separates a per-run cost from a per-event one . . . 12
S5 Why the delay sweep is not an effect-size manipulation . . . 12
S6 One set of runs reproduces both published answers . . . 15
S7 Grid membership against a continuum null . . . 24
S8 Every matched pair . . . 26
S9 What the displacement costs at longer paths, and how wide the cost is . . . 27
S10 **Only the rest-of-a-slice shape splits the two readings as the data do.** The slope of the fall's halfway point on the slice, which the registered law put between 0.8 and 1.2, on the free reading and on the fitted shape, as the frozen programs return it from the measured rates, from Equation S4 put in place of every run's rate, and from Equation S5 put in the same place. . . . 37
S11 What each claim rests on . . . 42
S12 Audit gate applied to the powered transport campaigns . . . 42
S13 Workload, from 3,315 matches . . . 45
S14 Every configuration that can move a latency number, and why it holds that value . . . 46
S15 Audited harnesses, one row per span rather than per tool . . . 51
S16 The registered outcomes of the published-results audit . . . 53
S17 Replicate spread against the phase denominator q . . . 59
S18 Retention against the producer's publish interval, with message size, background load, duration and host held fixed. Only the commensurate rate is unstable, and the two incommensurate rates agree with each other despite intervals differing by 19%. Predictions were recorded before the runs. . . . 59
S19 Retention against payload at 457 msg/s, incommensurate with the millisecond grid so samples stay dephased. Implied T_{true} inverts Equation 6. The final column is the increment in implied latency per bit of added payload, in nanoseconds, relative to the 200 B baseline; the two smallest steps are noise-dominated and marked. . . . 60
S20 H2: negative-span rate against achieved CPU utilization, no core pinning, background load swept from idle to oversubscription. Each row pools a condition's Kafka and Redis runs (30 runs, 3,582 events); the mixture of Table S22 and the knee sweep count Kafka's alone. The withdrawn queueing form fits at $R^2 = 0.945$ against 0.640 for a straight line. . . . 60
S21 Bounding the unequal-retention selection effect in E1 . . . 61
S22 Mixture, measured across nine load levels (25 runs each, pre-registered) . . . 61
S23 Waiting probability, not utilization . . . 62
S24 Same utilization, different rate . . . 62
S25 Longer end-to-end latencies are more reliable measurements, twice . . . 63
S26 Kernel tracing predicts the negative-span rate, unfitted, in every configuration that has one . . . 63
S27 Consistency audit of every run behind a reported result . . . 64

S28 H3: median broker transport under the two timestamping modes, both configurations at one in-flight request under matched load, in the campaign (E-C3) and its replication (E-C4). `callback` is the default asymmetric timestamping mode; `inline` timestamps Kafka on the calling thread as Redis already does. Each cell is the median over runs of a per-run median. Each campaign closes with the change from `callback` to `inline` and a 95% percentile-bootstrap interval, resampling runs within each cell. The between-system gap shrinks by 0.071 ms (0.039–0.113) in E-C3 and by 0.039 ms (0.008–0.059) in E-C4; Kafka's change carries 98% and 107% of it, more than all of it in E-C4 because Redis's change there runs the other way, and Redis's interval spans zero in both. Cells are rounded independently, so a change can differ in its last digit from the difference of the printed cells. 64
S29 Injected one-way broker delay, applied identically to both systems, five feeds 65
S30 **Every block of the campaign, and how many of its runs counted.** The campaigns and runs the judging read, by block (C0, the calibration, includes the eight calibrations of L2's bridge days). Repeated: a condition did not take. Stopped: the stopping rule. Unjudged: the run ended before the integrity rule read it. Paused: a message in the run arrived more than 150 ms late (S36); every paused run counted. 72
S31 **One prediction confirmed on every campaign, three on one broker, and the rest not.** Every pre-registered prediction of the law campaign, the number it was judged on (all runs; a slope, ratio or move with its 95% interval, from resampling whole rounds), and its verdict, broker by broker where the two differ; a prediction judged broker by broker is confirmed only where it holds on both. The last column is the one verdict that moves when the flagged, recorded or paused runs are left out. 73
S32 **Nine reasons a run is marked, and what each did to the verdicts.** Runs of the whole campaign carrying each mark, pair by pair, of the registry's 8,108. The first five keep a run out of every judge; the last four leave it counted. The verdicts were read again without the flagged and the paused runs, and without every run taken while the stopping rule only recorded (S35). 74

On review. Before this submission, the manuscript had not been submitted to, or reviewed by, any journal; the corrections recorded below were found in the course of the work.

On the late messages. On 7 October 2026 we found that the cloud testbed's consumers first read earlier campaigns' messages, which nothing had cleared, so that each run's first messages waited behind them (the supplementary material's S1.5). Every number this document emits from the cloud runs now leaves those messages out, and the placement, padding and tracer tables of S26 are generated from the recount. A number typed into the retelling of a round is as that round computed it, with the late messages in; where a conclusion moved, the text says so.

On the vocabulary. This document uses the paper's terms, defined here once. A *condition* is one point of the experimental design; a *cell* is a condition together with a workload; a *run* is one execution of a cell and a *replicate* a further run of the same cell; where a manipulation compares settings that differ in one variable, each setting is a *configuration*. The *admission condition* is the benchmark's filter that keeps an end-to-end sample only when its millisecond-quantized difference is strictly positive; the audit registry (Section S25) uses *guard* for a tool's own conditional of that kind, where the word names a code path rather than the benchmark's. The *timestamp resolution* τ is the smallest difference a timestamp can express, one millisecond here. The *scheduling delay* is the time a thread that is ready to run spends queued before a core runs it; the *publish delay* is how late the publish call ran after the event was due. The measured quantities carry the industry's names: the *end-to-end latency* D, the *publish latency* A and the publish delay are the OpenMessaging Benchmark's own [S1], and the *event-time latency* E and the consumer's *processing-time latency* are Karimov et al.'s [S2]. The paper's two failures are the *scheduling failure* (Section IV of the paper: a negative span from a late acknowledgment timestamp) and the *deletion failure* (Section V: samples shorter than the timestamp resolution discarded uncounted).

PART I. WHAT WE SET OUT TO MEASURE, AND WHAT BROKE

This part is the postmortem proper, in the order the work happened. Section S1 retells the chronology and everything withdrawn along the way. Sections S2 and S3 hold the two results we took back: what the broker comparison rests on once the wrong corpus is removed, and the twentyfold gap in event-time latency that turned out to be publish delay. Section S4 turns the same scrutiny on our own record-keeping, which did not survive it. Sections S5 and S6 close the two reconciliations — where E1's shift comes from, and the exclusion we got wrong.

S1. HOW THESE RESULTS WERE REACHED: THE CHRONOLOGY, AND EVERYTHING WITHDRAWN

The main text is organized around what the evidence supports. This section holds what that reorganization displaced: the order in which the work happened, every claim withdrawn, and the pre-registrations that failed. It is kept in full because an audit's credibility rests on its own failures being visible, and because a reader checking whether the corrections were real should not have to take the summary on trust.

We are not alone in needing the practice, and the field has not settled how to provide it. In 2026 Bostanci et al. showed that a conference paper carrying all three artifact badges, including *Reproducible*, had reported incorrect simulator results through configuration errors, and that its artifact could not rebuild its own headline claims [S3]. Their closing recommendation is the one this section is an attempt at: that the community needs mechanisms for correcting results and artifacts *after* publication, rather than only gates before it. A withdrawal recorded in the document that made the claim is the cheapest such mechanism we know of.

S1.1. The phase excluded from every result

Measurement perturbs, and one early phase perturbed enough to be unusable: it pinned the load generator to a subset of cores while utilization was measured across all of them, diluting ρ. Every number in the main text excludes it, and the campaigns that remain measure utilization with no core pinning. It is recorded here rather than dropped because a phase removed without saying so is the same omission this paper is about.

S1.2. The answer we obtained first, and why it was wrong

The study began as a broker comparison on a live sports feed. The first complete result set was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and had a tidy architectural story: a twentyfold gap in event-time latency between the two brokers. It passed replicate counts, effect sizes, corrected significance and sanity plots.

Two independent defects destroyed it. The sign check of Section II-C rejected 862 of the $1{,}382$ runs on that testbed, including 109 of the 126 runs behind the headline and every one of its six conditions: only 8 of that testbed's 76 conditions survive it. Separately, the twentyfold gap itself was a per-run start-up cost read as a per-event constant: the median was taken over as few as seven events per run, and a deterministic per-run artifact reproduces across a hundred runs beautifully. The second defect is the more instructive one, because the run data were causally consistent throughout. Causal consistency is necessary, not sufficient.

S1.3. The claim about the benchmark that we withdrew

An earlier version of this work read the OpenMessaging Benchmark's discards as causality violations. They are not. The instrumented counter we had added could not distinguish sign, and separating it showed the Kafka-driver corpus's $10{,}913{,}263$ discarded samples contain not one negative: they are resolution failures, samples whose true end-to-end latency was shorter than the timestamp resolution. The refuting evidence had been in our own artifact from the day we committed it, in a field we had printed and not read. The same separated channel then caught $41{,}403$ genuine negatives in the Redis-driver replication, which is the result Section V-F reports.

We also called the mechanism behind those Redis-driver negatives a *candidate* — a sub-millisecond skew between two views of a virtualized clock — and hedged it as such through the round-1 internal draft. The hedge was unnecessary, and reading the driver removes it. `RedisBenchmarkConsumer.java` takes the publish timestamp from `entry.getID().getTime()`, the stream-entry identifier the Redis *server* assigns on `XADD`, while the Kafka driver uses a producer-set timestamp. The Redis span is therefore a difference between two millisecond-floored clocks on two hosts, for which $-1000\,\mu$s is the only negative value expressible; the Kafka span floors two timestamps differenced inside one JVM, and never goes negative. The corpus matches both predictions exactly. Hedging a claim the source code settles is its own kind of imprecision, and Section V-F now states the mechanism.

S1.4. The causality framing, withdrawn in this version

Through version 2 this paper described its first failure mode as a causality violation, on the argument that an acknowledgment causally precedes a receipt. That argument is wrong, and a reader pointed it out. The broker's append precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the record; those are two branches of one cause, and neither precedes the other. A negative difference between them is therefore possible without any physical impossibility, and is in fact routine when the producer's callback thread is descheduled.

We settled the question on the corpus rather than by argument. Every archived run that joins producer and consumer events was recounted, span by span (`scripts/recount_spans.py`, output committed as `docs/results/span_recount.csv`). Over $708{,}505$ events in $5{,}863$ runs, the acknowledgment-timed span is negative $62{,}264$ times and the publish-timed span is negative 0 times. Nothing in the corpus arrived before it was published.

What changed as a result: the framing, the justification of the gate, and the claim that the failure is self-detecting. What did not change: the rejection counts, which are unaltered as numbers and now described as reference-timestamp failures; and every manipulation result in the Section IV of the main text, which rests on moving scheduling and observing the rate move, not on what the negatives are called.

S1.5. The M/G/1 form, withdrawn

We adopted an M/G/1 waiting-time framing as a leading-order account of the stall, following its use for benchmark bias elsewhere, and pre-registered it as a hypothesis with a falsifier. Extended to the region where the candidate forms diverge ($\rho = 0.881$–0.990), the M/G/1 form fitted *worse than the mean*: $R^2 = -0.05$, against a fitted exponential's 0.93. Our own registered bracket also missed, predicting 2.45–$3.07\times$ growth against a measured $1.44\times$. Both candidates were parameterized in ρ, and the real-time-priority experiment shows ρ is not the variable. The Pollaczek–Khinchine result remains the motivation for expecting a heavy waiting tail; it is not a fitted result here.

S1.6. The traced tail-index cross-check, withdrawn

The traced tail index was re-estimated with an interval rather than read off a slope. Doing so refuted the cross-check it was meant to strengthen, and the claim is withdrawn; S8 gives the numbers and the argument. In summary: two proper estimators on the same histogram disagree sixfold because the survival is not a power law over the window we quoted, and the agreement we reported between the traced index and the payload-sweep exponent was a coincidence of window and estimator. The payload-sweep fit itself is unchanged and is now reported in S8 rather than in the main text, since four points do not earn an equation there. The mechanism results are untouched: none of them rests on the value of an exponent, only on the direction the rate moves when T_{true} moves, which is measured and monotone.

This is the third occasion on which a number in this work stood for months and failed the first time it was estimated with an interval attached. We record that pattern because it is the argument of the paper turned on its author: a statistic that carries no uncertainty is not yet a measurement.

S1.7. Pre-registration outcomes

Six predictions were registered with falsifiers before the comprehensive campaign. One was confirmed (the payload flip, Section V-D and Figure S5); three falsifiers fired; one was not evaluable once smeared configurations appeared, because the branch-count statistic it relied on has no value on a continuum; and one split. The jitter-kernel conjecture advanced after that campaign was killed by a subsequent alternation test. The plateau observation at 1053 and 1219 msg/s refuted the linear extrapolation that one registered prediction had assumed.

S1.8. The achieved publish rate was taken from a flag, not measured

For part of the campaign the achieved publish rate was recorded from the configuration flag rather than measured against elapsed wall time, and for a subset of runs the two disagreed. The affected conditions were re-derived from per-event timestamps and the ledger corrected; the incident is the reason Section VI asks for the achieved rate rather than the flag meant to produce it. It is also the reason Section II-A reports measured pacer jitter rather than a configured rate.

S1.9. A source we read wrong, and how we found out

Section VII-A of the main text places a published broker comparison inside the regime the deletion law binds on. An earlier draft of that placement attributed to it [S4] a Redis p99 of 0.8 ms against Kafka's 12.5 ms. Those figures are not on that page. Re-reading it on 2026-09-03 found only Kafka given a percentile — "sub-10ms latency at p99" — with Redis and NATS given "sub-millisecond" and no percentile at all, which is the sharper exhibit and the one the paper now uses. The figures we had attached to it belong to a different 2026 comparison [S5], which a search had returned in the same result set.

We record it here rather than in the reference note because it is the same class of error this paper is about, made in a citation rather than in a measurement: a number carried from one source to another because both arrived together, and nothing in the citation would have shown it. Both documents are now placed separately in S33.4, and the registry S33.4 counts from (`docs/results/external/literature_regime.csv`) holds one row each.

S1.10. The distributed-mode gap

The OpenMessaging Benchmark's distributed mode failed to complete a run on every attempt we made, across two release versions. The cross-host exposure of that particular tool therefore rests on reading its source together with our own two-host harness, not on running the tool itself cross-host. Diagnostics are in the upstream issue draft accompanying this artifact.

S1.11. The first result set

Table S1 is the first, rejected result set in full.

S1.12. The first corpus's prior corrections

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

TABLE S1
FIRST RESULT SET, LATER REJECTED. BROKER TRANSPORT (MEDIAN, MS) BY CONCURRENCY, WORKSTATION TESTBED, 10× REPLAY. EVERY CELL FAILS THE CHECK OF SECTION II-C.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

S1.13. The result

On the workstation testbed, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (Table S1). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

S1.14. The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
2) *A saturating replay speed.* At $120\times$ the producer could not keep pace, inflating publish delay to tens of seconds.
3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median event-time latency at one feed fell from $1{,}644$ ms to 16 ms once the Kafka producer was pipelined.
4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design of that corpus.
5) *A heavy tail contaminating every mean.* The 95th percentile of Kafka's event-time latency sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behavior; decomposition located it in publish delay, not transport.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median $\approx$170 bytes) and serialization costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgment experiment reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, left none of its conditions usable, and nothing in its output said so.

S1.15. Open questions from the comprehensive campaign

Three observations from the campaign are consistent with a single conjecture — a per-publish jitter kernel whose width in cell units grows with the numerator p of Δ/τ — and none of them is established. We separate them from the results above deliberately: they are post-hoc, the pacer they implicate is measured only in the non-JVM replication of Section V-F, and one of our own follow-up tests cuts against the conjecture.

Displacement toward θ. Replicates sit off their grid vertices toward the continuous value, from under a point to complete detachment. The two configurations that detached outright (a majority of replicates more than 3 points from every vertex) are 300 msg/s in the evening pass and 700 msg/s — both with $p = 10$; no configuration with $p \leq 8$ detached, and the same evening that 300 msg/s detached, 625 msg/s ($p = 8$) produced its cleanest two-branch split of the study. Suggestive; a census of two is not a law.

Mobility between passes, now measured under control. The same 300 msg/s configuration was branch-adjacent in the morning and detached in the evening; passes were hours apart, so epoch and configuration were confounded. The controlled test interleaves the two configurations A/B/A/B within one session — four pairs of (300, 625) msg/s — and **both configurations hold their grids**: 300 msg/s at 37.40, 38.21, 39.86 on the lower vertex and 67.41 on the upper; 625 msg/s at 42.97 and 55.91–60.18 (the `ultimate_alt` cells of the artifact). No configuration detached. The detachment is therefore a property of the *epoch*, not of the configuration, and the numerator census above loses its same-evening contrast — which cuts the kernel conjecture's strongest support and is reported accordingly. What remains established is that some sessions produce a detached

regime and others do not, at fixed configuration; the disciplined clock is exonerated throughout (residual 0.013 ppm), and the pacing loop's state remains the open suspect.

Branch suppression. The formal residual analysis finds mean retention *below* θ at every $q \geq 3$ configuration, and the rank correlation of |residual| with p is negative (-0.8), not the positive association a displacement-scaling kernel predicts — the sign is dominated by branch-weight sampling at $q = 1$. What the residuals do support is minority-branch suppression: a kernel narrower than the half-cell keeps runs on the nearest vertex, depressing the upper-branch weight below $\mathrm{frac}(q\theta)$, which would also account for the 1250 msg/s miss above. A quantitative displacement prediction from a measured jitter statistic does not yet exist; the harness of Section V-F records the statistic per run and is where one would be built.

S2. What the broker comparison rests on, and what it does not

What this claim rests on. We state the evidential basis first, because an earlier version of this section rested it on the wrong corpus. The load-bearing measurement is a powered replication at a replay rate that is *derived from the plan and verified against wall time before the campaign runs* — a documented setting, not an inferred one — over a median of 125 audit-surviving events per run. The original E1 corpus (Table S2) is reported afterwards, as the historical measurement it is, for two reasons that disqualify it from carrying the claim: its replay rate is an inference — recovered in S4 as true real time, but inferred from the runs rather than recorded — and it matched a *median of seven events per run*, the same seven-event opening burst as the publish delay we withdraw in S3. A sub-millisecond difference cannot be resolved on seven events, and E1 does not resolve one: its Hodges–Lehmann shifts wander (0.021, 0.116, 0.053 ms) and its $N{=}1$ estimators disagree. That E1 finds the two systems "equal" is as much a statement about seven events as about the brokers. What E1 does establish, and what the powered campaign confirms, is the *concurrency* result: Kruskal–Wallis finds no significant effect of concurrency in either backend ($p = 0.061$ for Kafka, $p = 0.091$ for Redis), and neither system degrades across the range.

The powered measurement: We measured transport directly, at a replay rate derived from the plan and verified against wall time, over a median of 125 events per run rather than seven, at $N \in \{1, 9, 12\}$ with 15 replicates each (Table S3). The numbers are computed over audit-surviving runs only — the campaign's own retention, the gate's effect on every estimate (at most 0.003 ms on the shift) and the worst-case imputation flip point are Table S12 in S18 — and the harness timestamps in microseconds on one clock, so neither failure mode of this paper touches the numbers that follow. The picture is sharper and it is not the one E1 painted. Broker transport is Kafka ≈ 0.54 ms against Redis ≈ 0.11 ms: a Hodges–Lehmann shift of 0.408 ms at one feed, Kafka slower, whose percentile-bootstrap 90% interval, 0.389–0.419 ms, lies far from zero, as it does at every N (Table S3). The two systems are *not* statistically indistinguishable, and the shift runs in the direction the gray-literature comparisons report.

And yet the same three estimators — Welch, percentile bootstrap and Hodges–Lehmann — agree that the difference is *equivalent within one millisecond* at every N (Welch TOST $p < 10^{-26}$ at each), because that shift is comfortably inside that margin. Both statements are true and neither is idle: against the seconds-scale human annotation latency that dominates any live sports feed's latency budget, a broker difference is parts in $100{,}000$ of the whole path, so for choosing a broker it is noise; but it is a real, tight, reproducible effect that a properly powered measurement resolves and a seven-event one cannot. The shift is also flat across concurrency (0.408, 0.416, 0.417 ms at $N = 1, 9, 12$, audit-gated; S18), so neither system degrades. A part of even this residual is the harness rather than the broker: Table S28 shows that the asymmetric acknowledgment timestamp accounts for 0.071 ms of the gap [95% bootstrap over runs: 0.039–0.113], and for 0.039 ms [same: 95% bootstrap over runs, 0.008–0.059] in the replication at 30 runs per cell — a different quantity from the inter-host clock offset of Section II-A, which happens to be the size of the first —, leaving a true broker-transport difference of 0.34–0.37 ms. That shrinkage is a concrete instance of the blind spot Section III-A names: it is a bias smaller than the quantity measured, so it never turns a value negative and the consistency check cannot see it. The check does reject 40 of the 160 runs behind these numbers, 39 of them Redis's, and the shrinkage is computed over all 160. The check guarantees a corpus is not *catastrophically* wrong; resolving a harness artifact of a few hundredths of a millisecond still took a controlled experiment, and sizing it took a replication.

An independent replication reproduces the shift and fails the criterion registered for it. A second powered campaign, run the same day about ten hours later with the same protocol and eight replicates per level, was registered with one criterion: its shift must fall inside the first campaign's 90% interval at every level. Over the runs the sign check keeps, as every number in this section is, it gives Hodges–Lehmann shifts of 0.381, 0.414 and 0.416 ms at $N = 1, 9, 12$. The last two fall inside; the first, computed from the two Redis runs the check kept at one feed, falls 0.008 ms below its interval. Over all runs the shifts are 0.397, 0.412 and 0.413 ms and the pattern reverses: the first falls inside and the other two fall below, by less than 0.001 ms. Either way the criterion failed by less than a hundredth of a millisecond, and either way the two campaigns' intervals overlap at every level. The criterion was the wrong one, which we saw only after the result: the second estimate carries its own sampling error, so even identical true shifts would put all three inside the first campaign's intervals only 38% of the time over the kept runs and 32% over all runs, under normal sampling at the two campaigns' own interval widths. The 0.41 ms advantage is not a fluctuation of one campaign; the test registered to show that could not have shown it reliably.

This refines E1 rather than contradicting it, and it sharpens the contradiction with the first-answer table (Table S1): the accelerated corpus reported Redis *degrading* with concurrency; the powered real-time measurement finds Redis uniformly *faster* and flat. The reversal is the audit's doing, and the measurement's.

TABLE S2

HISTORICAL CORPUS; THE PUBLISH DELAY COLUMNS ARE WITHDRAWN. EVENT-TIME LATENCY AND ITS DECOMPOSITION, CLOUD TESTBED, AFTER THE CONSISTENCY CHECK (164 OF 201 RUNS RETAINED). MEDIANS, MS. **THE EVENT-TIME LATENCY AND PUBLISH DELAY COLUMNS RECORD A PER-RUN START-UP COST, NOT A PER-EVENT PROPERTY, AND ARE RETAINED ONLY AS EVIDENCE FOR THE WITHDRAWAL IN S3; THEY SHOULD NOT BE READ AS SYSTEM BEHAVIOR.** THE TRANSPORT COLUMNS ARE COMPUTED OVER THE SAME SEVEN-EVENT OPENING BURST, AND SO MEASURE START-UP RATHER THAN STEADY STATE: SECTION S5 REPRODUCES THEM FROM THE POWERED RUNS BY RESTRICTING THOSE RUNS TO THE SAME WINDOW. FOR STEADY-STATE TRANSPORT SEE TABLE S3. THE REPLAY RATE WAS NOT RECORDED AND IS RECOVERED AS TRUE REAL TIME IN S4.

	Event-time latency		Publish delay		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

TABLE S3

POWERED TRANSPORT REPLICATION AT A VERIFIED TRUE-REAL-TIME RATE, OVER A MEDIAN OF 125 EVENTS PER RUN (NOT THE SEVEN OF TABLE S2), AUDIT-GATED (SECTION S18). MEDIANS IN MS; THE HODGES–LEHMANN SHIFT IS $Kafka - Redis$ WITH A PERCENTILE-BOOTSTRAP 90% INTERVAL. THE BROKERS ARE EQUIVALENT WITHIN 1 MS AT EVERY N (ALL THREE ESTIMATORS) YET CLEANLY DISTINGUISHABLE: REDIS IS ≈ 0.41 MS FASTER, AND THE SHIFT DOES NOT GROW WITH CONCURRENCY.

N	Events/run	Kafka	Redis	HL shift [90% CI]
1	148	0.512	0.102	$0.408\ [0.389, 0.419]$
9	112	0.542	0.120	$0.416\ [0.408, 0.423]$
12	121	0.540	0.114	$0.417\ [0.408, 0.424]$

Why E1 saw near-equality, tested directly: If the disagreement between E1 (< 0.1 ms apart) and the powered campaign (0.41 ms apart) is the seven-event window and nothing else, then restricting the powered runs to their first seven events must reproduce E1, and the full runs must reproduce the powered answer. Both hold: full-length shifts 0.381, 0.417, 0.414 ms against the powered 0.41; first-seven shifts 0.088–0.248 ms against E1's 0.021–0.116, with absolute medians matching as well. The window explains the disagreement with one parameter fixed in advance; the full test, including the start-up decomposition it rests on, is Section S5.

S3. THE TWENTYFOLD GAP IN EVENT-TIME LATENCY WAS PUBLISH DELAY, AND WE WITHDREW THE CLAIM

Event-time latency and transport give opposite answers in Table S2. Of Kafka's 105.5 ms, 102.9 ms is producer publish delay against Redis's 1.8 ms, while broker transport is 0.84 and 0.80 ms. An earlier version of this paper reported that twentyfold gap as a finding, attributed it to the client's sender thread waking from idle, and built a practitioner recommendation on it.

That gap does not survive re-measurement: it was a start-up cost paid once per run, read as a cost paid by every event. We report it at length, as we did the first result set, and we retain the figure that once carried it (Figure S2) rather than deleting the evidence.

The offset is a start-up cost, not a per-event constant: Re-measuring at $N{=}1$ on the same driver and the same broker, at a replay rate derived from the plan and verified against elapsed wall time, gives a median producer publish delay of 1.59 ms for Kafka, not 103 ms — with a *maximum* of 103.5 ms. Two independent instrumentation paths agree: the per-event loop trace and the per-run summary. The 103 ms is real, but it is paid once per run, not once per event.

We cannot call this a repetition of E1's condition, because no record of E1's achieved replay rate survives (S4). That is why the argument below does not rest on the comparison between the two campaigns at all. It rests on a sweep that is internally controlled — one campaign, one rate, one match, only the window varying — and on an arithmetic relation that holds at every rate E1 could have used.

A controlled test that separates the two: A median cannot distinguish a per-run cost from a per-event one, because how much of the cost the median sees depends on how many events the run contains — which is precisely what misled us. The quantity that *does* separate them is the *number* of affected events per run. A per-run cost predicts that number is fixed however long the run gets; a per-event constant predicts it grows with the run. We therefore replayed the same match at true real time under three observation windows, holding everything else fixed, and counted events waking more than 50 ms late (Table S4).

The count does not move (Figure S1). Going from the 60 s window to the 600 s window multiplies the events emitted by 8.9 — 57 to 507 — while the number waking more than 50 ms late stays at exactly four and exactly one publish call blocks, so the share of events paying the cost falls from 7.0% to 0.8%. The median is flat at 1.6 ms throughout and the maximum flat at 103.5 ms: the cost is present in every run and dilutes in every run, which is the signature of something paid once. Had it been a per-event constant, the count would have grown with the window and the median would have stayed at 103 ms.

A once-per-producer cost of this kind is documented upstream: a Kafka producer performs a metadata lookup before its first publish to a topic it has not yet cached [S6]. We did not isolate that call, so we do not claim it *is* the mechanism — the

window sweep constrains the cost's *shape*, not its origin. We note it because the shape we measure and the shape the client documents agree, and because a reader deciding whether this generalize beyond our testbed should know the behavior is a property of the client rather than of our rig.

Redis was run in the same sweep on the same events, with the same loop trace, and has *no* events waking more than 50 ms late and *no* blocking publish call at any window. Its worst single event anywhere is under 25 ms. Instrumenting both configurations identically matters here for a reason internal to this paper: an earlier version of this table reported Redis's counts as zero when its producer in fact carried no trace, which is the absence of a measurement dressed as one — exactly the failure the paper is about. We added the trace to the Redis producer and re-ran both configurations together.

The mechanism, read directly off the loop trace: The per-event trace shows the same five-event signature in every Kafka run, at every window:

event	wake late (ms)	publish blocked (ms)
0	0.17	**102.60**
1	**102.96**	0.07
2	**103.08**	0.05
3	**103.17**	0.04
4	**103.30**	0.05
5	0.94	0.27
6	1.42	0.06

The first `produce()` blocks for 102.6 ms fetching metadata and creating the topic. The replay loop is single-threaded, so every event due while it is blocked wakes roughly 103 ms late and then publishes in tens of microseconds. There are exactly four of them, and the reason is the data rather than the harness: this plan's first five events all carry $t_{\mathrm{sim}}{=}0$, two lineup records, two half-start markers and the kick-off pass. That is not particular to this match — every one of the eleven plans in our corpus opens with at least five events at $t_{\mathrm{sim}}{=}0$. A football feed therefore delivers its densest burst precisely when the producer is coldest, which is the worst possible alignment for a start-up cost and the reason it lands in the median rather than the tail. From event 5 the loop is in steady state at about 1 ms. One cost, paid once, is charged to five events; the sixth onward pays nothing. Redis shows no such prologue because `XADD` to a new stream creates it in the same round trip.

That closes it. Five events carry the cost, seven were matched, and the median of seven values of which at least four are 103 ms *is* 103 ms. The published figure is the arithmetic of its own prologue.

Why the original corpus could not see that, and how we know: The E1 runs behind Table S2 matched a **median of seven** events per run, 745 events across 100 Kafka runs. Their campaign script, recovered later (S4), replayed two seconds of match time per run, so the seven are the opening of the match and not a random seven.

The measurements show the same thing without the script, by arithmetic. Those runs report a median producer publish delay of 102.93 ms over seven values, and a median of seven values at 102.93 ms requires *at least four of the seven* to be 102.93 ms or greater. The loop trace says exactly how many events per run are ever that late: four (Table S4), and always the same four — those due while the first publish call blocks. The matched set must therefore consist almost entirely of the prologue. The plan's first five events are all timestamped at $t_{\mathrm{sim}}{=}0$, so any window that starts at kick-off opens with them.

The failure thus compounds: a sample of seven is small, but a sample of seven *selected towards the contaminated events* is worse than small. Redis reports 1.75 ms on the same events because opening a stream with `XADD` does not pay what Kafka's first publish call pays in metadata fetch and topic creation, so the same selection costs it nothing.

Why not knowing E1's replay rate does not matter here: The blocking first publish call lasts 102.6 ms of *wall* time, so at a replay rate of R times real time it spans $0.103R$ seconds of *match* time. Evaluating that against the plan for each rate E1 could have run at:

Replay rate	Match time spanned	Events inside the prologue
1×	0.10 s	5
120×	12.3 s	16
1200×	123.1 s	112

E1 matched a median of seven events per run. At every one of the three candidate rates the prologue already contains at least four events — enough to carry the median of seven — so the matched sample's median is the start-up cost, which is what the corpus reports. The conclusion is therefore insensitive to the replay rate; S4 goes on to recover that rate from the same start-up cost, but the withdrawal does not depend on the recovery.

This also explains the three properties we previously offered as evidence, and each of them is equally consistent with a start-up cost: a per-run constant looks *constant* within a run; it is *concurrency-invariant* because every run pays exactly one regardless of N; and it is *rate-dependent* because accelerated replay packs more events into the window and dilutes it. We read three signatures of a per-event mechanism into measurements that a per-run mechanism explains at least as well.

The general lesson: The consistency check of Section II-C does not catch this. Every one of those Kafka runs is causally consistent; nothing is negative; the four cells' medians agree within half a millisecond, 105.3 to 105.7 ms, across a hundred runs. What went wrong is that a median over seven events was reported as a property of the system rather than of the window, and the tight agreement across runs made it look robust precisely because the artifact is deterministic. A benchmark can be

TABLE S4

OBSERVATION-WINDOW SWEEP SEPARATES A PER-RUN COST FROM A PER-EVENT ONE. SAME MATCH, SAME DRIVER, SAME BROKER, $N{=}1$, TRUE REAL-TIME REPLAY; ONLY THE WINDOW CHANGES. BOTH PRODUCERS ARE LOOP-TRACED, SO THE COUNTS FOR THE TWO SYSTEMS COME FROM THE SAME TRACER. EVENTS EMITTED GROW $8.9\times$ ACROSS THE SWEEP, BUT THE NUMBER OF KAFKA EVENTS WAKING MORE THAN 50 MS LATE DOES NOT MOVE, AND EXACTLY ONE PUBLISH CALL BLOCKS IN EVERY RUN; REDIS HAS NEITHER AT ANY WINDOW. COUNTS ARE MEDIANS OVER THREE REPLICATES FROM THE PER-EVENT TRACE; PERCENTILES COME FROM THE PER-RUN SUMMARY.

	Window	Events		Publish delay (ms)		Events >50	Blocking
	(s)	emitted	matched	median	max	ms late	publishes
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

TABLE S5

WHY THE DELAY SWEEP IS NOT AN EFFECT-SIZE MANIPULATION. INJECTED ONE-WAY DELAY AT THE BROKER, VERIFIED PRESENT AT THE INTERFACE, AT A VERIFIED TRUE REAL-TIME RATE. EVENT-TIME LATENCY TRACKS THE INJECTION; BROKER TRANSPORT DOES NOT MOVE, BECAUSE THE SAME DELAY IS ADDED TO BOTH ARRIVALS WHOSE DIFFERENCE DEFINES IT.

Injected delay	Event-time latency median	Transport median	Transport IQR
0 ms	3.72 ms	0.535 ms	0.162 ms
1 ms	4.67 ms	0.508 ms	0.166 ms
5 ms	8.66 ms	0.494 ms	0.159 ms
20 ms	23.61 ms	0.480 ms	0.150 ms

internally consistent, gate-clean, and still measure its own start-up, and a percentile over single-digit samples is a statement about the harness. The rule this episode produced, to count the events before quoting a percentile, stood in the paper until 29 September; Section VI keeps its substance as the request to print the retention and the discards, counted by sign, beside every latency.

What we claim instead: On broker transport — the quantity that survived the audit and is measured over the same events for both systems — Kafka and Redis remain equivalent within one millisecond (Section VI-D). From E1 we make no event-time claim; the event-time equivalence of S13.1 comes from the powered campaign, over a median of 125 events per run. The twentyfold figure is withdrawn.

The benchmark's own output corroborates the insensitivity: The benchmark's own output corroborates this without our instrumentation: across eight runs all 32 reported percentiles are whole milliseconds (only the averages, means of integers, are fractional), and three runs report $\mathrm{p50} = \mathrm{p95} = \mathrm{p99} = \max = 1.0$ — not a narrow distribution but one with a single value in it, printed to three decimal places.

S3.1. The delay sweep failed its manipulation check

We withdraw the intermediate points, and the reason is worth more than they were. An earlier version supported H1 additionally with a `netem` delay sweep, reporting a rank correlation across injected delays of 0 to 50 ms. We then re-ran that sweep at a verified real-time rate with a manipulation check, and the check failed — not because the treatment was too weak, but because it does not act on the quantity at all. Injecting delay at the broker moves event-time latency almost exactly as commanded ($3.72 \to 23.61$ ms across $0 \to 20$ ms) while broker transport stays flat ($0.535 \to 0.482$ ms; Table S5).

The cause is that the delay is **common-mode**. It delays the acknowledgment travelling broker-to-producer and the record travelling broker-to-consumer by the same amount, and transport is the *difference* of timestamps taken at those two arrivals, so the injected delay cancels exactly. A manipulation can be applied correctly, verified present at the interface, and still not touch the quantity under study, and this time the failure was in our own experimental design rather than in our harness. The original sweep's correlation was therefore never an effect-size relationship, and we report the sweep only as the negative result it is.

S4. OUR OWN REPLAY-RATE RECORD DID NOT SURVIVE THE AUDIT

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. This section is the full record, kept because the protocol rule it produced is weaker without it.

Replay plans built from StatsBomb carry a **baked-in** $120\times$ **time compression**: a plan's publish offset is already the match time divided by 120. The producer's `--speedup` flag then multiplies that. So `--speedup 1` against such a plan is *not* real time, it is $120\times$; true real time needs $1/120$. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a

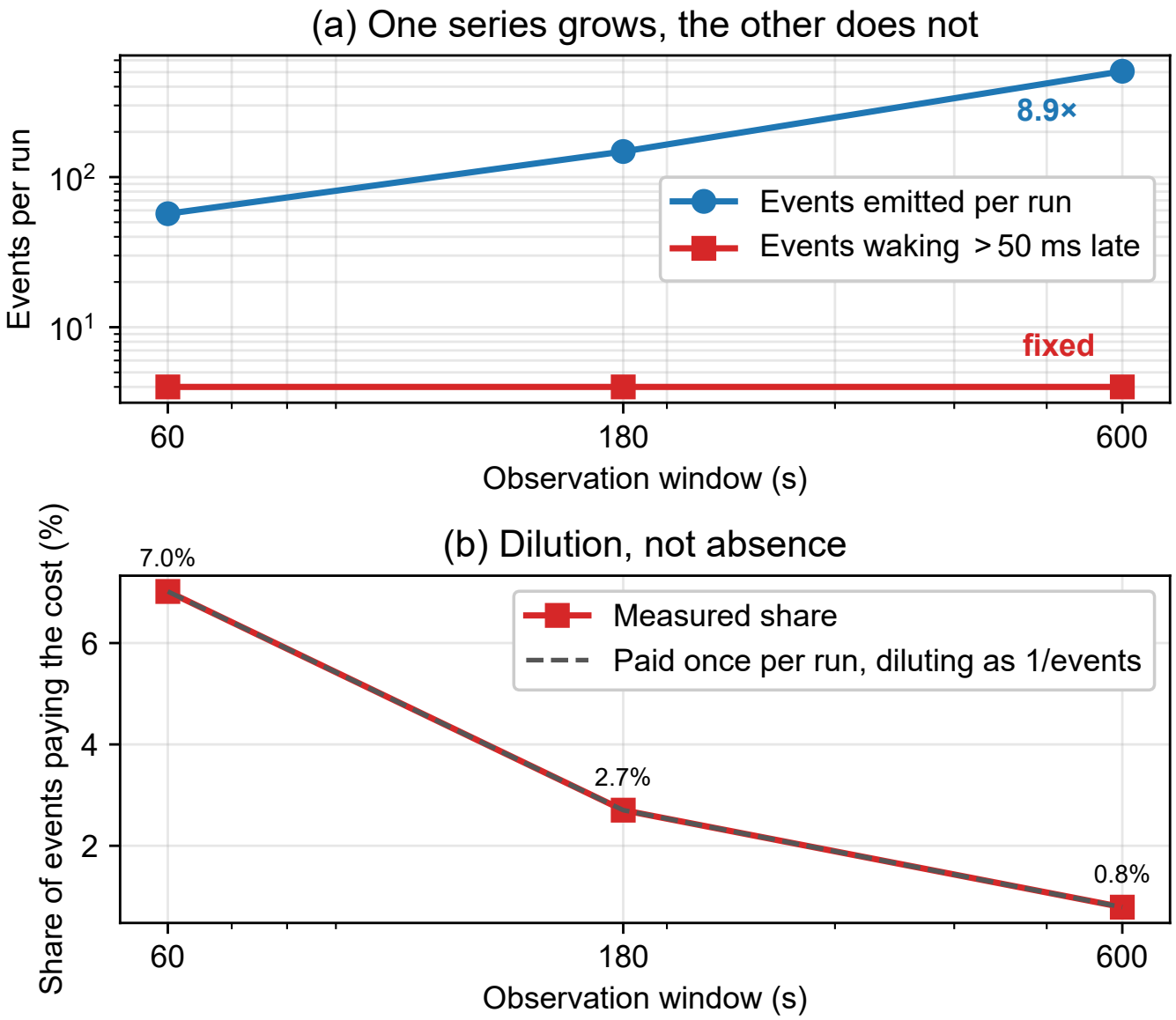


Fig. S1. **Once-per-run cost, diluted by the window.** (a) Events emitted per run grow 8.9× while the number waking more than 50 ms late stays at four — a per-event constant would put the two series on parallel lines. (b) The same fact as a proportion: the share of events paying the cost falls as $1/\text{events}$, the dilution a once-per-run cost predicts. Redis, on the same events, has no affected events at any window.

campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artifact records the achieved replay rate of any run we report.** Several superseded workstation-testbed scenarios record the nominal `--speedup` *flag*, which is not the same thing: converting a flag to a rate requires the plan's compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote `speedup` into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus (S2) the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `--speedup 10`, and the flag's meaning was corrected 21 hours later the same day. E1 was replayed at 1×, 120× or 1200×, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata: Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of Section S3 is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly

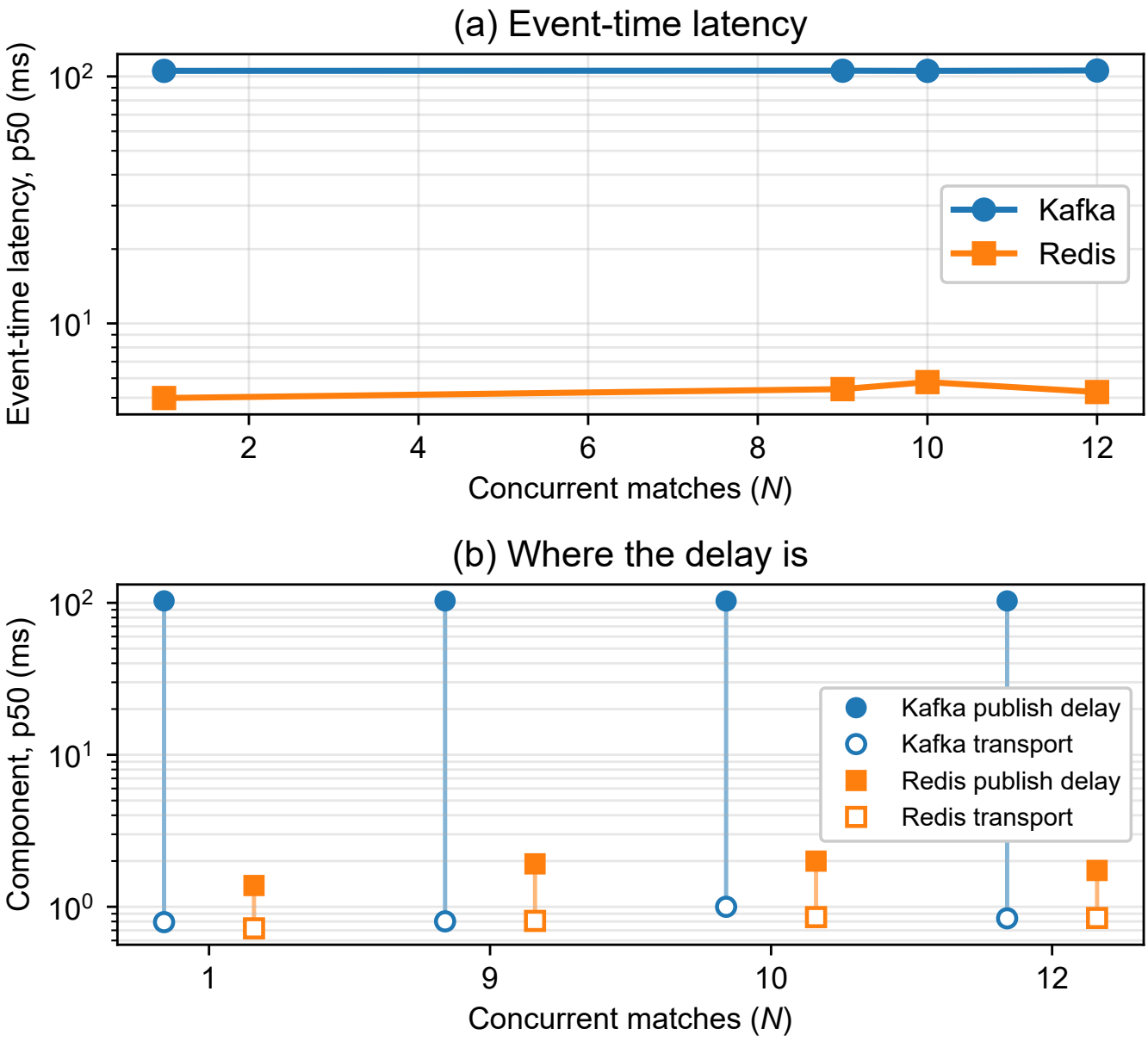


Fig. S2. **Withdrawn: a per-run start-up cost, read as a per-event gap.** (a) Event-time latency against concurrency as originally measured. (b) The decomposition that appeared to place the whole gap in producer publish delay. Both panels are retained as evidence for the withdrawal: the runs behind them matched a median of seven events each, so the apparent per-event offset is a per-run start-up cost. Broker transport, the lower series in (b), is unaffected and is what the paper claims.

eight events, and their median publish delay is $\mathbf{52.34}$ ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms against 52.34 observed. Four is also exactly what the loop trace shows at a verified real-time rate (Table S4), and it is what the plan predicts, since five events share $t_{\mathrm{sim}}{=}0$ and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At $120\times$ the prologue spans 12.3 s of match time and holds 16 events; at $1200\times$, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

The inference has since been checked against the original script, and it holds. Archiving the cloud driver before releasing it turned up the campaign as actually invoked, in a shell script in the operator's home directory that no part of the repository had ever referenced. It specifies `--speedup 0.008333` — $1/120$, against a plan already compressed $120\times$, which is true

TABLE S6
ONE SET OF RUNS REPRODUCES BOTH PUBLISHED ANSWERS. TRANSPORT MEDIANS (MS) FROM THE POWERED CAMPAIGN, COMPUTED OVER EVERY EVENT AND OVER EACH RUN'S FIRST SEVEN EVENTS IN PUBLISH ORDER — THE WINDOW E1 SAW. THE ALL-EVENTS COLUMNS REPRODUCE THE POWERED 0.41 MS SHIFT; THE PROLOGUE COLUMNS REPRODUCE E1'S NEAR-EQUALITY (TABLE S2) IN BOTH SHIFT AND ABSOLUTE LEVEL. THE WINDOW, NOT THE BROKERS, ACCOUNTS FOR THE DISAGREEMENT.

		All events			First 7 (prologue)		
N	Runs	Kafka	Redis	Shift	Kafka	Redis	Shift
1	5	0.476	0.095	0.381	0.831	0.743	0.088
9	45	0.533	0.116	0.417	0.939	0.810	0.129
12	55	0.527	0.113	0.414	1.018	0.769	0.248

real time — and its log's first line, `E1 N=1 reps=8 00:39:40`, names the run directory the E1 artifact's first row still points at. The arithmetic above and the flag agree.

We report that as a check rather than as the answer, because the order matters. The rate was determined from the measurements while the metadata was genuinely unavailable, and the script surfaced afterwards, by luck, from a machine that was about to be switched off. What the agreement establishes is not that E1's provenance was fine all along — it was not — but that the recovery method returns the right answer when there is an answer to check it against. That is the one thing a physical-constraint argument can rarely demonstrate about itself.

The same script also explains a feature we had described and not accounted for. It passes `--max-t-sim 2`: two seconds of match time per run. The median of seven matched events that makes E1 under-powered, and that Section S2 treats as an opening burst, is that window. The burst is real and is why those seven events are the densest in the match; the reason there were only seven is that the campaign asked for two seconds.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artifact we were chasing happened to be a wall-clock constant with a sharp signature, and the confirmation only because a virtual machine had not yet been destroyed; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from and nothing left to check it against. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched: The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal of S3 is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of Section VI of the main text were measured after the pre-flight existed, at a derived and verified rate, and so was the broker comparison of Section VI-D, which rests on the powered campaigns and not on E1. E1's own external validity is restored by the recovery above, and the recovered script now documents it as well — though the inference stood on its own before the script appeared, and is what we would still be relying on had the machine been released a day earlier. Section VI of the main text carries the rule that would have made all of this unnecessary.

S4.1. The co-location campaign we withhold

A manipulation that did not manipulate, and what it cost: We also withhold a co-location campaign (E-A8). The intent was to shorten T_{true} by removing the network hop — the opposite of the payload sweep, and a useful check that the T_{true} result is not an artifact of how we lengthened it. It did not work. Moving the broker onto the driver made transport *longer* at idle, $0.512 \rightarrow 0.573$ ms, because it traded network latency for CPU contention on the same cores; under 88% load it moved transport by $1.03\times$, which is no manipulation at all. The negative-span rates duly failed to separate (intervals overlapping in both cells). Reporting those rates as a null would misdescribe the experiment: nothing was manipulated, so nothing was tested. We record it because the pattern is itself informative. Two of our three attempts on this axis failed for structurally similar reasons — netem delayed both paths and canceled in the difference, co-location traded one delay for another — which suggests T_{true} is a quantity that resists manipulation in a shared-resource system, because every route to shortening it lengthens something else. The payload sweep worked because padding adds serialization without contending for the scheduler, and we were fortunate that it does.

S5. E1'S SHIFT IS CONFINED TO THE FIRST SEVEN EVENTS

Moved from Section VI-D of the main text: the complete windowed re-analysis showing E1's near-equality is the observation window and nothing else.

Why E1 saw near-equality, tested directly: The two campaigns disagree by more than power: E1 puts both systems near 0.8 ms and within 0.1 ms of each other, while the powered campaign puts them at 0.53 and 0.11 ms and 0.41 ms apart. Attributing that to seven events is an explanation only if it survives being checked, so we checked it. If the difference is the observation window and nothing else, then restricting the *powered* runs to their first seven events — in publish order, the same

events E1 would have caught — must reproduce E1, and the same runs must give the powered answer over their full length. That is a prediction with one free parameter fixed in advance, and it can fail.

It does not fail (Table S6). Over all events the runs give shifts of 0.381, 0.417 and 0.414 ms, reproducing the powered 0.41 ms. Over their first seven they give 0.088, 0.129 and 0.248 ms — E1's wandering near-equality, and of the same magnitude as E1's own 0.021, 0.116 and 0.053 ms. The *absolute* medians match as well, which the shift alone does not force: the prologue puts Kafka at 0.83–1.02 ms and Redis at 0.74–0.81 ms, against E1's 0.79–1.00 and 0.72–0.86. One set of runs, one rate, one protocol, two windows, both published answers.

So E1 is not wrong and the campaigns do not contradict each other. E1 measured the opening burst, during which both systems are slow for reasons this section sets out — topic auto-creation, connection setup, and the first blocking `produce()` — and that shared start-up cost swamps a 0.41 ms difference. As the run lengthens the cost dilutes, and the brokers' steady-state separation emerges. The seven-event corpus answered a question about start-up while appearing to answer one about brokers, which is the same failure of window selection as the withdrawal of Section S3.

S5.1. Where the knee sweep placed the load conditions

Moved from Section IV-B: the load-axis post-mortem in full. Section IV keeps the admissions and the numbers.

Both load-axis predictions failed, including ours: The knee sweep, named for the *knee*, the load near saturation beyond which the negative-span rate climbs steeply, placed conditions at achieved ρ of 0.881, 0.920, 0.950, 0.970 and 0.990, where the candidate forms diverge most; the earlier ladder stopped at 0.878 and everything above it collapsed onto a degenerate $\rho = 1.000$, which is why Section VI of the main text could not settle the question. With those points in hand, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93). The withdrawal does not merely stand: the forms are now separated, and M/G/1 is the one ruled out.

The bounded prediction fared better but also failed. Over $\rho = 0.881$ to 0.990 the rate grew by a factor of 1.44. M/G/1 predicted 13.8; the bracket we registered before the sweep predicted 2.45–3.07. The rate saturates, as a bounded mechanism requires — but harder than our own extrapolation allowed, and a prediction that misses low is still a prediction that missed. We report it as a failure rather than as the nearer of two misses.

The two failures have one cause, and E-A5 names it. Our bracket was still parameterized in ρ, through $p = \rho^C$; E-A5 shows ρ does not determine waiting probability. What survives is the mechanism — established by manipulation — and not any curve in ρ, ours included.

S6. THE EVEN-DENOMINATOR EXCLUSION WAS WRONG, AND WHAT REPLACED IT

We report the correction because the first version was weaker and we published it to ourselves: Treating $100/q$ as a point prediction made the even denominators look like failures, and we introduced an exclusion for them — an configuration cannot test a grid the continuous value sits on, so $q = 2$ and $q = 4$ were set aside. That exclusion was defensible and pre-registered: it was derived from the $q = 4$ result and committed at 14:09Z on 2026-07-27, and the first cell of the first odd-q configuration was measured at 15:06Z, so the odd denominators were run *because* the rule said they would separate. It was also unnecessary. Under the phase account of Section V-D an on-grid configuration is a prediction of a flat configuration, so $q = 2$ and $q = 4$ are evidence *for* the account rather than cases to be excused. We had reached for a law that excluded its exceptions when one that explains them was available.

What exposed it was checking whether the exclusion survived re-estimating one of its own inputs. The continuous value is not constant across rate — it runs 51.2% at 333 msg/s down to 47.5% at 889, a trend of 4.15 points against a replicate noise of 1.55 — and under a rate-local estimate $q = 4$ ceased to be degenerate and became a failure. A classification that flips on that choice is not a classification. Measuring position against the cell half-width instead of a fixed noise floor removes the sensitivity: every call in S11 is the same under either estimate.

A second prediction of ours failed, and no reading of it rescues the number: We recorded, before running it, that $q = 5$ would put 2 of 5 phases across a boundary and so retain about 40%. The ten replicates gave a median of 51.04% — inside the incommensurate range and 11.0 points from what we wrote down. The error was again naming one branch of a two-branch prediction: with T_{true}/τ at a half, 2 *or* 3 phases cross depending on where a run's initial phase falls, so a replicate is predicted at 40% or 60% and the average of many is near 50. The values — 40.71, 41.07, 41.96, 42.11, 46.58, 55.50, 57.02, 58.60, 59.05 and 60.04 — are bimodal about exactly those two branches, five on each, which is what the account requires and what no continuous account produces. The miss grew when the configuration did: at five replicates the median stood 6.6 points from the registered value and at 10 it stands 11.0, because the upper branch was under-sampled in the smaller set. The grid held; the point we picked on it did not.

Where the account is approximate, stated rather than trimmed: Two observations sit outside it, and a third did until the configuration was filled. One replicate at 500 msg/s sits at 18.0, which a run confined to a single phase cannot produce. We read this as the limit of the idealization: a nominal rate is $q = 1$ only if the pacing is exact for the whole run, and over three minutes a drift of a few parts per million walks the phase across a boundary part-way through. The grid that governs retention is the one the pacing realizes, not the one the nominal rate implies. Second, $q = 2$'s spread of 17.61 rests on a single replicate

at 68.03; the other four span 3.10 points, which is the incommensurate noise level and exactly what an on-grid configuration should show. We report the spread over all five rather than the four that agree.

The one that resolved is worth recording, because it is an argument for one of our own rules. The $q = 1$ configuration at 250 msg/s was listed here as a departure from the account when it had three replicates and spread little more than half its cell. At five it spreads 98.73 of 100, which is the full step width the model predicts, and the anomaly is gone. It was never a departure; it was an configuration reported before it had enough replicates to show what it was doing — which is exactly what Section VI asks benchmark authors to count before they quote.

PART II. THE MECHANISM, AND THE LAW IT PRODUCES

This part is the argument. Sections S7 to S9 establish the scheduling failure by manipulation: priority and geometry each moving the rate on their own, what the payload sweep does and does not support, and the two-state model with its tracer checks. Sections S10 to S13 establish the deletion failure — retention on the grid, membership configuration by configuration, every real-time-priority pair with the exposure curve they make, and what the brokers actually do. Sections S14 to S17 derive both: the statistics of a discarded zero, the clocksource bounded from a measurement, why the acknowledgment timestamp is late, and the 1970 counter note behind the grid geometry.

S7. PRIORITY AND GEOMETRY EACH MOVE THE RATE ON THEIR OWN

The experiment that settles it: Move one of the two and hold the other fixed: raising the timestamping processes to real-time priority makes them preempt the background load rather than queue behind it, so waiting probability falls sharply while utilization does not move — and unlike our delay injection, which canceled in the subtraction (S3.1), priority acts on the timestamping threads themselves, where the model locates the failure. A waiting-probability mechanism predicts collapse toward the running-state floor; any account in which the rate is a function of ρ predicts no change, because ρ is unchanged by construction. The result is unambiguous (Section S26): utilization held to 0.001 (the pre-registered manipulation check passes), and the negative-span rate fell from 0.132 to 0.0034 at $\rho = 0.75$ and from 0.305 to 0.0057 at $\rho = 0.88$ — factors of 39 and 54, disjoint Wilson intervals, both residual rates on the floor the model names in advance, $C_0 \approx 0.004$.

Utilization is not the variable, shown at identical utilization: Two loads can reach the same ρ by different means: k cores flat out leaves $C - k$ genuinely free, while duty-cycling every core leaves none. If the mechanism is whether the timestamping thread can find a core, the second should be worse at equal ρ — and it is (Section S26): at $k = 6$ both configurations sit at $\rho = 0.7531$, identical to four decimals, same machine, same hour, and the negative-span rates differ by $2.07\times$ ($z = 10.3$ over 2 985 matched events per cell, disjoint Wilson intervals). A function of ρ returns one value for one input. The gap narrows at $k = 7$ ($1.06\times$) — the mechanism showing itself: one free core out of eight is nearly as useless as none. *A full replication (E-A6b) corrects us on one point*: it reproduces the load-bearing cell almost exactly ($\rho = 0.7531$ in all four configurations; $2.02\times$ against $2.07\times$), reproduces $k = 5$ in direction but not size, and at $k = 7$ resolves a small difference ($1.17\times$, $z = 3.20$) where the original could not — so we withdraw an earlier draft's reading of that convergence as a null. Both campaigns agree on the shape: twofold at $k = 6$, near-parity at $k = 7$.

The other side of the inequality, and it moves the rate the wrong way: The mechanism predicts that a *slower* path is a *more reliable* measurement, because the same stalls have a longer interval to outrun. Padding the payload lengthens T_{true} without touching the scheduler (Section S26): transport rose $74.3\times$ while the negative-span rate *fell* $3.8\times$ [Katz 95%: 3.3–4.5], monotonically, with utilization held to a 0.012 spread — and pushed slightly *up* by serialization, against the observed fall, so the effect is measured against its own confound. The same numbers constrain the stall distribution sharply: a $74\times$ larger threshold bought only a $3.8\times$ lower rate, so the tail barely thins across two orders of magnitude — which is why mean-based counters could not see the effect and the direct trace was necessary.

S7.1. The overnight campaign, prediction by prediction

Section V-D reports the outcome in a sentence. This is the tally, because a pre-registration is worth what its failures are worth and three of these failed.

Sixty-three cells ran at $n \geq 5$ with six predictions and their falsifiers registered in advance. Every cell completed validly. One prediction was confirmed, three falsifiers fired, one was not evaluable and one split.

Duration invariance, which killed the drift account: If retention were set by an accumulating drift, longer runs would sit further from the grid than shorter ones. Intermediate counts over one, three and ten minutes are 1, 1 and 0, with the vertices pinned throughout. Nothing accumulates. What remains is per-publish jitter, which is the mechanism the retention law of Section V assumes and not one we chose after seeing the data.

The linear extrapolation that failed: One registered prediction assumed retention would extrapolate linearly in rate across the incommensurate configurations. At 1053 and 1219 msg/s the measured values are 47.2 and 48.2% — flat where the prediction wanted a slope. The grid law does not predict a trend in rate, only a position against the tick, so the failure is of the extrapolation rather than of the law; we record it because it was registered and it did not come true.

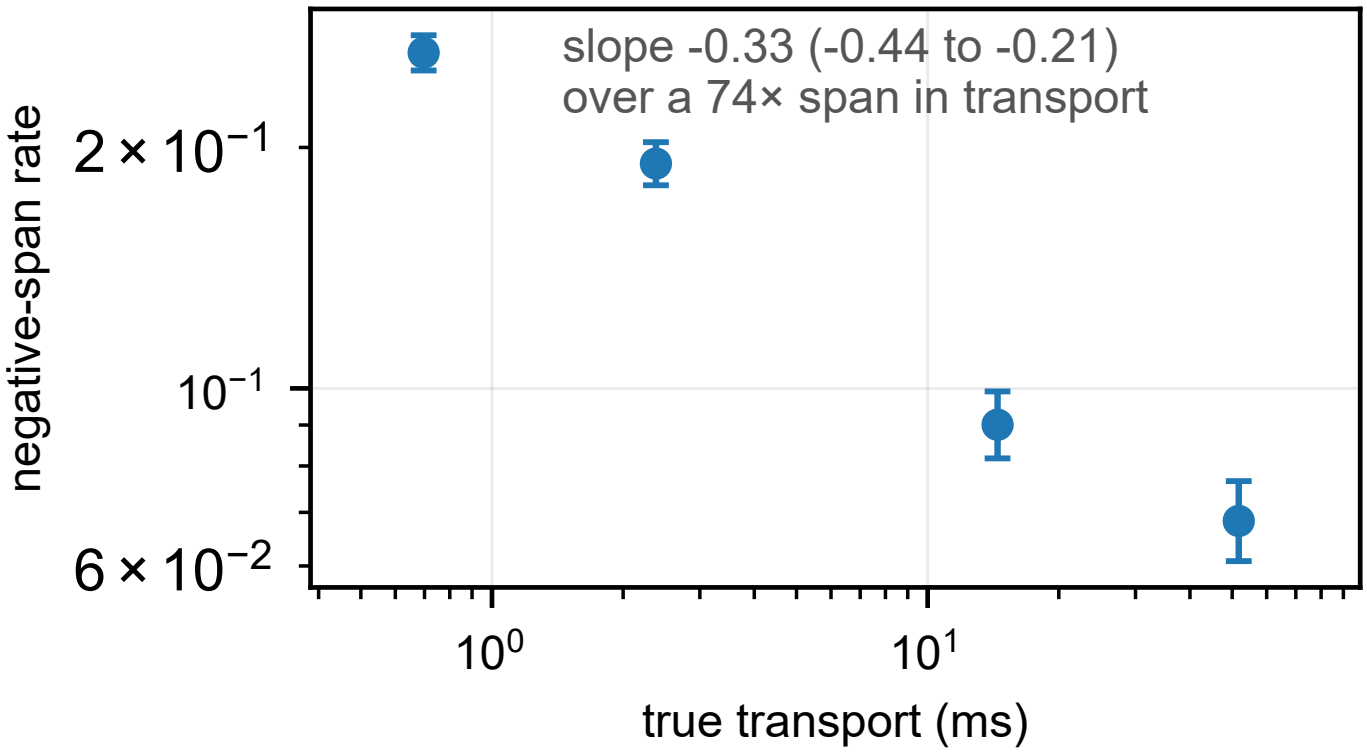


Fig. S3. **Longer end-to-end latencies invert less often.** Negative span rate against true transport over the payload sweep, Wilson 95% intervals. The same stall distribution overlaps a short end-to-end latency almost entirely and a long one hardly at all, which is the latency factor $G(T_{\mathrm{true}})$ of Equation S2 as an experiment: lengthening what is being measured lowers the rate, where a load-driven account predicts no change. The figure sits here rather than in the main text for the same reason Equation S1 above does — it draws four points.

S8. What the payload sweep fits, and what it does not support

The payload-sweep fit, demoted here from the main text: Over a $74\times$ payload span, least squares of $\log \Pr$ on $\log T_{\mathrm{true}}$ over 4 payload levels gives

$$\Pr[\text{span} < 0] = 0.238\ T_{\mathrm{true}}^{-0.328}, \qquad R^2 = 0.987, \tag{S1}$$

with an interval on the exponent of (95%, Student-t: 0.213–0.443) at 2 degrees of freedom. E-A10b repeats the sweep on a different day and returns 0.333 against 0.328, with a prefactor of 0.242 against 0.238. Four points do not earn an equation in a main text, which is why this one is here, and Fig. S3 plots them.

What we withdraw about it, and why: Two claims previously made around Equation S1 do not survive, and both were withdrawn in the round-2 internal revision.

The first is the infinite-moment reading — "the exponent is below one, so the stall distribution has neither a finite mean nor a finite variance". A slope fitted to four application-level points over one decade of T_{true} does not license a statement about the moments of the underlying stall distribution, which is a different object measured over a different range. What we should have written, and now do, is the weaker and sufficient claim: over the decade that matters the survival declines so slowly that a mean-based counter is a poor measure. That is enough to explain the observation it was invoked for — cumulative scheduler counters moving by factors of two to five against a fortyfold change in the negative-span rate — without asserting that a moment fails to exist.

The second is the traced cross-check. We reported that the per-wakeup survival's windowed log–log index, 0.332 over 0.25–2 ms, was "indistinguishable" from the fitted 0.328 and therefore confirmed it independently. It is not, and it does not. Estimated properly on the same histogram of 551,956 wakeups (`scripts/tail_index_traced.py`), the exceedance index over 0.5–2 ms is 0.21 (95% Wilson on the nested counts: 0.20–0.21) while grouped maximum likelihood over the same decade returns 1.19 (95% profile likelihood: 1.18–1.20). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and it is: the per-octave indices over that window run 1.96 and then 0.03, where a power law would give the same value twice. A bootstrap goodness-of-fit drawn from the fitted model itself agrees: $p < 0.0004$ over 2,500 replicates. The 0.332 was ordinary least squares through four nested survival points, which returns a number whether or not the points lie on a line. The agreement with 0.328 was a coincidence of window and estimator, and we should not have offered it as a confirmation.

The prediction check against the trace is unaffected and stands: the two-state model predicts $\Pr[\text{stall} > 0.5\,\text{ms}] = 0.298$ against the traced 0.181, over-predicting by $1.65\times$ ($1.69\times$ on the replication), the expected direction since not every stall lands

on a timestamping instant. That comparison is between two probabilities, not between two fitted exponents, and it needs no power law to be meaningful.

What the histogram says instead, which is better: Asking what shape the data do have, rather than which index to quote, gives the mechanism its most direct evidence. The bucket counts are not monotone, so no power law can describe them: there are 3 local maxima. The first is the jitter core of a running thread; the second a broad hump in the hundreds of microseconds; the third is a *mode* at 2–4 ms carrying 10.5% of all wakeups and standing $4.5\times$ above its lower neighbor. Beyond it the counts collapse rather than trail: they fall $5.0\times$, $3.4\times$ and $4.7\times$ over the next three octaves and then $357\times$, where a power law falls by one factor every time.

The second index, withdrawn, and why it survived so long: We had ended that paragraph with a number: grouped maximum likelihood above 4 ms gives 2.04 (95% profile likelihood: 2.00–2.07), quoted with the words "a finite variance". Three things were wrong with it and only the third is interesting. The moment claim needs an index above two and the interval's lower limit *is* two, so the data do not separate a finite variance from an infinite one. The last populated bucket falls $357\times$ where the fit implies about four, which is a truncation and not a tail. And the bootstrap goodness-of-fit rejects the fit outright: $p < 0.0004$ over $2{,}500$ replicates, the same verdict, from the same estimator, that we use six lines earlier to withdraw the payload-sweep index.

The interesting part is why it survived. `estimate()` in `tail_index_traced.py` ran `gof_pvalue` on the 256–$2048\,\mu$s window and on nothing else, so the test that condemned one index was never pointed at the other. Every gate we own passed the sentence, because none of them asks whether a published fit was judged by the test the same file already contains. A check applied to one result and not to another says nothing about the other, and the repair is the one the paper recommends to others: run the check on everything it can judge, and report what it says. The estimator now returns the tail's verdict whatever it is, the emitter carries it into the ledger beside the index it judges, and `test_the_paper_meets_its_own_rules.py` refuses a moment claim whose interval crosses the boundary it depends on.

The mode's position is not arbitrary. On this machine — one host, 8 online CPUs, recorded as "8 cores" in the campaign log itself and recovered independently from the geometry conditions' own k/ρ rather than from a shape description – the EEVDF base slice is $0.75\,\mathrm{ms} \times (1 + \lfloor \log_2 8 \rfloor) = 3$ ms; run-to-parity holds a woken thread off the CPU until the incumbent has exhausted that slice unless it wins the wakeup-preemption check, and with the high-resolution tick disabled the expiry is observed at the ordinary tick. A timestamping thread woken at the wrong moment therefore reads its clock late by what is left of that slice, anything up to a whole one (S16.10). That is the second state of the two-state model of Section IV-B, observed directly, at the constant the scheduler is configured with — which is a stronger statement than any tail index would have been, and it is what replaced the withdrawn one. The constants should be read off the image under test (`base_slice_ns` and `CONFIG_HZ`) rather than assumed from the kernel version, and we report them as the explanation the mode's location supports rather than as an independently measured cause.

The confirmation is the one that manipulates the physics: Payload moves T_{true}, so it moves θ's grid position, and the phase account of Section V-D dictates the configuration's class with nothing else changed: measured $\theta(32\,\mathrm{KB}) = 0.685$ and $\theta(64\,\mathrm{KB}) = 0.853$ give, at $q = 3$, $\mathrm{frac}(q\theta) = 0.055$ (on the $2/3$ point: flat, pinned there) and 0.56 (mid-step: full width back). The registered prediction was that one manipulation would move the configuration onto a grid point and off it. It did (Figure S5): at 32 KB the five replicates spanned 62.06–75.69% — spread 13.6 points, three replicates agreeing to 0.06 — and at 64 KB they spanned 73.06–99.78% — spread 26.7, the upper vertex hit to a quarter point. The flat-full boundary at half the step width (16.7 points) is crossed in the predicted direction by both halves. One registered detail failed: the 32 KB configuration pins at 69.2%, not on the vertex at 66.67% — a displacement toward θ that the next paragraph makes systematic.

The other side of the inequality, and it moves the rate the wrong way: A *slower* path should be a *more reliable* measurement, and it is: payload padding lengthened transport $74.3\times$ while the negative-span rate *fell* $3.8\times$, monotonically, utilization held to a 0.012 spread and pushed slightly *up* against the observed fall (Section S26). A $74\times$ larger threshold buying only a $3.8\times$ lower rate means the tail barely thins across two orders of magnitude — why mean-based counters could not see the effect and the direct trace was necessary.

Closing the loop: measuring the stall distribution directly: Every result so far infers the scheduler's behavior from the negative-span rate it produces. The mechanism says something stronger and directly checkable — that the negative-span rate should equal the probability that a run-queue stall outlasts the interval being measured — so we measured that probability without reference to the negative spans at all. Using `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints, we recorded the delay between a timestamping thread becoming runnable and reaching a core, for every such transition in the run, and computed $\Pr[\mathrm{stall} > 0.5\,\mathrm{ms}]$ against the measured transport of that cell (Section S26).

The two quantities share no probe, no estimator and no data: one is a count of negative transports in application CSVs, the other a kernel histogram of scheduler transitions. Across three configurations at two load levels they agree to within a fifth — ratios 0.84, 1.01 and 1.17 against the rate of both brokers' runs, whose threads the trace counts (Section S26), nothing fitted between them. The residual has no consistent sign, so what one configuration had suggested as a directional effect was scatter; a traced scheduler quantity predicts an application-level failure rate to within a fifth, unfitted, and no bias is resolvable inside that.

The two-state reading accounts for the clustering (a preemption is an interval, so the events inside it are corrupted together) and predicts tails that shift vertically rather than rescale horizontally — measured close to parallel: median log-ratio spread 0.29, worst 0.91, set against the $23\times$ failure of the scale-family collapse. We found this by looking at the data — the separability test is exploratory — so we pre-registered a confirmatory version and ran it: the campaigns below. And read as a function of load alone the model is a tautology, with no content until it is pinned on the threshold axis, where it passed: the form that restores the divided-out σ reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 (power law) at equal parameters, but freezing σ at its mean improves the fit to $R^2 = 0.9982$, so σ and p move together on this ladder and neither can be credited over the other; the commentary in full is Section S9.

The experiment that settles it: Raise the timestamping processes to real-time priority: waiting probability falls sharply while utilization does not move, and priority acts on the timestamping threads themselves — where the model locates the failure — rather than canceling in the subtraction as our delay injection did. The result is unambiguous (Section S26): utilization held to 0.001, and the negative-span rate fell by factors of 39 and 54 at the two load levels, with disjoint Wilson intervals, both residual rates landing on the floor the model names in advance, $C_0 \approx 0.004$.

Utilization is not the variable, shown at identical utilization: Two loads can reach the same ρ with $C - k$ cores genuinely free or with none; at $k = 6$ both configurations sit at $\rho = 0.7531$, identical to four decimals, and the negative-span rates differ by $2.07\times$ ($z = 10.3$; Section S26). A function of ρ returns one value for one input. A full replication reproduces the load-bearing cell ($2.02\times$), reproduces $k = 5$ in direction but not size, and resolves a small $k = 7$ difference ($1.17\times$, $z = 3.20$) the original could not — so we withdraw an earlier draft's reading of that convergence as a null; both campaigns agree on the shape, twofold at $k = 6$ collapsing toward parity at $k = 7$, one free core in eight being nearly as useless as none.

S8.1. Tail recovery without a reference clock

Recovering the tail without a reference clock, and where it reproduces: Because negative-span rate at a threshold *is* a quantile of Δ's tail, the tail can be read off negative span counts alone — a measurement-quality estimate needing no reference clock. The honest test of it is not internal consistency but reproduction: do the recovered tail quantiles agree between two independent campaigns at matched utilization? The intervals resample whole runs, because negative spans cluster within a run (S19) and an interval over events would treat them as independent draws. Below saturation, mostly: 15 of 17 matched quantiles overlap at 90%, every one of them at $\rho = 0.003$, 0.253, 0.504 and 0.878. The two that do not are at $\rho = 0.628$, where the later campaign's tail is 4.4 times the earlier one's at 0.5 ms, and at $\rho = 0.753$, where it is a fifth of the earlier one's at 2 ms. A direct test of each difference at the same level flags 4 of the 17, all between $\rho = 0.504$ and 0.753, where about two would be flagged by chance; intervals over events fail 5. At $\rho = 1$, where 4 of 15 fail, the campaigns are not at matched load at all: the coordinate is degenerate — eight, ten and twelve background workers all read as full utilization while representing different degrees of overload — so matching on it pairs different loads. The recovered curve therefore reproduces at most load levels below saturation, not at all of them: one campaign's recovered quantile is an estimate to replicate, not a measurement to quote.

S9. THE TWO-STATE MODEL COMMENTARY AND THE TRACER CHECKS

The form, and why it is stated here: Conditioning the identity $S < 0 \iff A > D$ on the end-to-end latency, and writing p for the timestamping thread's off-CPU waiting probability and G for the survival function of the residual delay in that state, gives to leading order

$$\Pr[S < 0 \mid T_{\mathrm{true}}] \;=\; p\,G(T_{\mathrm{true}}). \tag{S2}$$

This moved out of the main text, and the reason is worth stating rather than hiding. *Nothing in the paper computes with it.* Every reference Section IV-B makes to it names the two-state picture — the preempted state, the state a pinned thread removes — and the three consequences we test are attributable without it: the direction under payload is what the identity predicts, the floor C_0 is the $p \to 0$ limit of the picture, and the traced check compares two probabilities. The campaigns reported here never tested the form as a form. They measure p only as a time average: E-A7 registered that real-time priority would cut the thread's waiting probability at least tenfold, and its scheduler counters show a fall of 1.7- and 2.2-fold at the two loads, against a 35- to 47-fold fall in the rate, so that registered prediction failed. What the campaigns confirm is the mechanism and its consequences, not the factorization. A later campaign timed the second state's wait for every acknowledgment and predicted runs' rates from it with nothing fitted, on four of six pair and broker parts within its registered band, and found G's shape (S16.10). It was wrong once for exactly this kind of reason — written $p(\rho)\,G$ until Table II of the main text showed the rate is not a function of ρ — and was repaired by weakening it. A model that cannot be falsified as a model belongs where a reader can weigh it, not where the argument appears to rest on it.

Figure S4 draws the model this section defends.

What it accounts for: Clustering follows by construction: being preempted is an *interval*, so the events inside it are corrupted together and a two-state process cannot help but cluster. Going from idle to the knee multiplies the tail weight $61\times$ and the core width only $5\times$: two different parameters moving, σ_c belonging to the running state and p to the scheduler. The scale family fails because a scale family has one parameter and here two move at very different rates.

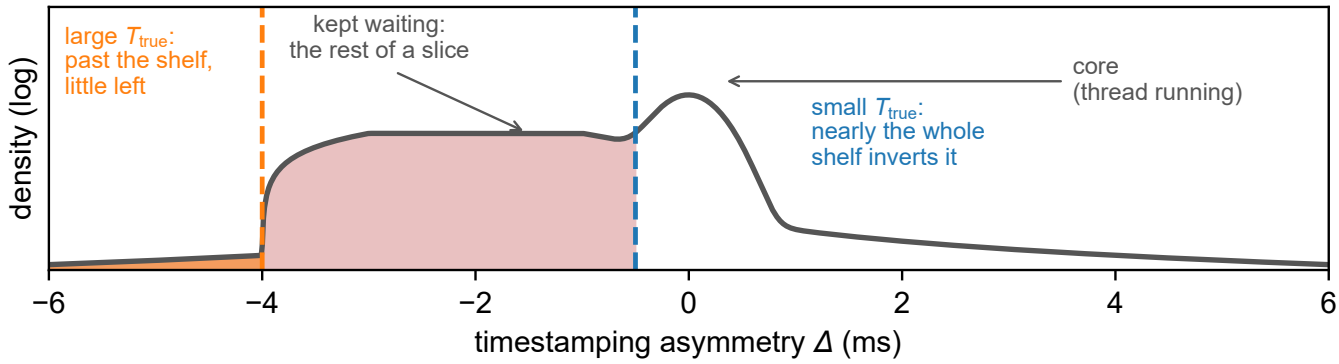


Fig. S4. **Two-state model, drawn.** The timestamping asymmetry Δ has a narrow core when the timestamping thread is running and, when it is kept waiting, a shelf reaching from the core to -2.8 ms, the scheduler's base slice, and a tick beyond it: the thread waits out what is left of another task's slice (S16.10). A small T_{true} is inverted by nearly the whole shelf and, in the model, one longer than the slice and a tick by none of it; the measured waits of S16.10 run past both, rarely — which is why the check bites hardest on small effects. The shape is schematic, two components drawn with the measured slice and tick rather than fitted; the measured distributions are the traced stall spectrum of Section IV-C and Figure S12.

A prediction that points the opposite way: The two structures disagree geometrically. A scale family changes σ, so its tail curves *rescale horizontally*; the two-state model changes p, so its curves *shift vertically and stay parallel* on a log scale. Taking the log ratio of two conditions' tail masses at several thresholds, the two-state model predicts a constant, independent of the threshold.

Restricting to tail estimates backed by at least 20 events — nine far-tail points resting on five to eight events are excluded, as they carry log-scale errors larger than the effect — the median spread of that log ratio across conditions is $\mathbf{0.29}$, a factor of 1.34, with a worst case of 0.91. Set against the $23\times$ failure of the scale-family collapse, the tail curves are close to parallel. The two conditions with the largest spreads sit at intermediate load, where the residual distribution is plausibly still shifting, so the honest statement is $\Pr[\mathrm{inv}] = p(\rho)\, S(c;\rho)$ with p varying fast and S slowly: separability is a good first-order description rather than an exact law.

The two failures are not one failure seen twice: The paper treats them as two failures of one regime, and the separation does more work than it looks: it is what stops a reader rebuilding the shared-number framing this project withdrew. Opposite movement would not establish it. Both failures answer to T_{true}, and a longer end-to-end latency makes one of them rarer and the other harmless, so a reader is entitled to ask whether they are one dependence seen from two sides. What settles it is not how fast either responds — any comparison of one response *factor* against the other *across the payload sweep* is the exponent of Equation S1 recomputed from fewer points, and a weaker copy of a result cannot corroborate it — but that only one of them can be escaped.

Their limits differ in kind. Retention saturates: $\min(1, T_{\mathrm{true}}/\tau)$ reaches one and stops, and the failure switches off. Of the 75 instrumented cells, four report above the grid, and of those only the two at 256 KB lose nothing. The two at 64 KB print medians of 235 and 519 ms and still keep only 34.4–35.9% of their samples, every sample they delete having computed to exactly zero, so a median above the grid is not by itself the exit. The negative-span rate has no such exit. At saturation it reaches 0.37, and the scale real-time priority settles on, 0.004, is a scale and not a limit: across all 8 pairs the manipulated configuration runs 0.0017–0.0144, on both sides of it. Neither number is a bound, and neither needs to be — what the contrast rests on is that the rate has no floor at zero, which is why Section VI keeps the sign check after every remedy it recommends. One failure can be lengthened out of existence and the other cannot, and no rescaling of a single dependence produces that.

What this does *not* show is that the failures are unrelated. They share an axis, and a benchmark on a sub-millisecond path meets both. They are not the same function of it, and the difference that establishes it is a difference of limits rather than of slopes.

What this is worth, and what it is not: It is mechanistic — waiting probability names the moving part — and actionable: if the failure is waiting probability, the mitigation is a timestamping thread the workload cannot preempt, not a better clock. But we found this by looking at the data; the separability test above is exploratory, so we pre-registered a confirmatory version and ran it: the campaigns below, the decisive one being real-time priority at fixed utilization. Two states is the smallest model consistent with the data, not a kernel claim.

On the load axis the model is a tautology, and we say so before using it: Read as a function of ρ alone the model has no content: with p unconstrained, any monotone rate curve can be written $p(\rho)\, S$; the model's content is on the threshold axis, where it passed. Constraining p to a power law fits poorly ($R^2 = 0.65$ in log space). The form that restores the divided-out σ, $\Pr[\mathrm{inv}] = p(\rho)\, S(T_{\mathrm{true}}/\sigma(\rho))$, reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 ($(\rho/(1-\rho))^k$) at equal parameters — *and we do not report that lead as a result*: freezing σ at its mean improves the fit to $R^2 = 0.9982$, because σ and ρ move together on this ladder and no fit can credit one over the other. That is a design limit no further analysis of these data can lift.

The tracer check, and what it can and cannot rule out. Tracing a system to measure its scheduling behavior is exactly the kind of intervention that can create what it observes, so the traced cell has an untraced twin. Our first attempt at this campaign ran with a probe that failed to attach: the histogram was empty, but the negative-span rates are valid, and we kept that run as the control. As first computed it measured 0.2717 against the traced run's 0.2315 — the traced rate 14.8% *lower*, two hours apart, same design. Both counts held messages the consumer received late, because it first read earlier campaigns' messages, and the traced runs held more of them (Supplement S1.5). Without those messages the two agree: 0.273 untraced against 0.272 traced ($z = 0.1$).

We report the first reading and not only the second, because an earlier version of this section compared the traced cell against a different campaign's 88% configuration (0.2214) and called the 4.6% gap a clean bill of health, and the backlog then made the gap look like a cost of tracing. Neither comparison was fair. The normal-priority 88% configuration was measured five times before the replication and spans a factor of about 1.4 between campaigns, so no tracing effect smaller than that drift can be read off another campaign. The twin, two hours apart with the same design, is the one fair comparator, and against it the tracer leaves the normal-priority rate where it was.

That comparison bears on the *level*, which no claim here rests on. The comparison in Section IV is between two quantities measured in the *same* run — a kernel histogram and a count of negative transports over the same events — so a common perturbation of that run would move both and leave their agreement intact. The comparison therefore uses the traced configuration's own rate.

The gate fired on our own replication, as first computed, and the backlog was why. The check is a rule fixed in advance: a traced configuration whose negative-span rate differs from its untraced comparator by more than 25% is not compared. As first computed, the replication's 88% configuration measured 0.1956 against the same untraced control's 0.2717 — a drift of 28% — and its verdict was withheld. Its consumers had read the longest backlog of the three traced configurations. Without the late messages it drifts 5%, and all three are compared (Table S26). The rule did what it was written to do with the data it was given; the data were what was wrong.

The real-time configuration's zero is an artifact of the tracer, and the replication settles it. Every traced real-time configuration we have run — three of them, across two campaigns and two load levels — recorded *zero* negative spans, in $2{,}447$ to $2{,}723$ events each. The untraced twin of one of those configurations recorded 15 in $2{,}985$. Under that rate the chance of a single traced configuration showing none is 1×10^{-6}, and of all three about 10^{-17}.

We first reported this as unresolved: either the configuration genuinely differed or tracing perturbed it, and one measurement could not separate the two. Three can. Attaching BPF to `sched_switch` suppresses the negative spans in the real-time configuration, and the effect is not subtle — it removes all of them. At normal priority, where values below zero are common, it leaves the rate alone.

We attached a tracer to measure why a measurement was failing, and the tracer changed the thing it came to observe, in the configuration where the quantity was smallest: exactly where Section IV-A says a measurement is most exposed. It costs us nothing here, because no claim rests on the real-time configuration's level, and it is worth stating plainly rather than leaving in a footnote: a kernel tracer is not a free observer.

S10. Retention sits on the grid, and the spread rule says when

Figure S5 is the confirmation the main text's Section V-D reports: the three payload configurations against their $q = 3$ grid, and the spread each one produces against the boundary the rule below puts at half a cell.

Why the path does not explain retention: Across the captured cells, publish latency takes only two values while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (Spearman $+0.31$; Fisher 95%: $+0.08$ to $+0.51$; 71 cells) carries the sign the retention law predicts. What moves retention is the phase of the publish schedule against the grid, which is what the rest of this section tests.

What the replicate spread does, and does not, predict: It is tempting to write the prediction as "spread $\approx 100/q$", and we did; that is wrong in a way that matters, because $100/q$ is the *step width* — an upper bound, reached only when T_{true}/τ sits midway between two grid points and replicates realize both. On a grid point, one point takes nearly every run and the spread collapses. One statement covers both:

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } T_{\mathrm{true}}/\tau \text{ sits mid-step,} \\ 0 & \text{when } T_{\mathrm{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{S3}$$

The measured continuous value is $T_{\mathrm{true}}/\tau = 0.495$, a half to within replicate noise. A half lies on every even grid ($1/2$, $2/4$, $4/8$) and never on an odd one, so at this operating point the odd denominators are predicted to show their full step width and the even ones to show almost none. That is a prediction about specific configurations before they are run, not a description after.

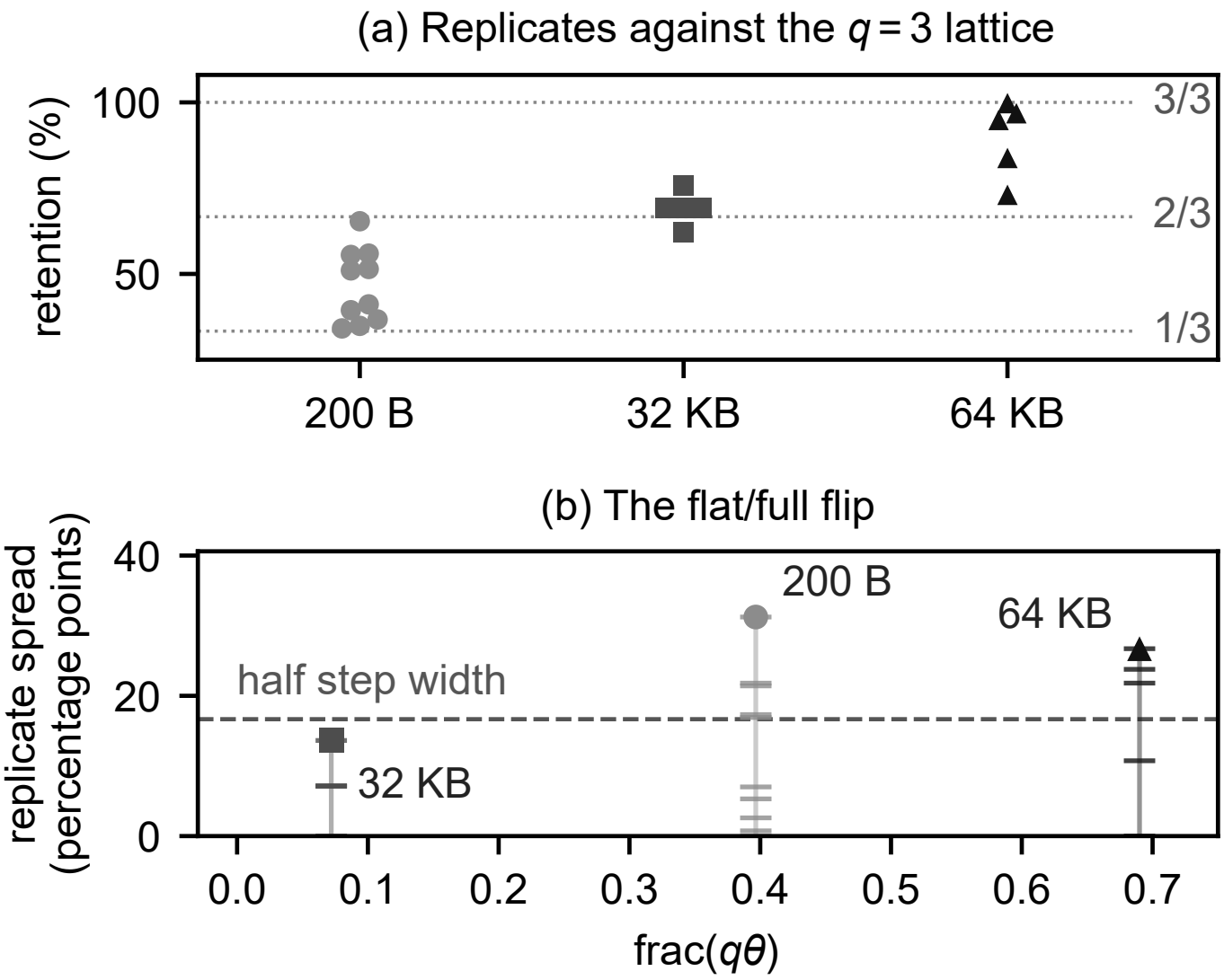


Fig. S5. **Campaign's confirmed prediction.** (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, 64 KB re-opens to the upper step. Horizontal position within an configuration separates replicates that would overprint and carries no variable. (b) Replicate spread against $\mathrm{frac}(q\theta)$, with θ each configuration's own mean retention in this campaign; the registered prediction placed 32 and 64 KB at 0.055 and 0.56 from the payload sweep's θ (S8). The flat–full boundary at half the step width is crossed in both directions by the one manipulated variable. The spread is a range, so each stick shows the replicates it is made of — 32 KB clears the boundary with three of five pinned together, 64 KB is broad.

Most configurations behave as their position predicts, and the two that do not are the reason this statistic was replaced: Table S17 states each rate's step width, cell position, prediction and measured spread against Equation S3; the test needs no tolerance, since a spread above half the step width separates full from flat. Of the 12 commensurate configurations, 10 match. The predictions are ten full against two flat, so both classes are realized and the agreement is with a prediction that varies, not with a constant. The table's last two columns are that comparison, one row at a time.

The two that miss are 700 and 1250 msg/s. Both are predicted full and measure flat, and we report them as failures of *this* statistic rather than of the model, for a reason that was stated before either was run and does not depend on how they came out. Spread is a range over replicates of a quantity with two-point support: an configuration realizes the full step width only if its replicates land on *both* bracketing vertices, and five replicates that happen to fall on one of them produce a flat configuration however exactly right the grid is. At $q = 5$ and $q = 7$ the upper branch carries a minority of the probability, so this is the outcome to expect some of the time, and the estimator cannot tell it from a refutation.

That is precisely why the replacement below does not use spread at all. It was built after the campaign had run and was not registered, and it answers both configurations: 700 and 1250 msg/s each reject the continuum after Holm correction, at 0.009 and 0.003, and the table of that test records both as *grid*. An estimator that cannot separate "on the grid, one branch sampled" from "not on the grid" was the whole reason for building one that can, and the two misses are the clearest case in the corpus for having done it.

The denominator, not the rate that carries it: Every denominator above appears at one rate, so the pre-registered control is the 600 msg/s configuration: 1.667 ms $= 5/3$, the same $q = 3$ as 300 msg/s at *half* the interval, recorded in advance to land near 33%. It landed at 33.98, 34.29, 34.90, 36.17, 65.86 and 66.67 — four replicates on the $1/3$ branch, two on the $2/3$ branch, nothing between (one replicate's shutdown-hook totals arrived after our first read; the recovered 65.86 was on the upper branch). The loosest of them, 36.17, sits 2.83 points from its vertex, inside the 3-point rule stated below and the widest departure the configuration shows. 66.67% is $2/3$ to two decimals, matching the 65.35 that 300 msg/s produced on the same branch: two rates an octave apart land on the same grid because they share a denominator.

A quantity with two-point support breaks the replicate median, so the replacement (`grid_membership_test.py`, written after the campaign and committed with its first results) tests each configuration's mean distance to its nearest grid vertex, normalized by step width, against a continuum null of Normal($\theta_{\mathrm{local}}, \sigma$=1.2) replicates — both parameters measured, neither fitted to the configuration under test; the p-value is one-sided by Monte Carlo.

Of twelve commensurate configurations, ten are powered — the null and the grid predict separably — and 9 **of the** 10 **reject the continuum after Holm correction**, eight at the Monte-Carlo floor of 2.5×10^{-4}, including configurations whose binary flat/full classification had read as smeared. The tenth (900 msg/s) rejects at 0.044 raw and at 0.131 corrected, so it does not reject: an earlier version of this document and of the main-text table counted it as a rejection while the caption claimed the correction had been applied. That is now fixed in the only durable way, by generating the table from the artifact. The two unpowered configurations are the degenerate ones (400 and 800 msg/s, whose grids pass through θ), and the test reports them as undecidable rather than counting them for either side. Where every replicate lies within 3 points of a vertex the configuration is branch-classifiable and the upper-branch weight carries an exact Clopper–Pearson interval in the artifact. In plain terms: judged by a test that cannot be fooled by a value with no center, the grid is present in every configuration that could show it.

S11. Grid membership, configuration by configuration

Figure S6 makes the claim about the set; Table S7 is the same twelve configurations one row at a time, for a reader checking an individual rate. Both moved here from the main text, where the grid refinement became the explanation of the deletion rather than a contribution in its own right.

Of the ten powered configurations, nine reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining two configurations, 400 and 800 msg/s, have no power by construction: where T_{true}/τ sits on a vertex the two predictions coincide, so the distance is replicate noise and which side of the diagonal it falls on is not evidence either way. Table S7 is generated from `grid_membership.csv` at build time by the same helper that produces the figure, so the two cannot disagree.

TABLE S7

Grid membership against a continuum null. D is the mean normalized distance to the nearest grid vertex; under the null its expectation is the value in the *null* column. That column was, until this revision, the distance of θ from its nearest vertex with no replicate noise — the distance of the expectation rather than the expectation of the distance, which are not the same number and differed by up to 0.04 here. It is now the mean of the simulated null itself, so the sentence above it is true. p_{Holm} is a Monte Carlo p-value (4,000 draws) after Holm correction within the family of all 12 configurations; the corrected value is what decides the verdict. Arms marked *flat (coincident)* are those the model predicts flat, where the two hypotheses make the same prediction and the test has no power. The 700 and 1250 msg/s rows are the two that the spread rule of Table S17 misses; this test resolves both, which is why it replaced that rule.

rate	a/q	n	$\bar{d}$	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.340	0.131	not resolved
875/s	8/7	5	0.209	0.382	0.003	grid
800/s	5/4	5	0.104	0.058	1.000	flat (coincident)
700/s	10/7	5	0.329	0.416	0.009	grid
625/s	8/5	10	0.112	0.447	0.003	grid
600/s	5/3	6	0.034	0.469	0.003	grid
500/s	2/1	5	0.043	0.490	0.003	grid
400/s	5/2	5	0.093	0.020	1.000	flat (coincident)
300/s	10/3	10	0.221	0.468	0.003	grid
250/s	4/1	5	0.181	0.489	0.003	grid

S12. Every real-time-priority pair, the ladder they make, and the exposure curve

Section IV quotes a range: real-time priority on the timestamping threads collapses the negative-span rate 7–80× at fixed utilization. That range is taken over 8 matched pairs from three campaigns, and until this revision only the two of Table S23 appeared anywhere — which left the two pairs that anchor the range, including the weakest result in the series, visible nowhere. Both are below.

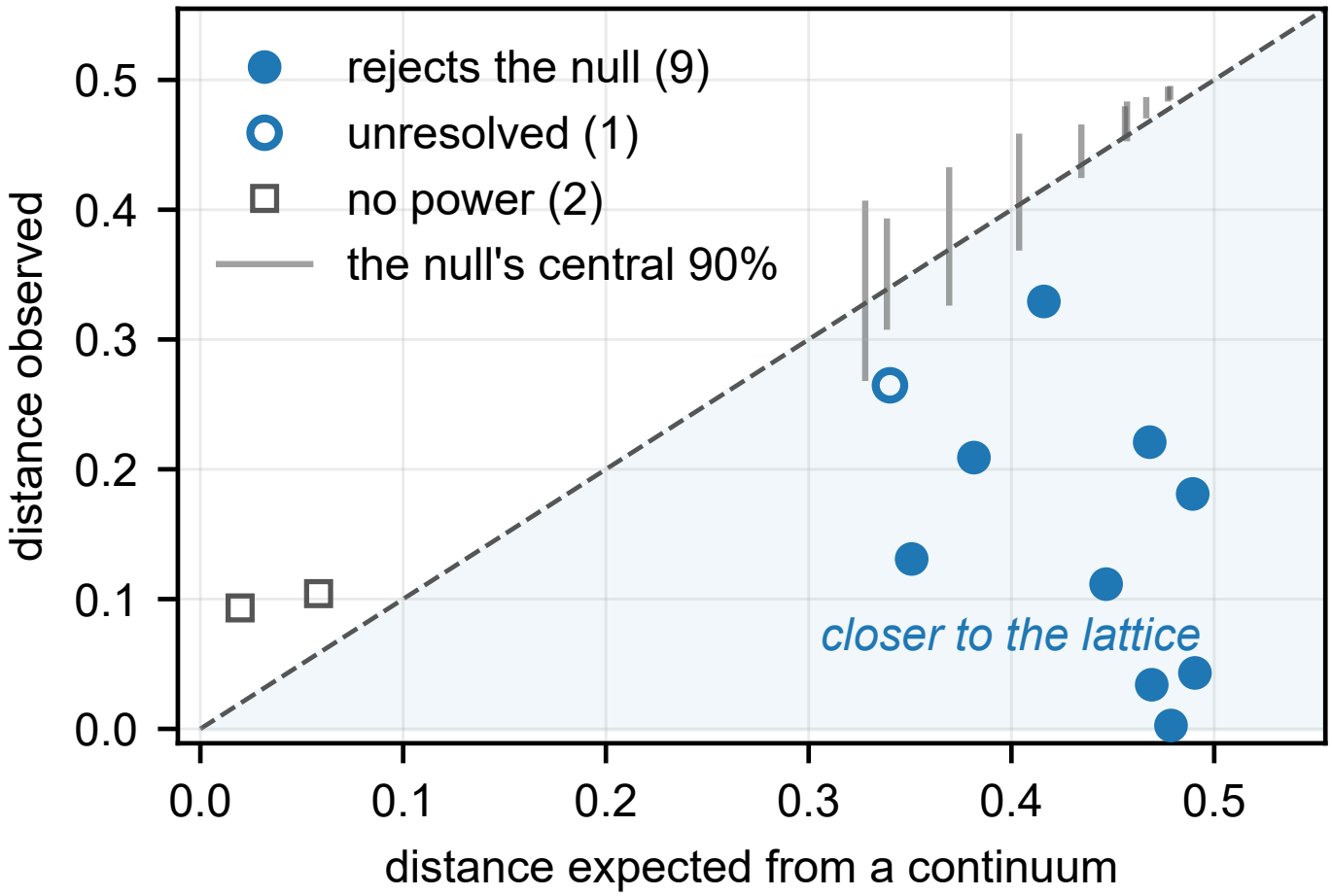


Fig. S6. **Grid membership, as a set.** Each configuration's observed mean distance to the nearest grid vertex against the distance a continuum would give, which is the diagonal. The gray bar beside a configuration is its own simulated null, *uncorrected*; the test is one-sided towards the grid, so a marker *below* its bar rejects that null. Filled circles also survive Holm correction. The open circle sits *below* its bar too, 0.2647 against a band opening at 0.2680, but at $p = 0.044$ raw and 0.131 corrected it is the one powered configuration that does not reject: correction is what the picture cannot draw. The two squares carry no bar because no test has power there: the two hypotheses coincide, and *the sign carries nothing*. Corrected p-values are in Table S7.

Table S8 is generated from `docs/results/model/stamping_priority*.csv` at build time, so the range in the abstract and the rows here cannot disagree. E-A5 is the original manipulation, E-A5b widened the load ladder to 60 and 95%, and E-A7 replicated E-A5. Every pair has disjoint Wilson intervals, and 0 were withheld for a failed manipulation check — the design assumes real-time priority leaves ρ alone, so ρ is measured in both configurations and a discrepancy above 0.05 withholds the pair rather than reporting it hedged. Naming the threshold matters more than the count it produced: a withholding rule whose criterion is invisible cannot be checked, and this one never fired because it never came close. The widest disagreement any pair showed was 0.002, some 25 times inside the tolerance, so the zero above is a clean manipulation rather than a loose test.

Figure S7 shows what the range does not: which configuration moves. The real-time configuration sits between 0.0017 and 0.0144 across the whole load ladder, close to the floor the model names in advance; the normal-priority configuration rises from 0.046 to 0.30. The collapse is therefore not a proportional reduction that happens to be larger under load — it is the normal-priority configuration following utilization while the manipulated configuration does not, which is the two-state model of Equation S2 with p moved directly.

The mechanism table's factors, with their intervals: The main text's Table II prints each factor without its interval. The Katz 95% interval on the ratio is [21–74] for the priority pair at 75% ($z = 19.8$) and [33–86] at 88% ($z = 31.9$); for the load geometry it is [1.80–2.39] in the original campaign ($z = 10.3$) and [1.71–2.40] in the replication ($z = 8.3$). The priority intervals are wide because the collapsed configurations are sparse; the Wilson intervals on the rates themselves are disjoint at both load levels. Over all 8 pairs the real-time configuration settles between 0.0017 and 0.0144, so the floor C_0 is where the rate settles and not a bound it respects.

Two exhibits here restate main-text Tables I and II and Section VI-B at a size a column cannot give them. Figure S8 draws the four matched pairs of Table II and the two brokers of Table I: no pair of configurations overlaps, which is checkable in those tables and is faster to see here. Table S9 tabulates the exposure curve whose headline points Section VI-B quotes —

TABLE S8

EVERY MATCHED PAIR. NORMAL-PRIORITY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED, SO THE CLAIM THAT THE MANIPULATION MOVED WAITING PROBABILITY AND NOT LOAD CAN BE CHECKED PAIR BY PAIR RATHER THAN TAKEN FROM A CAPTION: THE WIDEST DISAGREEMENT ANYWHERE IN THE TABLE IS 0.002, AGAINST THE 0.05 THAT WOULD HAVE WITHHELD THE PAIR. GENERATED AT BUILD TIME FROM THE THREE CAMPAIGN FILES.

				Kafka			Redis		
campaign	load	ρ normal	ρ real-time	normal	real-time	factor	normal	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	27.4 [8.1–∞]	0.1117	0.0186	6.0 [4.6–8.2]
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	7.2 [4.4–14.8]	0.1174	0.0197	6.0 [4.2–9.2]
E-A5	75%	0.7531	0.7525	0.1320	0.0034	39.4 [15.1–∞]	0.1284	0.0202	6.3 [5.0–8.3]
E-A7	75%	0.7550	0.7537	0.1504	0.0044	34.5 [15.4–∞]	0.1301	0.0176	7.4 [5.6–10.0]
E-A5b	88%	0.8806	0.8809	0.2223	0.0034	66.4 [26.2–∞]	0.1636	0.0220	7.4 [5.3–11.2]
E-A5	88%	0.8809	0.8812	0.3049	0.0057	53.5 [24.6–451.5]	0.1666	0.0255	6.5 [5.1–8.8]
E-A7	88%	0.8825	0.8825	0.2532	0.0054	47.2 [22.0–685.9]	0.1687	0.0226	7.5 [5.6–10.2]
E-A5b	95%	0.9501	0.9501	0.2943	0.0037	79.9 [34.0–∞]	0.1754	0.0225	7.8 [5.7–11.2]

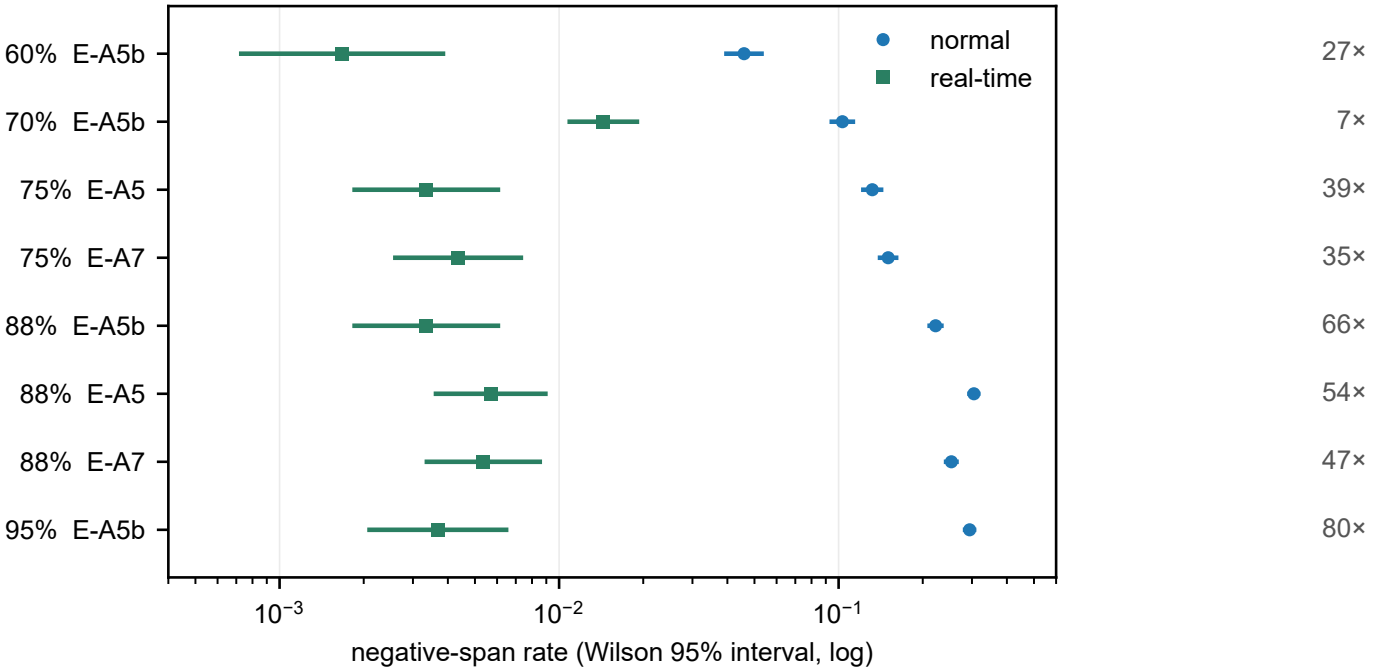


Fig. S7. **All 8 pairs, as a ladder.** Normal-priority and real-time negative-span rates with Wilson 95% intervals, one row per pair, ordered by achieved utilization; the factor is printed at the right. Reading down, the real-time configuration barely moves while the normal-priority configuration climbs with load, which is what makes the factor grow. The growth is a trend and not a law — 60% gives 27× and 70% gives 7× — and the range is reported as a range for that reason.

the relative error A/T_{true} against path length, and beside it the gap a comparison would report between two identical systems whose clients timestamp in different places — and Figure S9 draws the same publish latencies as the curve of main-text Figure 4, at full width. Neither carries a number that is not already stated where it is argued.

The rows added below one millisecond are the regime broker benchmarks actually report in (Section S33), and they are where an acknowledgment-timed harness stops measuring the path at all. A publish-timed one is unaffected: it reports the end-to-end latency itself, which is why the framework audited in S33.3 needs no correction from this table.

Which estimator produced the two headline ratios, stated because it decides them: Section VI-B says the reported span is 24% of the end-to-end latency it stands for, understating it 4.1×. Both are the *median over conditions of* median S/median D — a median of per-condition ratios, computed in `emit_paper_numbers.py` and gated there. The estimator is not a detail. Working from the medians this document prints, which is the natural recomputation, a reader who differences them condition by condition gets 3.5×, and one who divides the pooled medians gets 2.6×. Three defensible readings, a factor of 1.6 between the extremes, and only one of them is the number in the text. This project has been caught by the same distinction once before, on the independence overshoot: pooled and with no floor, median-of-ratios and ratio-of-medians differ by 12.4× against 21.4× (S24.2). Within runs and above a 0.1% floor they nearly agree, 7.4× against 7.0×, once the floor has removed the smallest denominators. Section III-A quotes the median of ratios at that floor, 7.4×. It is taken over the 54 conditions above 0.1%, and the estimator is stated here too.

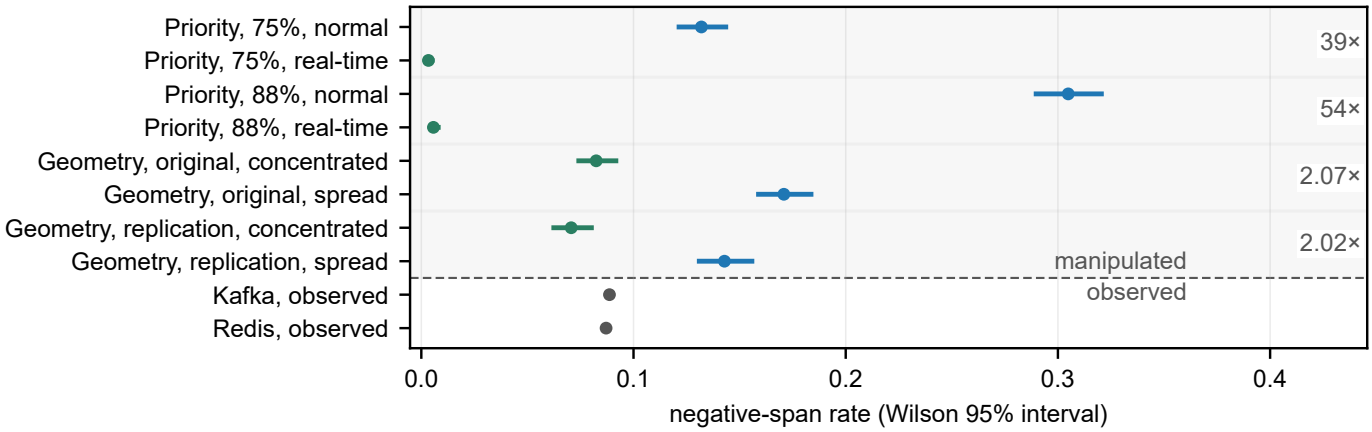


Fig. S8. **Every manipulation separates; the broker does not.** Wilson 95% intervals. Above the rule, the four matched pairs of Table II, utilization held fixed within each: what moves the rate is where the timestamping thread sits, not how loaded the machine is. Below it, observed rather than manipulated, the two brokers of Table I; their intervals are narrower than the plotted point.

TABLE S9

WHAT THE DISPLACEMENT COSTS AT LONGER PATHS, AND HOW WIDE THE COST IS. COMPUTED FROM THE MEASURED PUBLISH LATENCIES, NOT MODELED. LEFT: THE ERROR ON A SINGLE REPORTED LATENCY, A/T_{true}, AT THIS CORPUS'S MEDIAN PUBLISH LATENCY OF 684 μS. MIDDLE: THE SAME ERROR AT THE TENTH AND NINETIETH PERCENTILES OF THAT PUBLISH LATENCY ACROSS OUR CONDITIONS, 514 AND 1906 μS. THE BAND IS THE COLUMN TO READ. A PUBLISH LATENCY IS NOT A CONSTANT, AND A READER LOCATING THEIR OWN PATH AGAINST THE POINT ESTIMATE ALONE WOULD PUT A 10 MS PATH AT 6.8% WHEN A TENTH OF OUR OWN CONDITIONS SIT AT 19%. RIGHT: THE GAP A COMPARISON WOULD REPORT BETWEEN TWO IDENTICAL SYSTEMS WHOSE CLIENTS DIFFER ONLY IN WHERE THE ACKNOWLEDGMENT IS TIMESTAMPED — A CALLBACK THREAD AGAINST AN INLINE TIMESTAMP. WHERE THE RIGHT-HAND COLUMN READS "INVERTS", THE APPARENT GAP HAS CHANGED SIGN; ON EVERY ROW BELOW THOSE NOTHING IS NEGATIVE, SO THE SIGN CHECK OF SECTION II-C FIRES ON NONE OF IT.

True path T_{true}	Error on one measurement	p10–p90 across conditions	Apparent gap between two identical systems
0.25 ms	274%	206–763%	inverts
0.5 ms	137%	103–381%	inverts
1 ms	68%	51–191%	inverts
2 ms	34%	26–95%	66%
5 ms	14%	10–38%	22%
10 ms	6.8%	5.1–19%	10%
25 ms	2.7%	2.1–7.6%	4.0%
50 ms	1.4%	1.0–3.8%	2.0%
100 ms	0.7%	0.5–1.9%	1.0%

What the comparison costs, in full: The understatement is a median, and half the conditions are worse: its interquartile range runs 3.8–6.7× and the worst condition reaches 9×. Two *identical* systems whose clients timestamp in different places are separated by the difference of their publish latencies, a reported gap of 10% at a 10 ms end-to-end latency, with the sign check firing at neither. Over the 24 matched workloads of Section VI-B the proxy inflates the gap between the brokers 1.7×. The interquartile range of that factor is 1.3–1.8, and on 21% of the pairs the proxy narrows the gap instead.

S13. WHAT THE BROKERS ACTUALLY DO

Section VI-D states two conclusions and points here for the evidence: that the broker comparison is not a purchasing argument, and that one configuration choice, invisible on the standard testbed, decides the outcome. This is the campaign behind both. It sits in the supplement because the paper's contribution is two measurement failures, and what the brokers do is a smaller result than either — smaller, in the end, than the comparison that motivated the work was expected to be.

S13.1. Equivalence on gated artifacts

Over the runs the sign check leaves standing, the brokers sit within a millisecond of each other on the transport proxy: a Hodges–Lehmann shift of 0.408 ms (90% bootstrap interval 0.389–0.419), with TOST rejecting both one-sided nulls against a 1 ms margin at $p < 0.001$, holding between 0.408–0.417 ms across 3 concurrency levels.

That span is not a causal chain, so the equivalence is also stated on the quantity that is. Over the same runs and levels, the event-time latency (Equation 2) is equivalent against a 40 ms margin, one broadcast frame at 25 frames per second, fixed in the analysis script three days before the campaign ran: Hodges–Lehmann shifts of 0.682–0.859 ms, Kafka slower, and the highest upper bound of any 90% interval of the three estimators is 0.906 ms, so the transport proxy's margin would have held

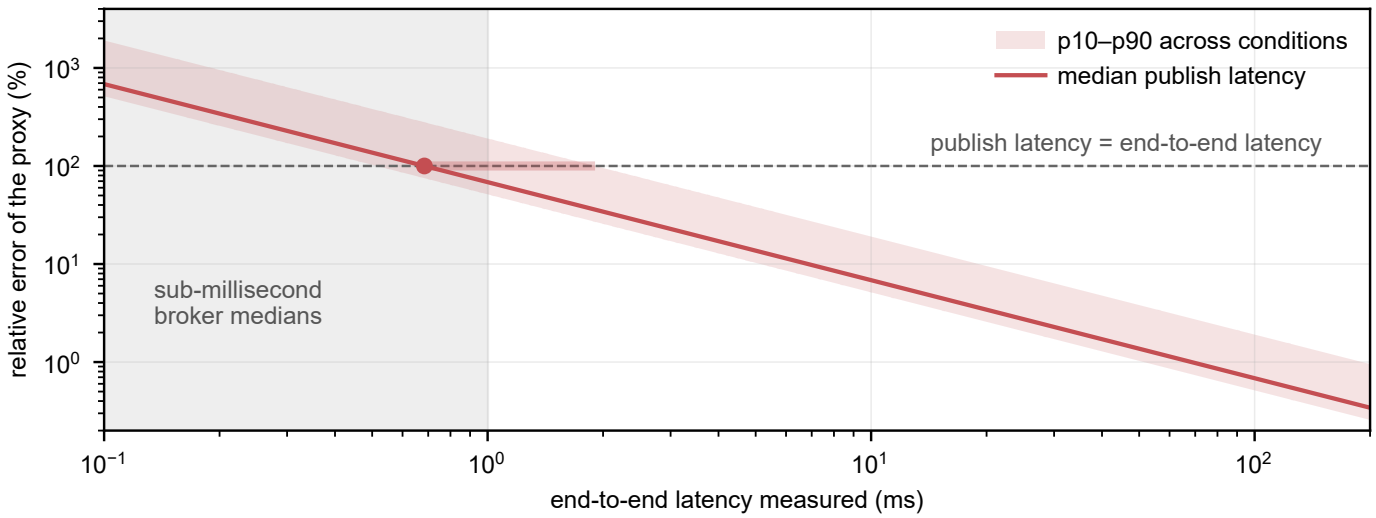


Fig. S9. **Where a path sits on the exposure curve.** Table S9 as a curve: the relative error of the acknowledgment-timed proxy, A/T_{true}, against the end-to-end latency measured. The line is this corpus's median publish latency and the band its tenth to ninetieth percentile across conditions, so a reader can bracket their own path rather than take the middle. The bar on the dashed line is the crossover, where the publish latency equals the end-to-end latency it is a correction to: 0.68 ms at the median publish latency and 1.91 ms at the ninetieth percentile, marked as an interval because it is one. To its left the correction is larger than the quantity corrected. The shaded column is where published broker medians sit (Section S33.4), which is the reason the curve matters.

as well. Equivalence is claimed rather than assumed from a failure to reject, which is the reason for the TOST rather than a two-sample test.

S13.2. The condition that reverses the ordering

One-way delay injected identically at both leaves Kafka tracking it almost additively, while Redis collapses: a median transport of 4.6 s at 5 ms of injected delay, 31.3 s at 20 ms and 102.5 s at 50 ms, over five feeds per configuration. That is about 930 to 2,050 times the delay applied.

The cause is the consumption pattern rather than the broker. A per-message acknowledgment turns each event into a sequential round trip, so injected delay multiplies once per event, where a prefetching consumer serves from buffer and pays it once per batch.

S13.3. The falsifier, registered in advance

The prediction was recorded before the run: if the acknowledgment pattern is the cause, then batching the acknowledgments must remove most of the effect without touching the broker, the network or the injected delay. Batching them two hundred at a time cuts the median from 4,138 ms to 103 ms, a factor of 40, replicated four times.

S13.4. Why the standard testbed cannot see it

Both settings are invisible on a co-located testbed: the round trip they multiply is a few hundred microseconds there, so the difference between paying it once per event and once per batch is below the noise. The choice that decides the outcome under any realistic separation is therefore not visible in the environment most comparisons are run in — which is the same shape of failure as the two the paper is about, a tool whose configuration determines the result while nothing in the output records it.

S14. The classical statistics of a discarded zero, and what the guard deletes

Section V argues that deleting the samples which compute to zero conditions the reported distribution on being positive, and that the retained fraction is therefore part of the measurement. Two classical sources sit close enough to that argument to be named, and both need care in the naming, so we give them the space here rather than compress them into a clause.

The baseline the admission condition invalidates: The GUM gives the standard model for a quantized reading. Clause F.2.2.1, "The resolution of a digital indication", states that if the resolution is δx then the stimulus "can lie with equal probability anywhere in the interval $X - \delta x/2$ to $X + \delta x/2$" and is therefore described by "a rectangular probability distribution … of width δx with variance $u^2 = (\delta x)^2/12$" [S7]. That symmetric model is the departure point for this paper: it holds for *retained* readings, and the admission condition of Section V of the main text removes one side of the interval, so neither the rectangular shape nor the $(\delta x)^2/12$ variance survives the filter. Quantization is not the problem; the asymmetry the admission condition introduces on top of it is. We cite the clause rather than the formula because the same $(\delta x)^2/12$ appears in F.2.2.2 for hysteresis, and the form alone does not identify the mechanism.

The oldest statement that discarding zeros is not free: Pratt [S8] showed that discarding zero differences before ranking violates a natural monotonicity requirement: a sample not judged significantly positive can become significantly positive when *every* observation is decreased. He calls this an anomaly and gives a procedure that avoids it — rank all observations including the zeros, drop only the zeros' ranks, and refer the statistic to its null distribution conditional on the number of zeros.

Two qualifications keep the analogy honest. First, Pratt *conditions on* the zero count; he does not treat it as evidence of a shift, and he explicitly declines to claim any optimality for the procedure. Our claim about the retention rate is the weaker and more practical one — that the count must be reported so a reader can see what the distribution is conditioned on — and it does not need Pratt's stronger machinery. Second, the reading that makes the parallel exact is not Pratt's own framing but one he reports from Meier in a remarks section: that a zero arises because "with only our crude measurement, we do not know" the sign a finer measurement would have resolved. That is precisely the sub-resolution sample of Section V of the main text, and it is a 1959 statement of it; we attribute it where it belongs rather than to Pratt's main development.

A second standard already owns the lemma: The arithmetic core of the grid — a rate whose ratio to the sampling tick is J/M in lowest terms visits exactly M distinct phases — is standardized in ADC metrology. IEEE Std 1241-2010's sine-wave histogram test requires the record to hold a number of input cycles "relatively prime" to the number of samples (§6.4.1, Eq. (28)); the same standard's sine-fit clause states the frequency choice as $f_i = (J/M)\, f_s$ and names the practice *coherent sampling* (§5.4.1, Eq. (18)) [S9]. Both clauses exist for the same reason we choose incommensurate publish rates. (The standard's notation is J/M where this document writes p/q.) As with the counter literature, we claim the consequence under deletion, not the lemma.

What the admission condition deletes here, measured: The three arguments above are about what deletion does in principle. Figure S10 is what it does to this corpus, in three panels. Panel (a) is the control Table I counts: over the same $708{,}505$ joined events on the same clock, the acknowledgment-timed span falls below zero $62{,}264$ times and the publish-timed span never does, so the negatives are a property of which timestamp serves as the origin and not of the latency being timed. Panel (b) puts a millisecond timestamp in front of that population. The difference of two truncated timestamps lands on 0 or on 1 according only to where the tick boundaries fall, and a guard that admits a sample only when the difference is strictly positive removes $338{,}242$ of $708{,}505$ events — 47.7% (95% bootstrap resampling runs rather than events: 47.4–48.1) — of which $303{,}468$ computed to exactly zero and $34{,}774$ to less. None of it is counted in what such a harness reports. Panel (c) applies five dispositions to that one measured population: three from the software the registry of Section S25 read (the benchmark's discard, `fio`'s zero and `btt`'s floor of one), the NaN that Kafka's KIP-489 proposes [S10], and keeping the data, which is ours. Two of them shrink the sample the statistic is computed from and report no count of what left; two hold the sample count fixed by reporting a value that was never measured; one keeps the data.

One qualification, because the panel invites the stronger reading. Panel (b) is that rule applied to *our* spans, not a record of the external benchmark deleting our events: it says what a millisecond admission condition would remove from this population. The per-cell retention Section V quotes is the other measurement — what that harness's own instrumented runs did report — and the two answer different questions.

S14.1. The two floored clocks behind the $-1000\,\mu$s signature

Section V-F states the signature and what it rules out. In the full ledger the Redis-driver replication's 9 runs carry all $41{,}403$ of the campaign's negatives, every one exactly $-1000\,\mu$s, and $41{,}397$ of them sit in a single run, where they were half of all samples taken, beside six kept. This is why the two drivers differ, which is a property of the harness rather than of either broker.

The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72 — the stream-entry identifier the *Redis server* assigns on `XADD` [S1]. The Kafka driver uses a producer-set timestamp. So the Redis end-to-end span subtracts a consumer-side wall clock from a server-side one, across two hosts, with both floored to the millisecond; the Kafka span floors two timestamps whose difference is taken inside one JVM.

Two consequences follow, and together they are what makes the sign channel legible.

First, the arithmetic admits exactly one negative value. Two clocks floored to the same timestamp resolution, offset by less than one tick, can disagree by one tick and by no other amount, so the only negative a floored cross-host difference can produce is $-1000\,\mu$s. The corpus contains $41{,}403$ negatives and every one of them is that value.

Second, the Kafka corpus's $10{,}913{,}263$ discarded samples contain no negative at all, because a difference taken inside one process has no second clock to disagree with. The same admission condition therefore hides two different failures — a resolution failure under Kafka and a clock failure under Redis — and an unsigned discard counter could not tell them apart.

It also means the two drivers do not share a measurand: Kafka's span is client-to-client on one clock, Redis's server-to-client across two. A comparison of the two numbers as though they measured the same interval is a further error the harness invites and does not flag.

S14.2. The continuum null, constructed

Section V-D names the test and reports its verdicts. This is how the null is built, which matters because the null is the whole content of the claim: a continuum hypothesis has to be turned into a distribution before anything can reject it.

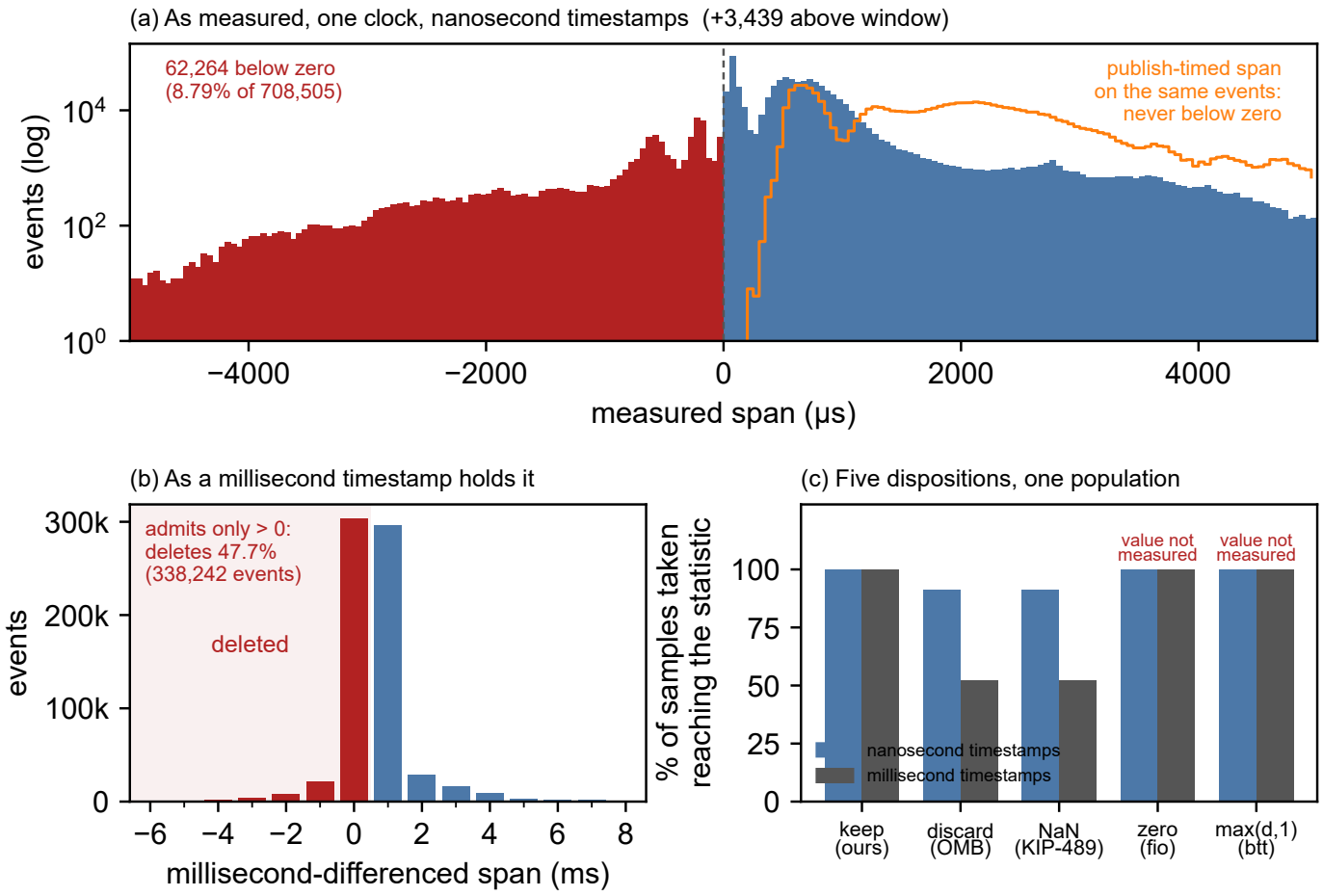


Fig. S10. **What the benchmark deletes, beside what it then reports.** (a) At nanosecond resolution the acknowledgment-timed span goes below zero 62,264 times in 708,505; the publish-timed span, same events and same clock, never does. (b) A millisecond timestamp cannot see (a): it differences two truncated timestamps, so a sub-millisecond end-to-end latency lands on 0 or 1 by tick alignment alone, and an admission condition then deletes 47.7% of the population uncounted. (c) Five dispositions of a non-positive sample, three of them from the registry of Section S25, applied to that same population. Every count is emitted from the committed corpus by `scripts/span_histogram.py`.

For each configuration, let D be the mean distance from its replicates to the nearest vertex of the q-grid, divided by the step width $100/q$ points. Under the continuum null, retention is a continuous function of rate, so a replicate carries no memory of the grid: it is drawn from a normal distribution centered on the configuration's rate-local continuous value, with the pooled replicate standard deviation of the incommensurate configurations. Both are measured, and neither is fitted to the configuration under test. The null distribution of D is obtained by Monte Carlo, $4{,}000$ simulated configurations of the observed size per test, and the p-value is one-sided, because membership in the grid makes distances small. That count fixes the smallest attainable p at 2.5×10^{-4}, which is why no configuration reports a p below that value however far it sits from the diagonal.

Two properties of this construction are worth stating because they bound what the test can say. It simulates the null at the observed size rather than leaning on an asymptotic distribution, which matters at $n = 5$. And it has no power at all where the grid and continuum predictions coincide, which is not a weakness to be apologized for but the reason the two configurations at those rates are reported as untestable rather than as evidence — a test that separated them everywhere would be measuring something other than what it claims.

S15. Which clocksource, bounded from a measurement

Section VIII states the verdict in one sentence: a read-cost bound admits only two clocksources, and the corpus closes the channel a second time on headroom, not on shared endpoints. Both halves are derived here — the increment the probe saw, what it excludes, and why the publish-timed span's floor settles the question independently of which clocksource was in use. An earlier draft argued from the endpoints instead: the two spans end at the same consumer read, so an artifact of that read would move both. That does not discriminate, because both spans are producer-to-consumer differences and both carry any offset; headroom does.

Which clocksource did the cloud instances use? The question matters because a virtualized guest need not be given a counter that is coherent across its vCPUs, and if ours were not, that would be a rival to the account in Section IV. We did not record

the field, and the instances have been reclaimed, so it cannot be read back. Our first answer was that we could not say. That answer was wrong, and the correction is worth setting out because the method is the same one the rest of the paper uses: a physical constraint deciding what the record-keeping did not.

The platform probe recorded the smallest non-zero increment observable from `clock_gettime(CLOCK_MONOTONIC)` on the machine that produced the reported runs: 90 ns. An increment is observed by reading the clock twice, so that number is an upper bound on the cost of a single read — and read cost separates the four candidate clocksources by more than an order of magnitude.

acpi_pm, excluded by arithmetic: The ACPI power-management timer counts at `PMTMR_TICKS_PER_SEC` = 3,579,545 Hz (`include/linux/acpi_pmtmr.h`), a period of 279 ns. A clock whose tick is 279 ns cannot report an increment of 90 ns, whatever it costs to read. This exclusion needs no timing figures at all.

hpet, excluded by cost: HPET is memory-mapped I/O to an off-die chipset timer. The x86 maintainers, in the commit removing it from the vDSO, describe reads as "exceedingly slow (on the order of several microseconds)"; published measurements of a `clock_gettime` call on HPET sit near 1.2 μs. We are careful about the ground here: HPET's own tick, near 70 ns at 14.318 MHz, is *finer* than the observed increment, so granularity does not exclude it and we do not pretend it does. The exclusion rests on read cost alone.

What remains, and why it answers the question: That leaves `kvm-clock` or `tsc`, measured at 33–60 ns and 24–29 ns respectively, both comfortably inside the bound. Each carries the guarantee that question asked about, by a different route.

For `kvm-clock` the route is the vDSO. Without vDSO acceleration the same clock costs about 156 ns per call, which the bound excludes; and `arch/x86/kernel/kvmclock.c` sets the vDSO mode only under a condition:

```
if (!(flags & PVCLOCK_TSC_STABLE_BIT))
    return 0;
kvm_clock.vdso_clock_mode = VDSO_CLOCKMODE_PVCLOCK;
```

`PVCLOCK_TSC_STABLE_BIT` is the flag the KVM guest ABI documents as "time measures taken across multiple cpus are guaranteed to be monotonic" [S11]. A `kvm-clock` fast enough to fit the bound is therefore one whose hypervisor asserts cross-vCPU monotonicity.

For `tsc` the route is selection. `kvm-clock` registers at rating 400 and `tsc` at 300, so on a KVM guest `tsc` is chosen only where `kvmclock_init()` demotes `kvm-clock` to 299 — which it does exactly when `CONSTANT_TSC` and `NONSTOP_TSC` are present and `check_tsc_unstable()` is false. Should warp appear later, `tsc_sync.c` reports "Measured %Ld cycles TSC warp between CPUs, turning off TSC clock" and the source is re-rated away.

What this does not establish: It identifies the set of clocksources consistent with the measurement, not which one was in use, and it leans on published read costs for hardware we no longer hold. Those costs vary by roughly a factor of two between independent measurements; the margin here is an order of magnitude, which is why the conclusion survives the spread. It is an inference from a measurement rather than a recorded fact, and Section II-A says so. The derivation is `scripts/clocksource_bound.py`, and its tests pin each exclusion to its own ground so that an exclusion firing for the wrong reason cannot reach the right answer by luck.

S16. Why the acknowledgment timestamp is late, and why the broker is not the reason

In plain terms: the span that inverts is the one whose reference timestamp is read only after a thread has been woken, and the corpus shows twice over that this has nothing to do with which broker is running. Section IV states the mechanism; this section gives the source lines, the control that separates the candidate causes, the rivals that were considered and dropped, and the parts we could not test.

S16.1. Where each timestamp is actually taken

Producer and consumer are separate processes on one host, and both read CLOCK_REALTIME. Five timestamps enter the spans this paper reports, and they are not taken in comparable places. Three of them are producer-side.

- **The publish timestamp** is read on the calling thread immediately *before* the client is invoked. In `scripts/redis_producer.py` the worker executes `send_ns = now_ns()` and only then calls `r.xadd(...)`.
- **The Redis acknowledgment timestamp** is read on that same thread the moment `r.xadd(...)` returns. `xadd` is synchronous: the thread writes the request and blocks in a socket read until the reply arrives. No second thread is involved anywhere.
- **The Kafka acknowledgment timestamp** is read, by default, in the client's I/O thread when the delivery callback fires. `scripts/kafka_producer.py` also offers `--ack-stamp inline`, which timestamps on the calling thread once the publish future resolves, matching what `xadd` does; it requires `--max-inflight 1`.

The two clients therefore differ in whether a second thread stands between the reply and the clock read. They do not differ in something more basic: in both, the acknowledgment timestamp is read only after a thread that was not running has been made runnable and then scheduled.

The consumer's two timestamps, and the asymmetry between the clients: The other two are consumer-side, and until this revision neither this section nor any other said where they are taken. That was a gap in an audit whose whole subject is timestamp placement: every span the paper reports ends at one of them.

- **The receive timestamp** t_{recv} is read when the consumer holds the record. In `scripts/kafka_consumer.py` the client is constructed with a `value_deserializer`, so the payload is parsed inside `poll()` and the parse precedes the timestamp. In `scripts/redis_consumer.py` the timestamp is read first and `json.loads` runs after it.
- **The output timestamp** t_{out} is read once the record is in hand. It is the next statement under Kafka and is separated from t_{recv} by the payload parse under Redis.

So the consumer's own processing-time latency, $t_{\mathrm{out}} - t_{\mathrm{recv}}$, is 290 ns under Kafka and 20,772 ns under Redis — medians over run medians across 5,863 runs, a factor of 72. It never goes negative on any of the 708,505 events, as a chain cannot.

Three consequences, and we state the direction of each because a disclosure without one is an adjective. *First*, the payload parse falls inside $D = t_{\mathrm{recv}} - t_{\mathrm{pub}}$ for Kafka and outside it for Redis, so the transport proxy and D carry one client's parse and not the other's; against Redis's median end-to-end latency of $824\,\mu$s that is 2.5%. *Second*, a longer D produces fewer negative spans, and the broker carrying the extra parse is the one with the *higher* reported rate, so correcting the asymmetry would widen the gap in Table I rather than close it. *Third*, event-time latency contains both parses and is unaffected, which is one more reason the equivalence in S13 is also stated on event-time latency.

We could not fix this in the pipeline where it belongs, because it would require re-running a campaign whose machines are released. What the pipeline now does instead is measure it: `recount_spans.py` carries the span as its fifth column, and a gate fails if the two consumers ever timestamp in places that differ again without the number moving with them.

S16.2. Why one span cannot invert and the other can

The publish timestamp is taken before the message leaves the process, so $t_{\mathrm{pub}} \rightarrow$ emit $\rightarrow$ deliver $\rightarrow t_{\mathrm{recv}}$ is a chain and its ordering holds by construction, however late the timestamping thread was: a late thread makes the span longer, never shorter. That is not an empirical claim about our scheduler, it is a property of the order in which the two operations are written, and it is why the publish-timed span is negative on 0 of 708,505 events under either broker.

The acknowledgment timestamp has no such guarantee. The broker's append precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the record, but those are two downstream branches with no happens-before between them (Supplement S1.3). Both branch timestamps are read after a wakeup, so the error on their difference is two-sided, and the difference goes negative whenever the acknowledgment-side delay exceeds the true interval plus the receive-side delay. The exposure lies in which operand can be late in the shortening direction, and that is fixed by how the span is constructed, not by what is being measured.

S16.3. The second thread is not the mechanism

The harness was built expecting the thread hand-off to be the cause; the comment at `kafka_producer.py` says that Redis "timestamps inline on the calling thread the moment XADD returns, and never pays that wait". The corpus does not bear that out, and the disagreement is informative.

Two committed results bear on it. First, the timestamping campaign (`ec3`) ran Kafka under both settings at matched load, and Table S28 is the outcome: moving from `callback` to `inline` lowers Kafka's median while Redis does not move, shrinking the between-system gap by about a quarter, and a thirty-run replication reproduces it. The hand-off is therefore real, it is worth some tens of microseconds of median inflation, and a comparison that leaves it in place compares two instruments as well as two brokers.

Second, and against the expectation, removing the second thread does not remove the negative spans. Redis, which never has one, inverts on 8.71% of its 348,739 events against Kafka's 8.87% of 359,766; by runs affected Redis is the worse of the two (90% of runs against 65%). A thread returning from a blocking socket read must also be made runnable and then scheduled before it can call the clock, and that wait is charged to the measurement exactly as a callback dispatch would be. What the two clients share is the wakeup; the hand-off is an increment on top of it, not the thing itself.

What we could not test: The clean form of this control would be the negative-span rate for Kafka under `--ack-stamp inline`, compared with the same runs under `callback`. The timestamping campaign retained only its median summary (`docs/results/model/ec3_stamping.csv`); the per-event records that would carry sign-separated counts were not kept, and we do not reconstruct them here. The cross-broker contrast above is the evidence we have, and it is weaker than the within-broker one would have been.

S16.4. Two rivals: the scheduler itself, and the hypervisor

Neither the operating system nor virtualization is the explanation, and it is worth saying why in each case.

The scheduler is not malfunctioning. A base slice with a wakeup-preemption check is a throughput and fairness trade-off, and the arithmetic of Section IV-C is the kernel maintainers' own. A general-purpose scheduler that guaranteed sub-hundred-microsecond wakeup latency to every thread would be a worse general-purpose scheduler, which is why the escapes from it are

opt-in: real-time priority, a preemption-free build, or a pinned busy-polling core. The kernel also already provides the repair — `SO_TIMESTAMPING` returns the time the packet arrived rather than the time a thread got round to asking — and none of the instruments we audited uses it.

Virtualization is bounded rather than excluded, because both testbeds are virtualized: the cloud instances are virtual machines, and the workstation runs its brokers in Docker Desktop containers inside a WSL2 virtual machine on Microsoft's hypervisor. An earlier version of this paragraph said the workstation had no hypervisor beneath it, and drew from that a conclusion the machine cannot support. Two observations disagree with a steal-time account, *steal time* being the time a virtual processor is ready to run but waits while the hypervisor runs another machine: real-time priority is a control inside the virtual machine and collapses the rate 39–$54\times$ (these two matched pairs; all eight are in S12, and Section IV-B of the main text gives 7–$80\times$ across the load sweep), which steal time would not yield to; and the 2–4 ms mode of Section IV-C sits on the base-slice arithmetic of the virtual machine's own kernel rather than on any quantity the hypervisor exposes. Steal time may still contribute on either testbed, and no machine here runs without a hypervisor, so none can show how much.

A third rival, inside the interpreter: The two rivals above are the ones a reader expects. A third is specific to how this harness is built, and the manuscript owes it a paragraph because the mechanism was restated around a thread reaching a CPU.

The timestamping threads are CPython (Section II), and both producers are multithreaded: `redis_producer.py` dispatches publish calls through a thread pool and runs a correction worker beside it. CPython releases the interpreter lock around blocking I/O and takes it back on return, and `now_ns()` is a Python call. So the sequence is not "reply arrives, thread scheduled, clock read" but "reply arrives, thread scheduled, *lock reacquired*, clock read". The extra wait is bounded by the interpreter's switch interval, which our campaigns left at the default and none of them recorded; the platform probe now records it, and `interpreter_serialisation()` is part of every future run's provenance.

Three things follow, and we state them because two of them are uncomfortable. First, the traced estimator cannot see this term: `runqlat` measures `sched_wakeup` to `sched_switch`, and lock acquisition happens after the switch, in userspace. Second, the priority manipulation does not cleanly separate it, because a thread raised to `SCHED_FIFO` also reaches the acquisition queue sooner. Third, magnitude does not separate them either: the default switch interval and the base slice of Section IV-C are both of millisecond order, so neither mechanism is excluded by the size of the delay it would produce. Note what the first point costs us here — the stall spectrum of Section IV-C cannot adjudicate between them, because it is built from the tracepoints that miss one of the two entirely.

What settles it is the direction of the disagreement rather than any of those. If a second serialization sat in series with the run-queue wait, an estimator built from the run-queue wait alone would fall systematically *short* of the observed negative-span rate. Over 3 configurations it does not: the ratios are $0.66, 0.80, 1.02$, straddling unity, with the largest above it. A term big enough to matter would have to hide inside a spread that already brackets the answer. The interpreter lock is therefore present in principle, bounded in practice, and not the mechanism — and the publish-timed span is immune to it entirely, since a timestamp taken before the action bounds its event whatever the interpreter is doing.

S16.5. Why averaging does not repair either mode

For the deletion failure the natural objection is that quantization error falls both sides of the truth, so averaging should recover it. That is exactly right, and it is why the deletion matters: the readings that straddle are the estimator, and HP AN 162-1 makes the point in 1970. But the correspondence stops one step further on, and the paper's concession to that note should not be read as covering the case here. A counter measures a *constant* interval, and which side of a tick a reading falls on is decided by phase, which is independent of the measurand. Our retention is $\min(1, T_{\mathrm{true}}/\tau)$, a function *of the measurand*: the shorter the end-to-end latency, the likelier it is deleted. The surviving population is therefore selected on the quantity being estimated, which a fixed interval cannot be, and averaging over the survivors does not converge on the truth however many are taken. It also cannot produce a percentile at all, and a latency benchmark reports percentiles.

The consequence is worth stating in its own right. For a sub-resolution path every survivor reads exactly one tick, so the reported median is pinned at τ while the truth is T_{true}, and the overstatement is τ/T_{true}, the reciprocal of the retention. A tool of this design penalizes the paths that are fastest, and by the largest factor exactly where it deletes the most.

For the scheduling failure averaging is worse than unhelpful. The error is not symmetric noise but a one-sided lateness on one operand, so it does not cancel; and nothing in the path carries what would be needed to remove it afterwards. The reply does not report when it arrived, the thread cannot know how long it was not running, and the only clock reading in existence is the late one. This is not a bias that can be subtracted but information destroyed at the moment of observation, which is why finer clocks, better synchronization and larger samples all leave it untouched.

What does work follows from that. Timestamp before the action rather than on resumption, so the reading bounds its event. Where the event is not yours to timestamp, attach the timestamp to it — a kernel or hardware receive timestamp, or a broker-assigned one — which is the probe-placement argument in Section VI. Keep a sub-resolution sample as a left-censored observation, since "$T_{\mathrm{true}} < \tau$" is a fact about the measurand and deleting it discards the censoring indicator along with the value. And treat a negative as a gate rather than a datum: it is not a censored observation but the reference timestamp failing, so it belongs in a sign-separated count and in a run-level rejection, never averaged into a distribution.

S16.6. The redesign that does not help

A reader who accepts the mechanism usually proposes the same fix: have the consumer acknowledge back to the producer, so that one clock times a round trip and the two-clock problem disappears. It is the right instinct and it does remove skew, but it does not remove this failure, for two separate reasons.

The first is that it measures the wrong quantity. A producer-side round trip is not the staleness of a record at the consumer, and staleness at the consumer is what a streaming pipeline is judged on. Substituting the one for the other answers an easier question than the one asked.

The second is that the stall does not care how many clocks are involved. Every negative in this corpus was produced on a single clock, and the configuration in which both processes read that one clock is the configuration the audit rejects most, at 62.4%. A round trip still ends with a thread being woken and scheduled before it can read the clock, so it inherits the same late reference timestamp the acknowledgment span has. Collapsing two clocks into one removes a term the corpus already shows is not the cause.

The monotonic clock, named because it is the remedy this section is most often asked about: The other standing proposal is to read `clock_gettime(CLOCK_MONOTONIC)` instead of `CLOCK_REALTIME`. It belongs here rather than in the ordering of remedies of Section VI-C, because it is the correct answer to a question this paper does not ask. A monotonic clock cannot step or slew, so it removes the failure a reader has usually met before: a reading that moves backwards because the synchronization daemon corrected the wall clock underneath it. Neither mode we report is that failure. The scheduling mode is a delay between the event and the read, and a monotonic clock is read by a thread that has to be scheduled first exactly as the wall clock is; the deletion mode is an admission condition applied to the value, and a monotonic clock changes which value arrives, not which values are kept. It also costs something: a monotonic clock has an arbitrary per-boot origin, so two hosts' readings cannot be differenced at all, and the end-to-end span is a difference across two hosts. The demonstration is in the corpus already. Kafka's bundled `EndToEndLatency` is one thread on one monotonic clock with a synchronous publish call, immune to the scheduling mode by construction — and it loses the measurement anyway, to an integer division (Section S25). A clock choice is not a measurement design.

S16.7. Where the repairs belong

No layer here misbehaves, which is why the fix is not obvious from inside any one of them. The kernel schedules as designed, the clients return as soon as they can, the brokers acknowledge correctly. The benchmark is the only party that assumes something none of them promised: that observing an event tells you when the event happened.

- **Operating system:** nothing is owed. The escape hatches and `SO_TIMESTAMPING` already exist.
- **Kafka:** `record.timestamp()` is a millisecond wire field, so anything built on it inherits a millisecond resolution and must carry its own finer timestamp in the payload to escape. Documenting that, or exposing a finer causally attached timestamp, is the ask.
- **Redis:** the `XADD` entry identifier embeds a server-clock, millisecond-floored time. One documented sentence saying it is not comparable with a client clock would have prevented the driver defect of Section VI-D outright.
- **Benchmark authors:** timestamp before the action; count discards sign-separated; report retention, timestamp resolution and synchronization state; never difference a server-assigned timestamp against a client clock without saying so; randomize the publish instants (Poisson sampling); and record the achieved rate.

S16.8. The scheduler constants, derived

Section IV-C states the base slice and moves on; this is where it comes from. The constants are *derived* from the published kernel package and the campaign's own data rather than measured, which is weaker evidence than a measurement and is reported as such.

The core count is recovered from the experiment rather than assumed: each geometry condition loads k cores and records the achieved utilization, so k/ρ recovers 8 six times over with a spread of 0.05. The kernel's own rule then gives the base slice, $4 \times 0.75\,\mathrm{ms} = 2.8$ ms — the maintainers' arithmetic, "0.75 msec * (1 + ilog(ncpus)) ... 3.00 for 8 cpus and above" [S12]. It is specific to this kernel: the constant became 0.70 ms in v6.15, after our 6.8. The build sets `CONFIG_HZ`$=1000$, a 1 ms tick.

With run-to-parity a woken thread that loses the wakeup-preemption check waits for the incumbent to exhaust that slice — what is left of it, since the wake-up falls at no particular moment of the incumbent's slice (S16.10) — and the expiry is noticed at the tick. That tick is also why the waits end in a band a tick wide and not at the slice itself: the same posting notes a task "can only wait until the next tick to consider that it has reached its deadline, and will run 1ms longer".

No scheduler tunable is written anywhere in the archived campaign, which is why the derived value is the one that applied. It is strong evidence rather than proof.

S16.9. The displacement recovery, distribution by distribution

Section VI-B reports the recovery's accuracy as four numbers and one comparison: that the data cannot tell apart the groups the check accepts and those it rejects. This section gives the distributions behind them, pooled and broker by broker, and shows that the comparison holds only inside each broker. It was rewritten on 15 October 2026, the day we found that the errors it reported came from an estimator no text described; once the estimator was corrected, the argument the section had made no longer held.

The recovery is the identity read backwards. The end-to-end latency is the reported span plus the publish latency, $D = S + A$ for every message, so a harness that records its own publish latency can report the end-to-end latency it failed to measure. A harness that keeps only summaries recovers a median: the median of S plus the median of A, whose error is its distance from the median of D, as a percentage of the median of D. Because $D = S + A$ holds message by message, that error is the whole gap between the median of a sum and the sum of two medians. There is one error per group of runs, read from `recovery_err_us` and `median_D_us` in `docs/results/span_symmetry.csv`; a group is one condition's runs that the consistency check of Section II-C accepted, or those it rejected. Until 15 October 2026 the recovery used the center of symmetry of S's histogram in place of its median, and read that center, the median of A and the median of D off the campaign's $50\,\mu$s histogram grid. Every median is now taken over the messages themselves, at the nanosecond the timestamps carry.

The table gives the corrected errors, pooled over the brokers and then for each broker.

Groups	Verdict	n	Median	Q3	Worst
Both brokers	accepted	42	3.0%	13%	35%
	rejected	28	12.7%	19%	31%
Kafka	all	32	0.4%	1.3%	5.1%
	accepted	24	0.4%	1.3%	5.1%
	rejected	8	0.6%	0.7%	1.3%
Redis	all	38	14.6%	20%	35%
	accepted	18	13.3%	19%	35%
	rejected	20	16.4%	20%	31%

Verdict is the check's verdict on the group's runs. *Median*, *Q3* and *Worst* are the median, the upper quartile and the largest of the groups' errors, in percent of each group's median end-to-end latency.

No group recovers exactly, neither among the 42 accepted (0%; Wilson 95%: 0–8%) nor among the 28 rejected (0%; Wilson 95%: 0–12%). The exact recoveries this section used to count were the grid's. On the grid every error was a whole number of $50\,\mu$s steps, so an error smaller than a step could print as zero, and for 14 accepted and 6 rejected groups it did. Every one of them was Kafka's. At native resolution the largest of their errors is $39\,\mu$s, less than a step, and no Redis group printed as exact. The pile at zero on which this section's old argument turned was Kafka's groups, read at a resolution too coarse to tell their errors from zero.

Pooled over the brokers, the rejected groups recover worse. The Hodges–Lehmann shift over all 1,176 pairwise differences, rejected minus accepted, is 3.7 points (95% percentile bootstrap: 0.2 to 11.3), an interval that excludes zero. On the grid the same interval contained zero, and this section argued that the gap between the two medians was the pile at zero and nothing else. At native resolution there is no pile, so pooled, the comparison Section VI-B draws does not hold. Taken at face value, the shift would say that the recovery works less well exactly where it is needed.

The shift is the brokers' mix, not the check's. The two brokers recover to very different accuracies, a median error of 0.4% over Kafka's 32 groups against 14.6% over Redis's 38, and the two verdicts hold different mixes of them: 24 of the 42 accepted groups are Kafka's, but only 8 of the 28 rejected ones. Pooling sets mostly Redis groups against mostly Kafka ones, so the pooled shift measures which broker a group came from. Compared inside each broker, rejected minus accepted, the shift is 0.16 points on Kafka (95% percentile bootstrap: -0.17 to 0.44) and 1.4 points on Redis (95% percentile bootstrap: -6.5 to 6.5). Both intervals contain zero: within either broker, no shift can be detected between the groups the check accepts and those it rejects. Figure S11 draws the three comparisons.

Undetected is not absent, and this section claims no equivalence. The 90% bootstrap intervals on the two shifts inside the brokers would support equivalence only against margins of 0.39 points on Kafka and 5.9 on Redis: about the whole of Kafka's median error, and two-fifths of Redis's. Section VI-D calls two brokers equivalent only after a TOST against a stated margin, and no reader would accept a margin that large a share of the quantity compared.

A shift cannot carry that sentence on its own, because two distributions can sit at zero offset and still differ in shape. So the distributional claim is tested distributionally, broker by broker. The two-sample Kolmogorov–Smirnov statistic between the verdicts is $D = 0.42$ on Kafka, at $p = 0.22$, and $D = 0.26$ on Redis, at $p = 0.44$, each p over 20,000 permutations of the broker's groups. Permutation rather than the asymptotic Kolmogorov distribution, which is a large-sample limit, and Kafka has only 8 rejected groups. Neither test detects a difference in shape, and at these sample sizes that is no evidence that there is none, which is why the margins above are stated. The tails in the table agree: on Kafka the rejected groups' upper quartile

and worst error are the smaller, and on Redis the two upper quartiles lie within a point of each other and the worst rejected group lies below the worst accepted one.

So the reading the second check of Table III needs survives, on a narrower footing than before: the recovery holds where the check rejects as well as where it passes, read broker by broker. It does not hold equally well on both brokers. The errors divide by broker, not by verdict, so a recovery validated on one broker is not validated on another: Kafka's median of 0.4% is no guide to Redis's 14.6%. That result is not free. It is the whole reason both verdicts were measured — a remedy validated only where the check already passes has been validated on the population that did not need it — and had the rejected groups been dropped as spoiled, neither reading could have been told from the other.

A Hodges–Lehmann shift rather than a rank test's p-value, for three reasons. It is a shift in the units the sentence is about, so a reader can act on its size. The two verdicts' distributions cross on both brokers, so a test built on ranks is weak here by construction and a large p would have carried no information about whether the populations agree. And Section VI-D already quotes a Hodges–Lehmann shift for the broker equivalence: one statistic, on the same footing in both places and each at the level its question calls for — a 90% interval beside the two one-sided tests of equivalence, a 95% interval for this two-sided estimate.

The median is still not a bound, which is the older lesson this table also serves. A reader who takes Redis's 14.6% as a guarantee will find a quarter of its groups at 20% or worse and one at 35%. On Kafka the same reading fails by less, 1.3% at the upper quartile and 5.1% at worst.

S16.10. How late: the rest of a slice, timed acknowledgment by acknowledgment

S16.8 derives where the scheduler's waits end, a slice and a tick after they begin at most. How long they last is a separate question, and it decides the shape of the negative-span rate against the end-to-end latency: a thread that waits out a whole slice keeps the rate level until the end-to-end latency passes the slice, and one that waits out what is left of a slice lets it fall from the start. A later campaign asked the question in advance, and the answer is the second: the wait that decides a negative span is what is left of another task's slice, anything up to a slice, and not a whole one.

The law, as registered: After the campaigns this paper reports, we wrote down a law for the negative-span rate and froze it, with the programs that would judge it, before its runs; the plan and every program are in the repository (`freezes/28-experiment-plan`). It ran on three pairs of cloud machines, two with x86 processors and one with Arm, in 8,102 runs over 156 campaigns in September 2026. Each run held the end-to-end latency near a set value T by delaying the receiver's traffic, set the scheduler's base slice s, and ran on a kernel whose tick h was 1, 4 or 10 ms. The acknowledgment's reply reaches the producer a time g after the publish call and wakes the thread that records its timestamp, which then waits a time W for a CPU, so a reading is negative when $W > T - g$. The law took the slice as a whole: a thread kept waiting waits out a slice and then up to a tick, so the rate stays at its plateau $c + p$ while the end-to-end latency is shorter than the slice, falls across one tick, and rests on its floor c beyond it,

$$r(T) = c + p \cdot \begin{cases} 1, & T \le s, \\ (s + h - T)/h, & s < T < s + h, \\ 0, & T \ge s + h. \end{cases} \tag{S4}$$

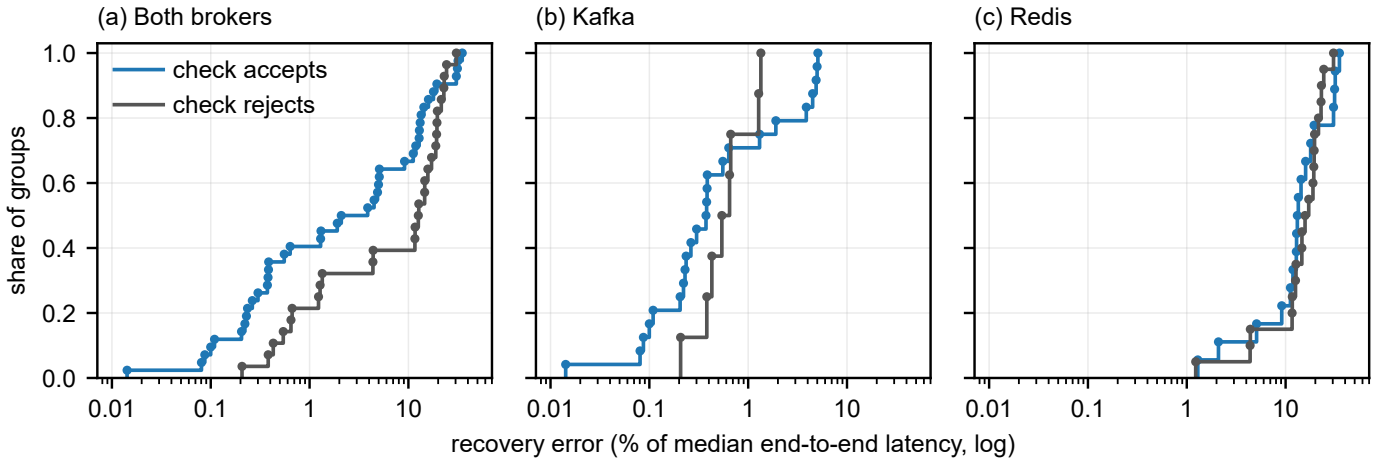


Fig. S11. **Within each broker, no shift between the verdicts can be detected.** Empirical distribution functions of the recovery error, on a logarithmic axis, over the groups the check accepts and those it rejects: (a) both brokers, 42 and 28 groups; (b) Kafka, 24 and 8; (c) Redis, 18 and 20. Each mark is one group. Pooled, the rejected curve rises later, because most rejected groups are Redis's and Redis's errors are the larger; inside each broker the two curves cross.

TABLE S10
ONLY THE REST-OF-A-SLICE SHAPE SPLITS THE TWO READINGS AS THE DATA DO. THE SLOPE OF THE FALL'S HALFWAY POINT ON THE SLICE, WHICH THE REGISTERED LAW PUT BETWEEN 0.8 AND 1.2, ON THE FREE READING AND ON THE FITTED SHAPE, AS THE FROZEN PROGRAMS RETURN IT FROM THE MEASURED RATES, FROM EQUATION S4 PUT IN PLACE OF EVERY RUN'S RATE, AND FROM EQUATION S5 PUT IN THE SAME PLACE.

		Measured		Eq. S4		Eq. S5	
Pair	Broker	free	fitted	free	fitted	free	fitted
first x86	Kafka	0.758	0.230	0.997	0.998	0.922	0.519
first x86	Redis	1.007	0.662	1.000	0.999	0.930	0.680
Arm	Kafka	0.830	0.285	0.889	0.908	0.802	0.525
Arm	Redis	1.153	0.798	1.000	1.005	0.954	0.723

The rate does not hold its plateau: At half the slice the negative-span rate is 1.45 to 3.94 times its value at 0.9 of the slice, in each of the ten slice and broker combinations whose runs reach half the slice, six on the first x86 pair and four on the Arm pair. Equation S4 puts the two at one level.

The wait, timed: Block L9 of the registered law campaign (Part V; A9 in its plan) ran on all three pairs at a 3 ms slice and a 1 ms tick, at 75% and 88% load, on both brokers. In a random half of its runs it recorded every scheduling event of every `python3` thread, and the clients named the thread that records each acknowledgment's timestamp. From those records the thread's wait for a CPU before each acknowledgment's timestamp, counted from the later of its last wake-up and its previous timestamp, is read for every acknowledgment: 1,516,342 of them in 304 runs. Of the acknowledgments whose thread waited more than 1 ms, 58.4 to 65.1% were still waiting at 2 ms, 24.3 to 35.8% at 3 ms and 3.3 to 6.8% past 4 ms, the ranges running over the twelve pair, broker and load parts (Figure S12a). A whole-slice wait keeps every one of them waiting to 3 ms. A wait for the rest of the slice keeps 60% to 2 ms, 20% to 3 ms and none past 4 ms: the thread waits out what is left of the running task's slice, R, uniform on $[0, s]$ because its wake-up falls at no particular moment of that slice, and then up to a tick, U, uniform on $[0, h]$, so that for $h \le s$

$$G(x) = \Pr[R + U > x] = \begin{cases} 1 - x^2/(2sh), & 0 \le x \le h, \\ 1 - (x - h/2)/s, & h \le x \le s, \\ (s + h - x)^2/(2sh), & s \le x \le s + h, \\ 0, & x \ge s + h. \end{cases} \tag{S5}$$

Given one free scale each, Equation S5 leaves a root mean square of 0.029 to 0.059 of that scale over the twelve parts, and the whole-slice shape 0.316 to 0.392. Read against each run's own margin $T - g$, the share of its acknowledgments whose wait outlasted the margin predicts its negative-span rate with nothing fitted, and the registered check of that prediction met its rule on four of six pair and broker parts, with medians of prediction over measurement of 1.11 to 1.30. The recording itself raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, so these figures describe the machines with the recording on; the prediction and the rate it is held against come from the same runs.

Why a preemption and a wake-up differ: Waits that begin with a preemption do gather at the slice. Over every `python3` thread's waits longer than 0.25 ms in the same records, the density per millisecond between the slice and a tick past it, over the density between 1 ms and the slice, is 1.57 to 1.94 for waits begun by a preemption, on all three pairs, and 0.68 to 0.73 for waits begun by a wake-up on the two x86 pairs, 1.13 and 1.21 on the Arm pair (Figure S12b). A thread taken off its CPU for a task starting its slice waits out the whole of it; a thread woken while another task holds the CPU waits out what is left. The thread that timestamps an acknowledgment sleeps until the acknowledgment's reply wakes it (S16.3), so its wait is the second kind. S16.8's derivation holds for where the waits end, a slice and a tick after they begin at most, and not for how long they usually last.

What it did to the law's verdicts: Every prediction about where the fall sits was judged by frozen programs on two readings: a free one, a curve required only never to rise, whose halfway point lies between the level at 0.9 of the slice and the floor; and the law's own shape, a plateau, a straight fall and a floor fitted to all eight end-to-end latencies. A rate that falls from the start pulls a fitted plateau's fall below the slice and moves it less than the slice moves, which is the split Table S10 shows. Put in place of every run's rate at its own end-to-end latency, slice and tick, and judged by the same programs, Equation S4 gives the two readings nearly the same slope, near one, and Equation S5 gives the split. This was read after the verdicts and changes none of them: the registered predictions that the halfway point follows the slice, on the first x86 pair and on the Arm pair, are reported as the frozen programs judged them, not confirmed.

What it changes in this paper: Section IV-C states which wait the timestamping thread meets, and Figure S4 draws the second state with the shape of Equation S5. The traced mode of Section IV-C is consistent with that shape: a wait spread as Equation S5 spreads it at a 3 ms slice and a 1 ms tick puts half its mass in the octave from 2 to 4 ms, the octave that holds the mode, and waits begun by a preemption add their gathering at the slice to the same octave. For the thread that timestamps an acknowledgment, the latency factor G of Equation S2 now has a measured form with no parameter the kernel does not set, at a margin $T - g$ rather than T_{true}; the waiting probability p in front of it is still not predicted.

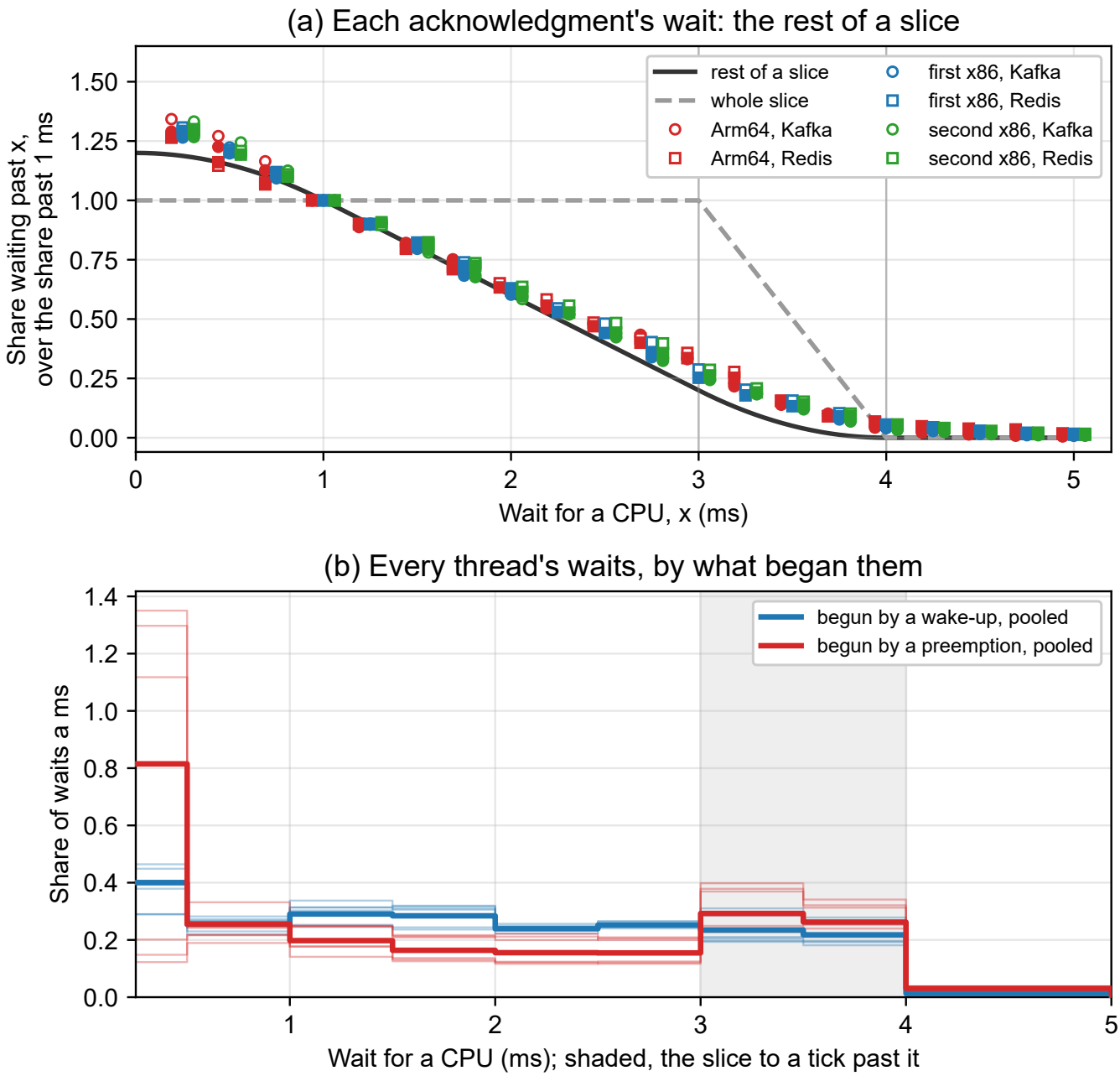


Fig. S12. **Each acknowledgment waits out the rest of a slice, not a whole one.** (a) For every acknowledgment in L9's recorded runs on the three pairs, the share whose timestamping thread was still waiting for a CPU at each wait x, over the share still waiting at 1 ms, for each pair, broker and load (open marks 75% load, filled 88%; Arm64 is the Arm pair; each pair's marks are set a little to one side of the wait they are read at); lines, the rest-of-a-slice shape of Equation S5 (solid) and the whole-slice shape of Equation S4 (dashed), at the 3 ms slice and 1 ms tick. (b) Every `python3` thread's waits over 0.25 ms, as a share per millisecond in each bin, by what began them, each pair and broker thin and the six pooled bold; shaded, the slice to a tick past it.

S16.11. Two faults in the law campaign's judges, and the runs a stalled disk held

Reading the law campaign's results after its verdicts found two places where a frozen program departs from the plan it was written for, and one fault of the machines. Each is recorded beside the verdicts it touches, and no verdict is changed.

P2, P2b and P2c read two slices as one curve: The campaign on the tick built three kernels differing only in the tick, 1, 4 and 10 ms, and ran two slices, 1.5 and 3 ms, on each. Equation S4 makes the fall from 90% to 10% of the plateau as wide as a tick, so P2 predicted it 2.5 to 5.5 times wider at the 4 ms tick than at 1 ms, P2b 6 to 14 times wider at 10 ms, and P2c a width proportional to the tick, a straight line through zero with a slope of 0.8 to 1.2. Their programs group the runs by tick alone, so each width they read runs from one slice's fall to the other's. With Equation S4 in place of every run's rate, the P2 program returns 1.98 on the free reading and 1.57 on the fitted shape for Redis, below its band. Read one curve per slice, the same formula gives 4.0 to 5.2 for P2 and 9.8 to 12.0 for P2b, inside both bands, at both of Redis's slices and at Kafka's

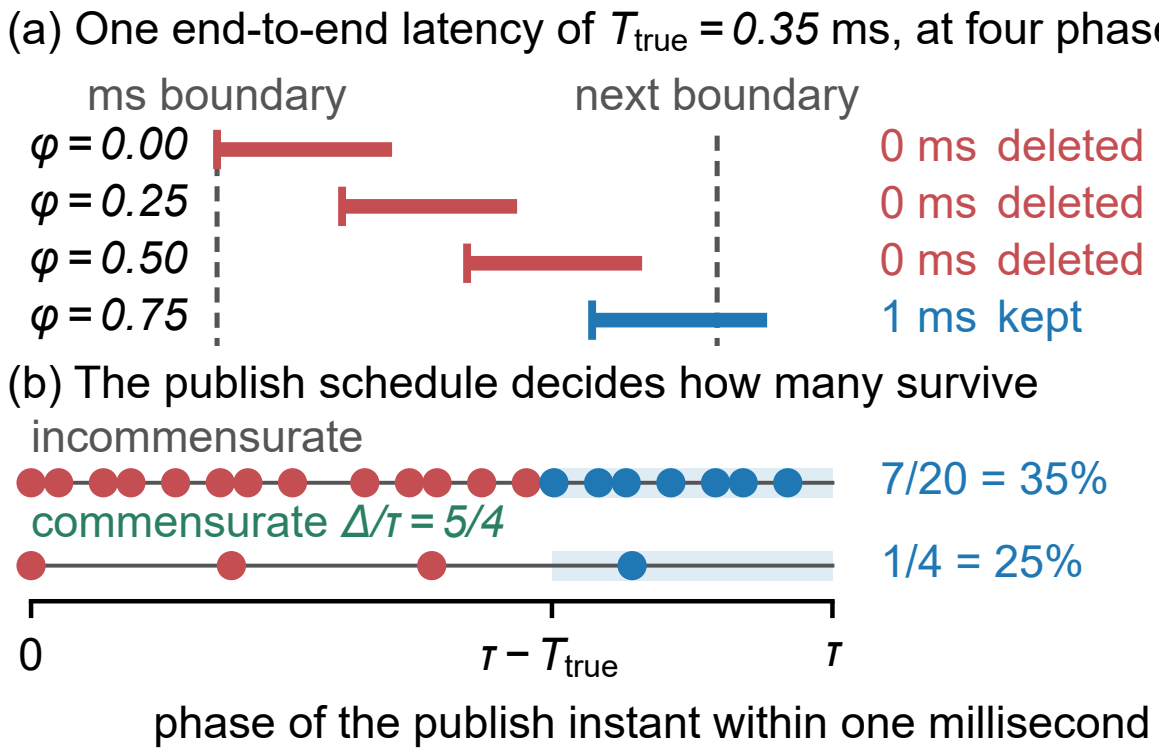


Fig. S13. **Why a sample disappears, and why the survivors are not a sample.** (a) One end-to-end latency of $T_{\mathrm{true}} = 0.35$ ms at four phases of one millisecond. Three finish inside it, so both timestamps round alike, the difference is exactly zero, and the admission condition removes them; the fourth crosses the boundary and is recorded as one whole millisecond — which is why the corpus prints 1.0 and 2.0 ms and nothing between. (b) An end-to-end latency survives from phase $\tau - T_{\mathrm{true}}$ onward — the shaded band — so uniform phases retain T_{true}/τ (Equation 6); a commensurate producer visits only q and retains a count out of q, one of the values Equation S3 allows. T_{true} here is illustrative and off-vertex by choice: at 0.5 ms the two answers coincide.

3 ms. Kafka's 1.5 ms slice is the exception: no trip can be shorter than the client's own with nothing added, and most of that slice's fall lies below it, as the 2.25 ms slice's did in L1. P2's and P2b's verdicts, not confirmed on either broker, therefore carry no evidence about the law either way; P2c reads the same pooled widths, and its verdicts, not confirmed on Kafka and confirmed on Redis, rest on them too. Read one slice at a time after the fact, the width is proportional to the tick at the 1.5 ms slice, $0.92h - 0.25$ ms on the free reading and $1.15h - 0.61$ ms on the fitted shape for Redis and $1.04h + 0.04$ ms fitted for Kafka, and grows less than in proportion at 3 ms, where the rest of the slice spreads the fall as the tick does: Equation S5 gives $0.87h + 1.62$ ms for Redis there, against the measured $0.81h + 1.48$ ms.

P4 makes no claim on Redis: P4 predicted that real-time priority on the timestamping threads cuts the plateau at least five times wherever the normal-priority plateau is at least 2%. Redis's normal-priority plateau in that campaign was 0.69%, so P4 makes no claim there, and the plan's rule reports such a part out of reach; the program words it "not confirmed", and the prediction as a whole with it. On Kafka, whose plateau was 4.05%, priority took the rate to zero and P4 is confirmed. Read by the plan's rule, P4 is confirmed on the one broker it makes a claim on; the frozen answer, "not confirmed", is the one reported.

The runs a stalled disk held: In 222 of the 8,102 runs a message arrived more than 150 ms late after the warm-up, 112 and 109 of them on the two x86 pairs and one on the Arm pair, and the longest pause lasted 20.6 s. On the receiving side the consumer and an unrelated process on the same machine stopped writing together while the processors waited on the disk; at the broker, Kafka's only, the broker failed to reach its own controller. The long receiving-side pauses keep the clock's hour, 35 of 68 in its first quarter, and not the hour counted from each machine's boot, and no long pause on one x86 machine began within 5 s of one on the other; what held the disks is not identified. Judged again without those runs, after the fact, every verdict stands but one: P3a on the first x86 pair, whose fitted interval ends 0.0026 ms outside its ±0.25 ms bar with them and 0.0037 ms inside it without them. The verdicts on all the runs are the ones reported.

S17. The 1970 counter note, line by line, and the grid geometry it explains

Figure S13 draws the identity this section traces: one end-to-end latency at four phases of a tick, three of them rounding to a difference of zero and deleted, and beneath it the phase diagram that decides how many of a run's publishes survive.

Section V-E concedes the arithmetic of the deletion failure to Hewlett-Packard's Application Note 162-1 [S13] and states what remains ours. This is the correspondence in full, so the concession can be checked rather than taken on trust.

- **The retention identity.** For an interval of $Q + F$ clock periods, AN 162-1 gives $\Pr[R = Q{+}1] = F$: the fraction of readings landing on the upper of the two straddling counts is the fractional part itself. That is the retention law of Section V.
- **The class of a repetition rate.** Its *class* is the denominator of the ratio of repetition rate to clock rate in lowest terms, under the same co-prime condition that fixes our q, and it determines how many distinct readings the counter can produce.
- **The ceiling on averaging.** The gain in resolution available from averaging N readings is bounded by a factor $1/q$ and not by $1/\sqrt{N}$, which is the quantization ceiling of Section V-D. AN 162-1 names $q = 1$ as the corner where averaging cannot help at all — the case a paced producer on a round interval walks straight into.
- **The symptom.** It tells the operator what to look for: a counter reading that appears to hang on a round number while the input is known to vary.
- **The cure.** Phase jitter on the repetition rate, so the samples stop landing on the same points of the clock's cycle. We arrived at it independently and recommend it in Section VI.
- **The mechanism, and a patent on the cure.** Hewlett-Packard did not only describe the symptom; it patented the remedy. US 3,938,042, filed in 1974 and granted in 1976, varies the phase of the counter's own time base "to destroy coherence between the clock signal and an unknown signal thereby allowing statistical averaging to take place" [S14], and states the failure it repairs in terms this paper could have used: a counter that "appears to hang up on some value … a reading that is an integral multiple of the clock period". That is the corpus of Section V, described in 1976. The one substitution we make to a fifty-year-old remedy is forced: the time base belongs to the operating system, so a benchmark cannot randomize its phase and must randomize the publish instants instead.

The one thing the note does not contain is the failure this paper reports, because its instrument keeps both readings. Retention is the estimator there; deleting the population that carries the fractional part is what converts a solved metrology problem back into an open one.

When dither works, which is a question with an answer: Our remedy is to randomize the publish instants, a form of dither, and a reader from signal processing will ask under what conditions dither actually recovers the underlying quantity. It is a fair question and the literature has answered it. For quantized Gaussian measurements subtractive dither helps when the ratio of the noise's standard deviation to the quantization interval is roughly *below* one third; above that it costs more than it buys [S15]. The condition is one-sided. Their second threshold does have a lower bound, but it decides which *estimator* wins inside that regime rather than whether dither helps at all, and we do not lean on it. We name the result rather than apply it, because our dither is not that dither. Theirs adds noise to the *value* being quantized; ours randomizes the *phase* of the sampling instant against the tick, which is RFC 3432's construction [S16] and is what breaks the commensurability of Section V-D. The two coincide only in the uniform-phase limit that Equation 6 already assumes. The neighboring result is worth the reader's attention anyway: it is the closest thing to a proof that the recommendation has a domain of validity rather than being folklore.

PART III. APPARATUS, DATA AND TABLES

This part is reference material, meant to be looked things up in rather than read through. Section S18 maps every experiment to the claim it supports, and Section S19 states the threats to the scheduling attribution with what rules each one out. Sections S20 to S23 describe the workload, every setting that can move a latency number, the harness and the transfer procedure. Sections S24 and S25 give the campaign ledger's schema and the source evidence behind the count of tools that dispose of a sample silently, with a note on every tool read at source and an account of every tool run. Section S26 collects every table.

S18. EACH CAMPAIGN MOVES ONE VARIABLE, AND THE AUDIT THRESHOLD CHANGES NO CONCLUSION

Every campaign here manipulates one variable and holds the others fixed, so each settles one claim and no claim rests on a single cell. Figure S14 maps the campaigns onto the claims, Table S11 gives the runs behind each, and Figure S15 shows that the audit threshold changes the run count without changing any conclusion.

S18.1. What the audit gate selects, bounded

Three artifacts bound the selection the audit gate introduces, each recomputable by a committed script. Table S12 applies the audit gate to the powered transport campaigns and bounds the selection the gate introduces (`powered_gate_sensitivity.py`); the condition-level threshold sweep behind Section II-C of the main text is `first_result_threshold_sweep.csv` (no first-result cell fully usable at any threshold up to 20%; best cell $23/30$); the traced survival slopes that once backed an effective-exponent statement, since withdrawn (S1.6), are `traced_tail_slope.csv` (windowed log–log index 0.332 over 0.25–2 ms, 4.128 over 4–32 ms). That last artifact is retained for the record but is *superseded*: the slopes in it are least-squares fits through nested survival points with no interval, and S8 explains why they do not estimate what we claimed they estimated. The estimators that replace them are in `scripts/tail_index_traced.py`.

Campaign	Manipulated	Held fixed	What it settles
E1 concurrency	feeds *N*: 1, 9, 10, 12	rate, host, plan set	the original question: does broker choice matter?
E-B delay sweep	injected delay 0-50 ms	load, feeds	H1 effect size (confounded; negative only)
E-A / E-A3 load sweep	background load 0-12	rate, feeds, plan	H2 utilization, H10 mixture, F_Δ recovery
E-A2 process count	processes at fixed rate	aggregate event rate	H4 oversubscription
E-C3 / E-C4 timestamping	ack timestamp: callback vs inline	load, in-flight, backend	H3 asymmetry (replicated)
window sweep	observation window 60-600 s	rate, host, match	start-up cost vs per-event constant
transport (two runs)	feeds, powered	verified real-time rate	broker transport, equivalence + shift
E-A5/A5b/A7 timestamping priority	SCHED_FIFO on the timestamping threads	utilization, to 0.003	scheduling, not utilization (8 pairs, 7-80×)
E-A6 / E-A6b load geometry	cores free vs all duty-cycled	achieved utilization	utilization is not the variable (2.07×, 2.02×, at rho 0.7531)
E-A9 run-queue trace	nothing: observation only	load, priority setting	P(stall > T_{true}) predicts the rate, unfitted
E-A10 transport sweep	payload size; T_{true} 74×	load, hosts, code path	other side of the inequality; exponent 0.33, not settled
E-A8 co-location	broker on the driver	utilization, to 0.003	nothing: transport did not move, so it is withheld
OMB external	instrumented discard counter	our broker, under load	the exposure is not ours alone

Every claim of these campaigns traces to one row. No row both generates and tests the same hypothesis, and a row that settled nothing says so.

Fig. S14. **One variable per campaign, one claim per row.** Each campaign holds the others fixed. The delay sweep is marked confounded because the injected delay is common-mode and cancels in the transport proxy (S3.1), so it is reported as a negative result only.

S19. Threats to the scheduling attribution, and what rules them out

The mode count is checked against the binning, because it has to be: A count of local maxima read off a histogram belongs jointly to the counts and to the bin edges, and this paper argues that instruments impose structure which is then reported as a property of the system. Figure IV-C reports local maxima of a `bpftrace` $\log_2$ histogram, and those bucket edges came from the tool exactly as the millisecond resolution of Section V came from the benchmark. The objection applies to us.

The usual answer is unavailable and will stay unavailable: `runqlat.txt` holds the tool's summary and **the per-event stall durations were not retained**, so the raw data cannot be rebinned now or later. What the archived histogram does support is coarsening. Merging adjacent buckets doubles the bin width, and the merge can be phased two ways — pairing from the first bucket, or from the second — which puts the edges in different places. A mode that survives both phasings is not sitting inside one bucket by luck.

It survives. 3 maxima on the tool's own 16 buckets, 3 again under both phasings at $2\times$ the bin width, and fewer at $4\times$. The claim therefore holds to within a factor of 2 in bin width and is resolution-limited beyond it — a weaker statement than we could have made carelessly, and a stronger one than a disclaimer. `scripts/stall_mode_robustness.py` recomputes

TABLE S11
WHAT EACH CLAIM RESTS ON. H1, H3, H4, THE BROKER TRANSPORT COMPARISON AND THE START-UP WITHDRAWAL ARE CLAIMED ONLY IN THIS DOCUMENT NOW, AND E1 IS HISTORICAL; THEY STAY HERE WITH THE CAMPAIGNS THAT TESTED THEM. WHERE THE AUDIT GATE DISCARDS RUNS, THE SELECTION IS BOUNDED: IN E1 BY THE WORST CASE OF TABLE S21, WHICH HOLDS ONLY WHILE RETENTION EXCEEDS ONE HALF, AND IN THE BROKER-TRANSPORT CAMPAIGNS, WHICH KEEP 25.0–53.3% OF THEIR REDIS RUNS, BY RESTORING THE CONDEMNED RUNS, WHOSE VALUES SURVIVE (TABLE S12).

Claim	Campaign	Runs per cell
Audit rejection rates	all corpora	2,266 runs total
H1 effect size	co-located vs network	15 per delay level
H2 utilization shape	E-A, E-A3	25 per load level
H4 process count	E-A2	15 per level
H3 asymmetry	E-C3, replicated E-C4	10, then 30 per cell
Mixture structure	E-A3, nine levels	25 per level
Scheduling, not utilization	E-A5, E-A5b, E-A7	25 per configuration
Load geometry	E-A6, both geometries	25 per configuration
Traced run-queue tail	E-A9	25 per configuration
T_{true} sweep	E-A10, four payloads	25 per level
Broker transport	powered, replicated	15, then 8 per cell
Start-up withdrawal	window sweep, twice	3 per window
Historical: E1	E1	8–36, *52.8–100% retained*

TABLE S12
AUDIT GATE APPLIED TO THE POWERED TRANSPORT CAMPAIGNS. "FLIP V^*" IS THE SMALLEST VALUE AT WHICH IMPUTING EVERY CONDEMNED REDIS RUN AT V^* WOULD FLIP THE HODGES–LEHMANN SHIFT'S SIGN; THE CONDEMNED RUNS' OBSERVED MEDIANS CENTER ON 0.10–0.12 MS.

Campaign	N	Redis ret. (%)	HL all	HL gated	Δ	Flip V^*
powered	1	53.3	0.4095	0.4084	-0.0011	—
powered	9	43.7	0.4181	0.4156	-0.0025	0.60
powered	12	37.0	0.4197	0.4170	-0.0027	0.60
replication	1	25.0	0.3969	0.3807	-0.0162	0.55
replication	9	45.8	0.4120	0.4137	$+0.0017$	0.60
replication	12	39.8	0.4132	0.4157	$+0.0025$	0.55

it, and the suite fails if the committed record and the trace disagree.

Threats to validity (full)

Construct validity: the check may not measure what we claim: A negative span shows that the acknowledgment cannot serve as the origin of an end-to-end latency; attributing that to scheduling is an inference. Section VIII of the main text lists each rival and how it is eliminated, clock skew among them: the configurations the check rejects run on one clock by construction. The workstation testbed enters only through the audit, and no attribution rests on it.

Coarse kernel timestamp granularity would also produce negative spans, and that rival makes a different prediction we can test. Quantization acts on each event independently, so its negative spans would be scattered at random through a run; a descheduling event, by contrast, makes a *consecutive run* of events wake late together. A Wald–Wolfowitz runs test on the sequence of span signs separates the two: independence gives $z \approx 0$, clustering $z \ll 0$. Across conditions the negative spans are strongly clustered — median $z = -6.9$ at the co-located floor, -6.8 idle, -5.1 saturated — and the clustering is present even with no background load, so it is intrinsic to the timestamping path rather than induced by contention. This is the signature of a shared scheduling delay, not of independent quantization.

Internal validity: the audit selects: The check rejects unequally between configurations, so the surviving corpus is not a random sample of the measured one. This is the most serious internal threat, and we bound it rather than argue it away, in the way each corpus allows. E1's rejected values are lost, so Table S21 imputes every rejected Redis run at the largest value its own cell allows. That bound holds only while retention exceeds one half, and our tightest cell sits 2.8 points above that line. The powered transport campaigns, on which the equivalence of Section VI-D of the main text rests, are below it: the gate keeps 25.0–53.3% of their Redis runs, under one half in five of the six cells (Table S12). Their condemned runs' values survive, so S18.1 computes the selection rather than bounding it: restoring those runs moves the Hodges–Lehmann shift by at most 0.016 ms, and reversing its sign would take every one of them at 0.55–0.60 ms, about five times the 0.10–0.12 ms at which their own medians center. In the sixth cell no value reverses it.

Internal validity: the cloud's own noise: The mechanism campaigns ran on Oracle Cloud instances, and run-to-run variability there is a rival explanation for any effect they show. It is too small to be this one. Over more than three months of a stream-processing benchmark on another public cloud, Henning et al. [S17] measured a coefficient of variation below

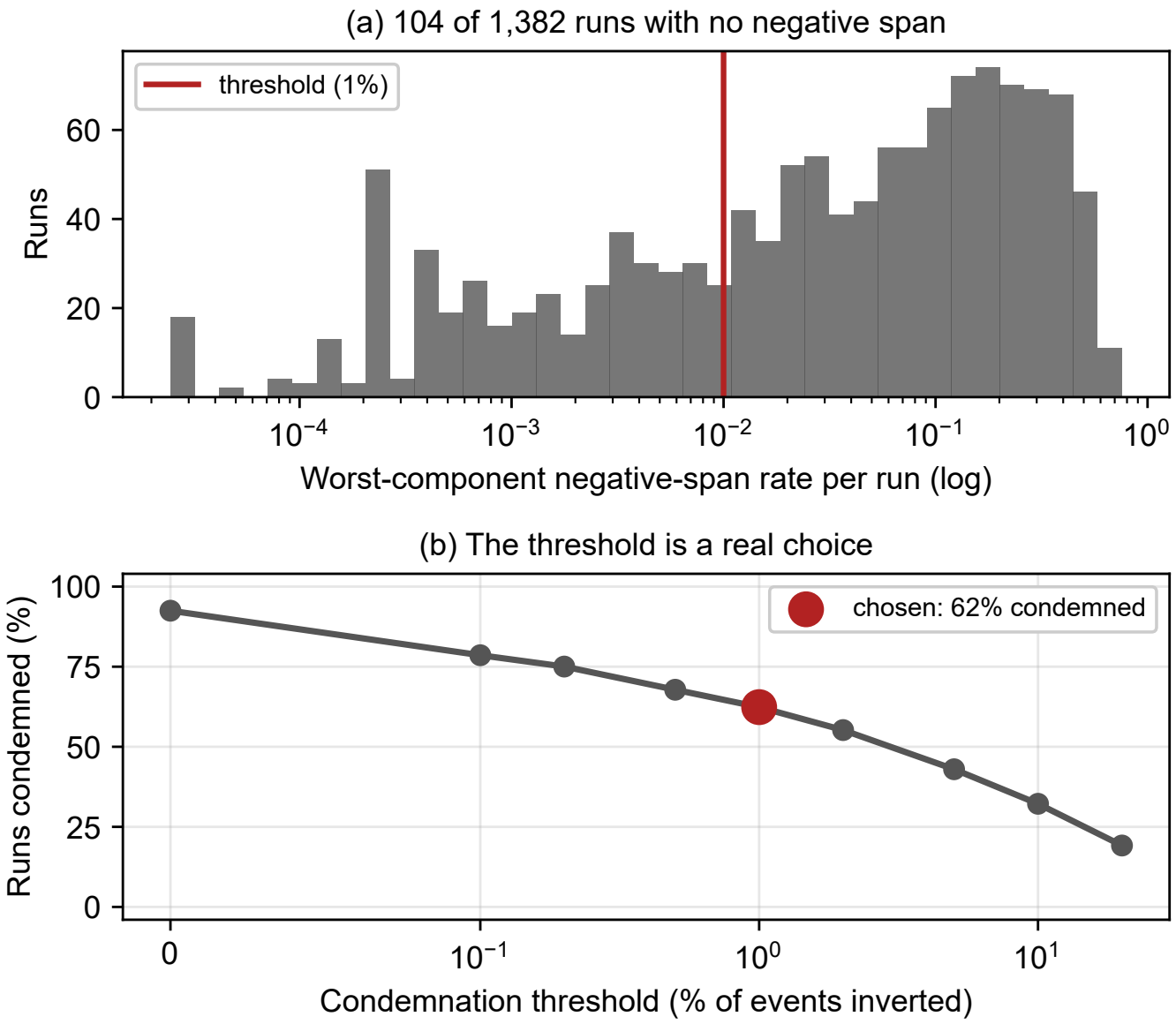


Fig. S15. **No natural threshold, and the choice moves no conclusion.** Audit on the workstation testbed. (a) Per-run negative-span rates: 104 runs carry no negative span at all and the rest spread over more than four decades, so there is no natural threshold. That is the count at which the rate is exactly zero, not the number the audit passes: the one-percent rule condemns 62.4% of the 1,382. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

3.7% and could detect differences above 5%. Their metric is not ours, so this is context rather than a test; but the smallest manipulation effect here is 2.02× in the negative-span rate, a change of about a hundred percent, and the largest is 80×.

External validity: one workload, two systems, two platforms: We demonstrate the failure on a sparse bursty feed, two brokers and two operating systems. The model of Section IV-B predicts it wherever cross-process spans are measured near the harness's own jitter under load, but we have not shown that. Sharma et al. [S18] report the same symptom in an unrelated domain, which is suggestive but is not our evidence.

Construct validity: a percentile is not automatically a property of the system: The 103 ms producer offset reproduced on both testbeds, and on the cloud testbed across four concurrency levels and 164 runs, and we took that breadth as evidence it was a property of the system. It was a property of the window (S3 and S5). Reproducibility across runs distinguishes a deterministic effect from a noisy one; it does not distinguish a system effect from a harness effect, and we treated it as though it did. Every percentile in this paper is now reported with the events per run behind it.

Construct validity: the injected-delay sweep: An earlier version estimated the slope along the end-to-end latency axis from a `tc netem` delay sweep. On this bursty feed injected delay builds a queue — transport variance climbs from 10 to 9,200 ms^2 between zero and 50 ms of delay — and confounds T_{true} with backlog, so S3.1 withdraws that sweep, and the T_{true} result rests instead on the payload sweep of Section IV.

The mechanism is established; the quantity is not predicted: The clearest limit on this work is what it still cannot do. We can say what causes a negative span, manipulate it on both sides, and predict the rate to within a third in the three configurations where we traced the scheduler directly. We cannot write down the rate a benchmark will suffer at a given load on a given machine. Two attempts failed and are reported in S1.5: the queueing form fits worse than the mean once the sweep reaches the range where forms diverge, and our own registered bracket missed a $1.44\times$ growth by predicting 2.45–$3.07\times$. Both failed for the same reason — both were parameterized in ρ, and ρ is not the variable. The latency factor $G(T_{\mathrm{true}})$ of Equation S2 is the nearest thing we have to a quantitative law, and it carries no load term at all: it predicts how the rate scales with T_{true}, not what the rate is. For the thread that timestamps the acknowledgment its shape is now measured, the rest of a slice and up to a tick (S16.10), and the level in front of it is still not predicted. So the practical guidance this paper supports is a check to run, not a number to look up. A reader wanting to know whether their own measurement is exposed must measure it, which is precisely why we argue the check should be routine rather than the model be trusted.

Conclusion validity: multiplicity and pre-specification: Holm families are declared over the design actually executed rather than chosen afterwards, and equivalence margins were fixed before testing. The threshold in the check was fixed before the final campaign but *after* earlier campaigns had run, which is weaker than pre-registration; the threshold sweep of S18 exists partly to compensate.

Limitations (full)

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster configuration, the durability quantification and every result from the workstation testbed (8 of its 76 conditions survive the audit, and none carries a claim). We cannot characterize behavior at the concurrency a multi-tenant platform would see.

E1's replay rate is inferred, not documented. S4 recovers it from the data and the recovery is, we think, correct, but it is an inference from a 52.34 ms median in one cell rather than a setting anyone recorded. A reader who does not accept the inference should read S2 as establishing that both configurations met the same rate, which is all the between-system comparison needs.

The E1 reconciliation is an explanation, not a re-run of E1. Section S5 shows that restricting the powered runs to their opening seven events reproduces E1's figures in both shift and absolute level, which is why we no longer treat the two campaigns as contradictory. But it demonstrates that the window is *sufficient* to produce the disagreement, not that it is the only difference between the campaigns: E1 ran on a corpus whose replay rate we infer rather than read, and we cannot rule out that some other difference contributes as well. What we can say is that no additional cause needs to be invoked.

The mechanism is established for our client, not for Kafka in general. The blocking first `produce()` is visible in the loop trace of every run (S3), so for this client the attribution is direct rather than inferred. Whether a different client, or a pre-created topic, or a broker configured against auto-creation would pay the same cost, we did not test. The practitioner advice of Section VI is written to be safe either way.

The waiting probability result is a mechanism, not a load model. Section IV shows by manipulation that the negative-span rate follows timestamping-thread waiting probability rather than utilization, and that is the strongest causal claim in this paper. It is not a formula. We can say the rate falls fifty-fold when the timestamping thread stops being preempted; we cannot say what the rate will be at a given load, and both attempts to write one — the queueing form and our own bounded bracket — failed against the sweep. A practitioner should read this as "keep the timestamping thread runnable and measure what you get", not as a curve to substitute into a budget.

The external evidence is single-host, and the cross-host case is unmeasured. The 223-run campaign of Section V ran in embedded mode, so both timestamps came from one JVM on one host. That is sufficient for the resolution failure, which needs no second clock — the timestamp resolution is a property of the timestamp, not of the network — but it means the cross-host channel is established by source audit and not by observation. We attempted a distributed run eight times, at three background-load levels including none at all, and the benchmark's own coordinator-to-worker protocol failed on every one; the load-starvation hypothesis we formed after the first failures is refuted by the unloaded attempts failing identically. That difficulty is worth recording rather than hiding, and it cuts both ways: a failure mode this hard to instrument is one a practitioner is unlikely to find by accident, and a distributed mode that does not complete a run in repeated attempts by readers of its source is itself a fact about the state of the tooling.

We are also explicit that the clock bound on this testbed does *not* rule the channel out. `chrony` reports the inter-host offset as 0.067 ms, comfortably sub-tick — but the bound it places on its own error, root dispersion plus half the root delay, is 4 to 7 ms per host across the 4 hosts captured, and the two worst of those bounds, 7 and 5 ms, sum to 12 ms against a 1 ms timestamp. An operator checking whether cross-host subtraction is safe reads the offset line and concludes it cannot happen; the figure that bounds the disagreement is two orders of magnitude larger and three lines further down the same output. The channel is therefore open on this testbed, and we could not measure it.

H3 is measured at one operating point. The symmetric-timestamping comparison (Table S28) was run at one in-flight request and one load level, because inline timestamping requires the single in-flight setting. The direction and the one-sidedness of the effect are what the model predicts, and the replication at 30 runs per cell keeps both while sizing the shrinkage at 0.039 ms against 0.071 ms (Table S28), but we do not characterize how the offset scales with load or pipelining.

TABLE S13
WORKLOAD, FROM 3,315 MATCHES. MEDIANS ACROSS MATCHES; PEAK RATE IS THE MAXIMUM OVER ANY SLIDING TEN-SECOND WINDOW.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

The five-feed acknowledgment null is unexplained.

One workload. The arrival-rate result should generalize to any sparse bursty feed, but we demonstrate it on one.

S20. THE WORKLOAD IS A LIVE FOOTBALL FEED, DESCRIBED AS FAR AS THE RESULTS REQUIRE

The workload is a live football event feed, and we describe it only as far as the results require. Full characterization of 3,315 matches from the StatsBomb open dataset [S19], spanning 52 competition-seasons from 2003 to 2023, is available in the artifact; event data of this kind is collected by trained human operators, whose accuracy has been studied [S20], [S21].

Three properties of the workload drive every experimental choice (Table S13). We name, for each, the later result that depends on it, because a setting section that cannot say what it is for should not be in the paper. A reader who wants only the measurement contribution can note that the workload supplies three things a synthetic generator would not have supplied — a sparse arrival process, a burst at a known instant, and an empirically bounded concurrency — and skip to the main text.

It is sparse. A match produces a median of $3{,}507$ events over roughly $8{,}412$ seconds, a mean rate of $\mathbf{0.415}$ events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes the results of Section IV possible.

It is bursty, and one burst has a known instant. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45; half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts. One of those bursts is special: every replayed plan opens with at least five events sharing $t_{\mathrm{sim}}{=}0$, two lineup records, two half-start markers and the kick-off pass, and across the corpus 575 of the $3{,}315$ matches open with four. That coincidence — the densest burst arriving when the producer is coldest — is what turns a one-off client start-up cost into the median of a short run, and it is the mechanism behind the withdrawal in S3. A synthetic Poisson generator has no kickoff, and would not have produced it.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches, and an 18-team league nine. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, which is also its upper quartile, the final matchdays of an 18-team and a 20-team league, and the observed maximum. This is what keeps the concurrency axis of S2 from being an arbitrary sweep — the levels are measured facts about the domain, not round numbers. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

Taken together these also explain why the broker question turned out to be the least interesting thing we could have asked, and we would rather say so here than let a reader discover it in the main text. At 0.415 events per second and at most a dozen concurrent feeds, this workload sits four orders of magnitude below where either broker strains. The domain that made the failure visible is the same domain in which the original question has a boring answer.

S21. EVERY SETTING THAT CAN MOVE A LATENCY NUMBER

Table S14 states every setting that can move a latency number, with the reason it holds its value.

Mohammad [S22] finds that published Kafka comparisons are frequently not comparable because client configuration and acknowledgment semantics differ silently between the systems being compared. We therefore state every setting that can move a latency number, with the reason it holds the value it does (Table S14). Three of these were not chosen but *learned*: the pipelining setting, the acknowledgment-batching setting and the replay speed-up each cost us a result before we understood them, and they are recorded in S1.14, S13 and S4.

The general principle is that the two clients must be equalized on *semantics*, not on parameter names. Kafka and Redis expose different knobs, and matching them syntactically is what produces the asymmetries this paper is about. Concretely, Kafka's producer buffers and pipelines by default while our Redis producer dispatches to a worker pool; leaving `max-inflight` at 1

TABLE S14
EVERY CONFIGURATION THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS THAT VALUE. SETTINGS MARKED *learned* WERE DISCOVERED BY A DEFECT THEY CAUSED, NOT CHOSEN IN ADVANCE.

Setting	Value	Why
Kafka producer		
`acks`	`all`	Durability semantics matched to Redis's synchronous `XADD`; weaker acknowledgment would compare different guarantees.
`linger.ms`	0	No artificial batching delay; the workload is sparse, so any linger would dominate the quantity measured.
`max-inflight`	64	*Learned.* At the default 1 the client blocks on a round trip per event, making the comparison one of harnesses rather than brokers (S1.14).
`compression`	none	Football events are small JSON; compression would add CPU to the path being timed for no representative benefit.
`ack-stamp`	`callback`	Default, and deliberately asymmetric with Redis; Table S28 measures what the asymmetry costs by also running `inline`.
Redis producer		
publish workers	pool	Non-blocking dispatch so the publish loop keeps schedule; publish and acknowledgment are timestamped in the worker, preserving a valid transport span.
Consumers		
Redis `ack-batch`	200	*Learned.* Acknowledging per message adds one network round trip per message, which at 20 ms of delay is the difference between 103 ms and 4,138 ms (S13).
Redis `count`	200	Read batch; matched to the acknowledgment batch so neither dominates.
Kafka `poll-timeout`	1000 ms	Client already amortizes fetches; no per-record round trip exists to remove.
Replay		
speed-up	derived	*Learned.* Plans carry a baked-in $120\times$ compression, so the flag is not the rate; it is derived from the plan and verified against wall time before every campaign (S4).
window	60–600 s	Swept deliberately: the number of events a run contains is itself a variable, and S5 is the reason.

makes the Kafka client wait for a broker round trip per event while the Redis client does not, which is not a broker difference but a harness difference.

S22. THIS HARNESS HAS DISCARDED LATENCY DATA BEFORE

Two silent losses that were bugs (moved from the main text): The admission condition is not the first time this harness has discarded latency data with nothing in its output to show it. A serialization defect found in 2023 truncated the histogram on long runs and lost most of the samples [S23]; a second defect — distinct, though the two interact in the repair's own account — reset each client's histogram on read during a retry, and was found only during "a statistical analysis on some results that looked strange" [S24]. Both were repaired. They are recorded here because they establish a contrast: those were bugs, and the admission condition is not one – it was added deliberately, for a stated and locally sound reason, and it is still there.

The second failure mode is measured in the OpenMessaging Benchmark rather than in our harness, so the instrumentation is a patch to somebody else's code and its design needs stating.

What the patch does, and what it deliberately does not: At the named commit, a sample is admitted to the histogram only by `if (endToEndLatencyMicros > 0)`, which has no `else`. We add one, split by sign, and a counter of the samples kept, and change nothing else: not the latency computation, not the admission condition's condition, not any reported statistic. The benchmark's own output is therefore exactly what it would have been unpatched, which is the property that lets us set its published summary beside the retention behind it. The patch is in the artifact as the campaign script that applies it, `cloud/campaigns/omb_discard_count.sh`, which checks that the condition is untouched and writes the resulting diff beside each run; the first run's single-counter version is `external/omb/workerstats_discard_counter.patch`.

The counters must distinguish sign, and an earlier version did not: Both a causality violation and a sub-millisecond end-to-end latency fail a `> 0` test. A single unsigned total cannot tell them apart, and we initially reported one, drawing a conclusion the sign-separated data later refuted (S1.3). The counters are now four — zero, negative, most-negative, kept — and the distinction between them carries the finding.

Counts must be exact, which required two corrections: The counters are declared `static`: an embedded run constructs several `WorkerStats` instances, and per-instance counters silently split the totals between them. Totals are printed once by a JVM shutdown hook rather than read from periodic progress lines, because those fire every $10{,}000$ discards and reading the last one quantizes the answer to a multiple of $10{,}000$ — three runs with genuinely different totals all reported $50{,}000$ that way.

Every cell records which source its counts came from, and cells whose counts are quantized are excluded from every analysis by rule rather than by inspection.

A validity gate that refuses to report a number: A benchmark that fails to run discards nothing, and a zero from such a run looks exactly like a measured zero. Our first distributed attempt wrote `discarded=0` after the worker died on a classpath fault, and that vacuous zero reached a draft of this paper as a genuine negative result. The campaign now validates before writing: the run must have produced publish-rate output and no failure lines, and if it did not, the row is written `valid=0` with the reason and no count. The gate has since refused eight distributed attempts (S22.1), and its refusals are in the artifact alongside its acceptances.

Denominators: OMB warms up for one minute by default, resets its histograms at that boundary, and reports percentiles over the test phase alone; our counters are never reset and would otherwise span both. We make the setting explicit and report a matched sweep with warmup disabled. It changes the sample count per cell from $\approx 120{,}000$ to $\approx 90{,}000$ and leaves the retention fraction unchanged, so the figures computed with warmup included stand.

Campaign design: Retention at a fixed configuration proved not to have a stable central value (S10), which dictates the design: every claim is made over whole distributions of replicates rather than over a summary of them, and the axes are swept one at a time with everything else held fixed — background load, message size, producer rate, and the warmup setting — because the variable that turned out to matter was one we did not initially think to hold fixed.

Every run is indexed, including the failures: Each cell is one row in a ledger built by reading the cell's own log rather than a receipt written afterwards, recording configuration, counts, whether the counts are exact, and the size and SHA-256 of the log they came from. This is not bookkeeping for its own sake: it recovered a valid cell whose script died between finishing the benchmark and writing its result row — a cell that is the median of its load level — and it exposed a fabricated negative sample that our own indexer had produced by mapping an older result schema positionally. A campaign that silently drops its failures is a selection of the runs that worked, the same uncounted selection this paper finds in a benchmark's samples.

The full record, stated once: The external campaign totals 223 instrumented runs in twelve named sub-campaigns: four load sweeps, a resolution sweep, a payload sweep at an incommensurate rate, a bimodality series (n=18 at one configuration), two rate-phase sweeps, the rate-independence control, a 63-cell comprehensive campaign (configurations to $n \geq 5$, three new denominators, a payload×grid manipulation, a duration sweep, and a two-rate probe of the continuous value), and a distributed-mode series that never completed a run and is reported as exactly that. Six predictions for the comprehensive campaign were registered with falsifiers before it launched, and each interim reading was committed to the version-controlled record before the block that followed; three registered predictions failed and are reported beside the one that passed. From 22:02 UTC on the final night a once-per-minute log of the driver's clock discipline (residual frequency, skew, offset) runs alongside every cell, so each run joins to the clock state during it.

Verification of the manuscript itself: Every quantity this paper states from the campaign is recomputed from the committed ledger by an automated suite (2,000-plus checks, branch coverage of the analysis code at or above 95%), including every row of Tables I and II and every checkable sentence of the main text; run counts and discard totals are generated into the text from the ledger rather than typed, after an earlier draft was found quoting a count stale in nine places at once. The rendered PDF is checked separately from the source, which has caught defects the compiler passes. The suite is mutation-tested: corrupting a table value, or a value and its derived classification together, fails a check.

S22.1. Distributed mode never completed a run in eleven attempts

The cross-host case is the one in which OMB's subtraction spans two clocks, and therefore the one in which its admission condition could discard a genuine causality violation. **OMB's own distributed mode cannot report a measurement of it.** Across eleven attempts — eight at three background-load levels including none at all, which refutes our own hypothesis that CPU starvation was the cause, and three further attempts run with full diagnostics for this revision — the benchmark's distributed mode never completed a run, failing each time inside its own coordinator-to-worker protocol rather than in our instrumentation or configuration. The first attempt ran before the validity gate existed and wrote a vacuous zero, which the artifact keeps marked invalid; the gate wrote `valid=0` with a reason on every later attempt. The outcome of each attempt is in the artifact and Section S24 summarizes the failure signatures, those of the three diagnosed attempts read from the console and not retained; a draft upstream report accompanies them.

The exposure of OMB-in-distributed-mode is therefore established by source audit — the timestamp is written on the producer's host and read on the consumer's, and the admission condition drops whatever the difference produces — and *not* by observing that binary. What *is* now observed is the pattern itself across two hosts: the harness of Section V-F runs the identical timestamp-subtract-guard logic over two disciplined clocks across 12 runs, and measures 0 negatives in 905,040 samples, an intact grid, and a deleted fraction coupled to clock-offset drift. At 457 msg/s, the incommensurate configuration and the only one of the three whose own grid spread does not swamp the comparison, cross-host retention wandered from 13.4 to 26.9% over four replicates, where the same configuration on one clock held to 0.98 points: across hosts the deleted fraction follows the synchronization state, and nothing records either. The two statements are complementary and we keep them distinct: the pattern's cross-host behavior is measured; OMB's own distributed binary remains unmeasurable by us.

The library's own documentation does not describe the behavior the fix was written against: The 2018 rationale rests on `HdrHistogram` rejecting negative values [S25], and it does. What its documentation says is another matter. The `Recorder.recordValue` entry documents the exception it may throw "if value is exceeds `highestTrackableValue`" [S26] — the upper bound, and only that — and the project's README describes the configurable range and the significant digits without mentioning negative values at all. The rejection surfaces as an `ArrayIndexOutOfBoundsException` from the counts-array index rather than as a documented contract. Two ports of the library document what the Java one leaves out: the Go port's recording call returns "an error if the value is out of range" [S27], and the Rust port's documentation says `record` "will error if the value is too small or large" [S28]. That makes the maintainer's local fix more defensible rather than less: it was written against behavior the library documents for the opposite bound. It also makes the wider case, that a disposal rule has to be stated by whoever depends on it, because the dependency will not state it for you.

Nor has anyone upstream returned to it. A search of the benchmark's issue tracker for the variable the condition tests, run on 2026-09-13, returns 1 item: pull request #56, the change that introduced the filter [S25]. In the eight years since, including the coordinated-omission revision the project accepted, no issue or pull request has named it. The search and its date are a ledger row rather than a remembered result, re-run by `check_omb_tracker.py`.

It has accepted a measurement-validity fix before, which is why the remedy is not a hopeful one: In March 2022 a contributor reported that the load generator did not record publish delay — the gap between an event's scheduled publish instant and its actual one — arguing that "whatever prevent the load generation to work as expected should be recorded and weighted correctly [. . .] otherwise inability to keep-up with the expected load will pass unnoticed in the percentiles ocean" [S29]. The report was accepted and the delay is now recorded. Two things follow. The maintainers act on a measurement-validity argument when one is put to them, so the rules of Section VI are addressed to a project that has taken this kind of correction before rather than to an indifferent one. And the admission condition came through that revision untouched: a review explicitly about what the harness fails to record left in place a filter that deletes what it did record. The two defects look nothing alike from inside the code, which is the argument of this paper in one repository's history.

The histograms it discards after reading them: In March 2026 a second contributor asked for the HDR histograms to be serialized into the result JSON, observing that the harness "already maintains HDR Histograms internally throughout each run and discards them after extracting the summary scalars", and that summary scalars alone cannot say whether a measured difference is significant "or within the noise floor of the measurement" [S30]. We record it because it is this paper's argument arriving from the other side, and because of what it would and would not fix. A serialized histogram supports every test the request names — Mann–Whitney, Kolmogorov–Smirnov, bootstrapped percentile intervals — computed over a population whose size is still not stated, because the admission condition runs before the histogram is filled. The export is worth having. It carries the deletion into a new container unless the retained fraction travels with it.

What we do not claim: That any published result obtained with this benchmark is wrong. We have shown that the deletion happens, that it is large, that it is irreproducible, and that it is unreported. Whether it has affected a specific published measurement depends on that measurement's path latency and producer rate, and we have audited none. That asymmetry is the reason we recommend reporting a retention rate as routine practice: its cost is one number, and the alternative is a benchmark that cannot report its own failure.

S23. Where the check transfers, and what it costs to run

The check is worth reporting only if it transfers, so we state it as a procedure rather than as a description of what we happened to do. It applies to any latency computed as a difference of timestamps taken in different processes, threads or hosts — which covers most end-to-end measurements in this literature.

Eight of the steps are the reporting rules Section VI closes on, and they are stated there rather than twice: name the span you subtract, report the rejection rate, report the retained fraction beside the latency, know where your path sits on the exposure curve, count your events before quoting a percentile, and record the achieved rate rather than the flag meant to produce it (Section VI). What follows is the four this document adds, each a check Section VI has no room for.

1) **Read your own reported percentiles for quantization. This costs nothing and needs no patch.** If every percentile a benchmark prints is a whole multiple of its clock's resolution, the measurement has no resolution below that timestamp resolution and the distribution is an artifact of the grid. In the benchmark we audited, all 32 percentile values across eight runs were whole milliseconds, and three runs reported $\mathrm{p50} = \mathrm{p95} = \mathrm{p99} = \max = 1.0$ — a distribution with one value in it, printed to three decimal places, in output anybody could have read. We found it only after instrumenting the source, which was unnecessary.
2) **Put your publish interval over the timestamp resolution, reduce the fraction, and look at the denominator.** A fixed interval that visits few phases within the timestamp resolution puts nearly every sample in a run at one of them, which makes the surviving fraction bimodal and irreproducible rather than merely small (Section V-D). The diagnostic is arithmetic on two numbers you already know, but it is the reduced denominator q that matters, not whether the interval looks round: 500 msg/s gives $q = 1$ and spread 99.52 points, and 300 msg/s gives $q = 3$ and spread 31.23, while

889 msg/s — which sits 0.00014 away from $9/8$ — gives a denominator in the hundreds and is stable to 1.70. A small q is the hazard; proximity to a simple fraction is not. The remedy is to randomize the publish instants, which works without having to get the arithmetic right.

3) **Before attributing an offset to every event, vary the run length and count.** Lengthen the observation window by a known factor and count how many events exceed the offset, rather than re-computing the average. A per-run cost holds that count fixed while the run grows; a per-event one grows it in proportion. The two are indistinguishable in any percentile, which is why we report the count. In our sweep the count stayed at four while events per run grew several times over (Section S3).
4) **Where retention differs between configurations, bound the selection it introduces.** Unequal rejection is itself a bias, and S18.1 gives the worst-case imputation we use.

The cost is one pass over data the harness already produces. In our implementation the rule is under two hundred lines and runs in seconds over 2,266 runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

When it matters most: Equation S2 tells a reader where to be careful without repeating our experiments: risk rises as the measured quantity approaches the harness's own scheduling jitter, and as utilization approaches saturation. A benchmark measuring milliseconds on a loaded machine is in the dangerous region; one measuring seconds, or running well below saturation, is not. That is a rule of thumb, and S19 says what would be needed to make it quantitative.

S24. The campaign ledger, and what distributed mode did instead

One row per cell of every instrumented-benchmark campaign, in the ledger `external_campaigns_index.csv`: the swept axis and its level, a validity flag whose zero rows are kept with a reason rather than deleted, the source of each count, the instrumented guard's three sign-separated counters, and the size and SHA-256 of the log the row was built from, so any row can be re-derived from its source. The column-by-column dictionary is `docs/campaign_ledger_schema.md` in the artifact, beside the file it describes; the Python-harness campaigns use the same cell naming and write `docs/results/external/harness_results.csv`.

S24.1. OpenMessaging distributed mode, failure diagnostics

Eleven attempts in total never produced a measurement. The three diagnosed attempts of chain 18, block E, fail identically at OMB commit `5b1fa70`: `java.util.concurrent.CompletionException` wrapping `java.lang.IllegalArgumentException` in the coordinator–worker path, before any workload starts. What the artifact carries is the outcome of each attempt rather than its logs: one row per load level in `docs/results/external/omb_distributed_result.csv`, `docs/results/external/dist_load0/` and `docs/results/external/dist_load50/`, each recording that the run produced no latency output. The classpath failure of the first class is reproduced in full in `external/omb/omb_worker_client.log` and `external/omb/omb_worker_driver.log`. The coordinator–worker signatures of the three diagnosed attempts were read from the console and not retained, so the signature quoted above is the only record of them this artifact holds — a limitation, and part of why the cross-host claim rests on source reading rather than on these runs. A draft upstream report is at `docs/omb_distributed_issue.md` (not filed — filing is the author's action).

Added on 10 October 2026. On the registered campaign's Azure machines the unmodified benchmark's distributed mode ran: a coordinator and two workers on one machine, against a Kafka broker on another, in the tools block that ran five more tools that month (the supplementary material's S4.5 and S5). It completed every run it was given. When the consuming worker's clock was moved back, its filter dropped what then computed to zero or below, and nothing counted what it dropped. The eleven attempts recorded here, all on the Oracle machines, stand as they were.

The cross-host case is the one whose subtraction spans two clocks: There OMB's subtraction spans two of them, and its own distributed mode never completed a run in our hands — eleven attempts, all failing identically inside its coordinator-to-worker protocol, including with no background load at all, which refutes our own load-starvation hypothesis. Its cross-host exposure therefore rests on source audit, the pattern itself measured across two clocks in Section V-F (per-attempt record and draft upstream report: above). We do not claim any published result obtained with this benchmark is wrong: exposure depends on a deployment's path latency and producer rate, and we audited none — why we recommend reporting a retention rate as routine practice, at the cost of one number.

S24.2. The statistical inventory

Section II-C keeps the choices a reader might contest — Wilson over Wald, Holm within a named family, and why a tail index on interval-censored buckets is not a slope. This is the rest of the inventory, so that every interval in either document has a named method behind it.

Every ratio between two proportions carries a two-proportion z with pooled variance, and we state whether the intervals are disjoint. Location differences use Hodges–Lehmann shifts with distribution-free intervals and rank-sum tests; equivalence claims

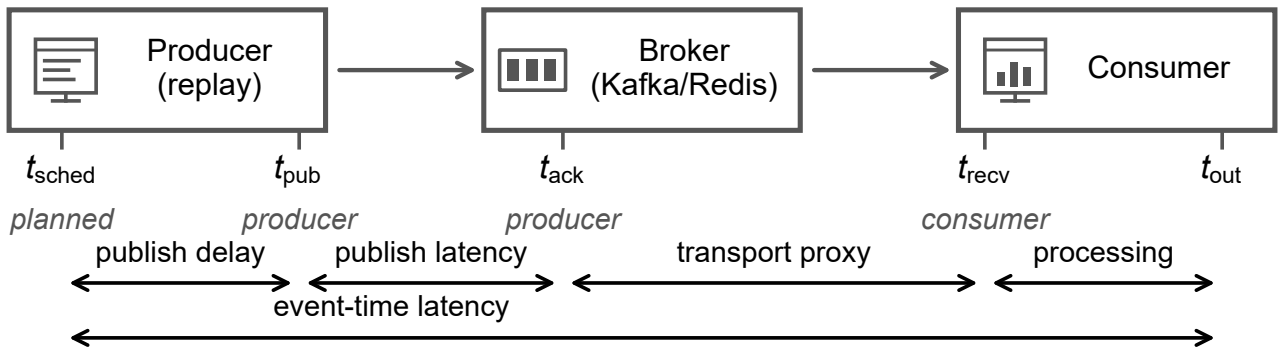


Fig. S16. **System measured, and where its clocks are read.** A replay process paces recorded match events at a chosen rate into a broker — Kafka or Redis Streams — and a consumer reads them back. The five timestamps and the five spans they bracket are those of main-text Section IV-A; the upper four are the terms of Equation 2 and sum to the fifth, the event-time latency, which the harness closes at t_{out}. The transport proxy subtracts a producer-process timestamp from a consumer-process one. Both processes run on one host reading one clock unless stated otherwise, so nothing drawn here depends on clock synchronization.

use TOST with the margin stated and the interval printed; Holm correction is applied within a named family, and the corrected value is the one the tables display; percentile bootstrap intervals cover statistics with no closed form. The grid-membership inference uses a Monte Carlo continuum null, described where it is used (S14.2). Regression intervals are Student-t at $n-2$ degrees of freedom. The correlation between end-to-end latency and publish latency is Pearson's, computed two ways over the same events: pooled over each condition's runs (median 0.85, interquartile range 0.79–0.89), and with each run centred on its own means (median 0.80, interquartile range 0.69–0.86). The within-run correlation is lower at the median and lower still at its lower quartile, so a quarter of the conditions move together less than either median suggests. Section III-A quotes the second, because its sentence is about a run. The price of assuming independence is a ratio of rates, and its size depends on its smallest denominators. Pooled over each condition's runs, the median overprediction is 12.4 over the 61 conditions with any negative and 7.3 over the 54 with a rate above 0.1%; with pairing confined to each run, 12.2 over 61 and 7.4 over 54. The unit barely moves it and the floor moves it by half, while its direction holds everywhere: independence overpredicts in 70 of the 70 conditions. Section III-A quotes the within-run figure at the floor. Tail indices are estimated on the traced histogram by grouped maximum likelihood with a profile-likelihood interval and by an exceedance ratio with a Wilson interval, because the buckets are interval-censored and a slope through survival points is not an estimator; the grouped estimator on log-spaced bins is Virkar and Clauset's, which is the Hill estimator for binned data, and we use their bootstrap for goodness of fit [S31].

All interval arithmetic is recomputed from the committed artifacts at build time by the same code that emits the numbers into the text, so no figure in either document is transcribed.

S24.3. The metric map

This section says which interval each reported quantity actually covers. Figure S16 draws the four bracketing timestamps; what belongs beside it is the mapping from those timestamps onto the metric names, because a reader comparing our numbers against another harness's needs to know what was subtracted from what.

Publish delay is $t_{\mathrm{pub}} - t_{\mathrm{sched}}$. Both timestamps are taken inside the producer process, so it is a causal chain and cannot invert. *Publish latency*, $A = t_{\mathrm{ack}} - t_{\mathrm{pub}}$, is also producer-internal and also cannot. *Broker transport* is the transport proxy $S = t_{\mathrm{recv}} - t_{\mathrm{ack}}$: it subtracts a producer-process timestamp from a consumer-process one, which is exactly why it can come out negative when either process is delayed, and it is the span this paper is about. *Event-time latency* is $t_{\mathrm{recv}} - t_{\mathrm{sched}}$, the sum of the three (Equation 2), and it inverts only if the whole path does. A fifth timestamp, t_{out}, is consumer-internal and is not drawn in that figure; S16.1 places it.

S25. THE EVIDENCE BEHIND THE COUNT OF TOOLS THAT DISPOSE SILENTLY

Section V-A says how many independent tools dispose of an inverted or sub-resolution sample without counting the disposal. This section is the evidence behind that count. Each row of `docs/results/external/harness_registry.csv` carries one source line, the file and symbol it came from, the upstream URL and the date it was read. The class labels are not stored in the file: they are recomputed on every build, and on every test run, by the same classifier that read the OpenMessaging Benchmark a month before the other tools were audited. A reader who disagrees with our reading of a guard can therefore check the line against the pattern that matched it rather than against our prose.

A fork that is read-only cannot be repaired: The admission condition survives downstream: 40 of the 40 readable public forks carry the expression unchanged (surveyed 2026-08-24), which bounds how few have diverged rather than how many chose it. A fact from 2026 bounds something else, and more sharply: Confluent's fork of this benchmark, one of the vendor distributions practitioners actually run, was archived by its owner on 28 February 2026 and is now read-only, with 60 forks of its own. Divergence and repairability are different properties. A live fork that carries the condition may drop it tomorrow; an archived one cannot, and neither can anything downstream of it. Whatever that tree is doing with sub-resolution samples, it will keep doing, which is the practical reason the rule of Section VI is addressed to benchmark authors and to their readers rather than only to maintainers. Read 2026-09-12.

Table S15 is the fold of those rows up to one verdict per *span*, which is the grain the finding lives at: one tool can guard one span and leave another alone, and the OpenMessaging Benchmark does.

TABLE S15

AUDITED HARNESSES, ONE ROW PER SPAN RATHER THAN PER TOOL. *Filter* DROPS WHAT ITS COMPARISON EXCLUDES, AND THE COMPARISON IS PRINTED WITH IT: > 0 DELETES EVERY SAMPLE THAT COMPUTES TO EXACTLY ZERO, WHICH ON A PATH SHORTER THAN THE RESOLUTION IS MOST OF THEM, WHILE ≥ 0 KEEPS THOSE AND DROPS ONLY THE NEGATIVE SPANS. *substitute* REPLACES AN INVERTED INTERVAL WITH A CONSTANT; *refused by own library* MEANS THE RECORDING LIBRARY RETURNS A REFUSAL THE CALLER NEVER CHECKS; *counts* MEANS THE DISPOSAL IS EXPOSED IN THE OUTPUT. THE OPENMESSAGING BENCHMARK APPEARS TWICE, BECAUSE IT TREATS ITS TWO SPANS DIFFERENTLY AND THAT DIFFERENCE IS THE FINDING OF SECTION V-C. SOURCES, IN ROW ORDER: [S1], [S1], [S32]–[S34], [S34]–[S39], [S39]–[S55], [S55]–[S57]. THIS TABLE AND THIS LIST ARE GENERATED FROM `HARNESS_REGISTRY.CSV` AT BUILD TIME.

Tool	Language, maintainer	Span measured	Clock	What happens to the value	Counts
Artemis `perf`	Java, Apache Software Foundation	end-to-end	microsecond	refused by own library	no
BookKeeper benchmark	Java, Apache Software Foundation	publish	nanosecond	filter, > 0	no
Apache Kafka (bundled)	Java, Apache Software Foundation	end-to-end	nanosecond	truncate to the resolution; substitute a value	no
Apache Kafka (bundled)	Java, Apache Software Foundation	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	Java, Apache Software Foundation	end-to-end	millisecond	filter, ≥ 0	no
RocketMQ benchmark	Java, Apache Software Foundation	end-to-end	millisecond	—	—
Pub/Sub load test	Java, Google	end-to-end	millisecond	substitute a value	no
Sarama producer perf.	Go, IBM	publish	millisecond	truncate to the resolution	no
Kafka Trogdor	Java, Apache Software Foundation	end-to-end	millisecond	substitute a value	no
Kafka Trogdor	Java, Apache Software Foundation	publish	millisecond	refused by own library	no
NATS CLI	Go, Synadia	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	Java, Linux Foundation	end-to-end	millisecond	filter, > 0; truncate to the resolution	no
OpenMessaging Bench.	Java, Linux Foundation	publish	nanosecond	—	—
PerfKit messaging	Python, Google	end-to-end	millisecond	truncate to the resolution	no
Pravega benchmark	Java, Pravega	end-to-end	millisecond	refused by own library	no
Quiver	Python, Justin Ross	end-to-end	millisecond	substitute a value	no
RabbitMQ PerfTest	Java, Broadcom/VMware	end-to-end	nanosecond	substitute a value	no
RabbitMQ Stream PerfTest	Java, Broadcom/VMware	end-to-end	millisecond	filter, ≥ 0	no
`kgo-repeater`	Go, Redpanda Data	end-to-end	microsecond	—	—
Rezolus	Rust, IOP Systems	scheduler wakeup	nanosecond	discard	yes
SBK	Java, Keshava Munegowda	end-to-end	millisecond	discard	yes
Strimzi Canary	Go, Strimzi	end-to-end	millisecond	filter, ≥ 0	no
Xinfra Monitor	Java, LinkedIn	end-to-end	millisecond	substitute a value	no
Yahoo Streaming Bench.	Clojure, Yahoo	end-to-end	millisecond	filter, ≥ 0	no
`blktrace/btt`	C, Alan D. Brunelle	block stage	nanosecond	substitute a value	no
`emqtt-bench`	Erlang, EMQ Technologies	end-to-end	millisecond	kept; zero in its printed average	no
`fio`	C, Jens Axboe	I/O completion	nanosecond	substitute a value	no
`rdkafka_performance`	C, Confluent	end-to-end	microsecond	filter, > 0	no
`rdkafka_performance`	C, Confluent	publish	microsecond	—	—
`mqtt-benchmark`	Go, Alexander Krylovsky	publish	nanosecond	—	—
wrk2	C, G. Tene	end-to-end	microsecond	refused by own library	no

The most interesting row is the one carrying no defect at all. The OpenMessaging Benchmark's publish span is taken inside one process, timestamped with a nanosecond clock and recorded unconditionally; its end-to-end span crosses processes, is timestamped in milliseconds, and is filtered. The coarse clock and the admission condition both landed on the span that needed care — the asymmetry Section V-C draws its conclusion from, and the reason this table folds by span.

What the round-56 additions changed, including a category the survey did not have: Four harnesses were added after a co-author asked what the audit had *not* looked at, and they changed the shape of the finding rather than adding to a tally.

Apache Kafka's own bundled tools were the gap that mattered, because they ship inside every distribution and are therefore installed wherever the broker is. `EndToEndLatency` is immune to the scheduling failure by construction — one thread, one monotonic clock, a synchronous publish call, so no negative span is possible — and it loses the measurement anyway:

```
System.out.println(i + "\t" + elapsed / 1000.0 / 1000.0);
latencies[i] = elapsed / 1000 / 1000;
```

The printed line is floating point and correct; the `long[]` the percentiles are read from is filled by integer division. Every sample is kept, nothing is filtered, and on a sub-millisecond path `p50`, `p99` and `p999` all report zero. The companion

`ProducerPerformance` takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.

This is what forced a new class into the classifier. Until then its categories were three ways a sample can *vanish*, and a survey whose only question is "what happens to a negative" cannot see a tool that keeps every positive and records it as zero. `quantized_retention` matches a division or cast that discards the unit the value was measured in, and applying it retroactively labels the OpenMessaging end-to-end line too — `MILLISECONDS.toMicros(now - publishTimestamp)`, where the microseconds are decoration on a millisecond subtraction. The deletion failure is therefore not a claim about a filter. It is a claim about a timestamp resolution, and deletion is one of the ways it collects.

The other two additions were filed as counterexamples, and they are in the table for the reason Rezolus is. The NATS CLI appends `time.Duration(time.Now().UnixNano()-sendTime)` unconditionally, so a negative goes through into its metrics, which is what RFC 7679 asks of a measurement, and it is exposed to skew rather than to quantization. RabbitMQ PerfTest was read the same way: it differences a timestamp carried in the message against a local clock and passes the result to `performanceMetrics.received(diff_time)` with no sign test at the call site. *Corrected on 2 October 2026, both of them.* PerfTest's difference comes from `TimestampProvider.getDifference`, which returns `Math.abs(ts1 - ts2)`, so a negative span enters its metrics as a positive one of the same size, uncounted: a substitution. The NATS CLI's slice keeps the negative, so its minimum, median and maximum do; but its percentile table is recorded into `hdrhistogram-go`, whose bound check refuses the index a negative value maps to, and the CLI ignores the error: a library refusal, wrk2's class. Both readings above stopped at the call site; the law campaign's documentation audit (block T5, `data/tools_audit/`) read the methods, recorded the PerfTest prediction as failed and the NATS one as holding only in part, and the registry now carries the lines it found. *The field still does not uniformly delete*: `emqtt-bench`'s histogram keeps the negative and Rezolus counts it, and the paper must not be read as saying otherwise.

Two tools were looked at and found not to make the measurement at all. SPECjms2007, the only messaging benchmark from a standards body, reports throughput (`SPECjms2007@Vertical` and `@Horizontal`); it defines Delivery Time as "the elapsed time measured between sending a specific message and that message being received" and never reports it. Redpanda's `rpk` ships no latency benchmark, its published comparisons being OpenMessaging runs. Neither is evidence for or against the failure, and both are recorded so that a reader asking "did you check X" gets an answer rather than a silence.

Two further details are worth stating plainly. First, `fio`'s guard appears in three separate timing functions, each returning zero for a negative interval, and the comment above it — "time warp bug on some kernels?" — is a question rather than a diagnosis. The behavior is defensive and reasonable in isolation; what is missing is any record that it fired. Second, `btt`'s `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page. A substitution of this kind is harder to detect than a filter, because the sample count is unchanged: any retention rate computed downstream reads 100%.

Fourth, `emqtt-bench` is in the table as a correction of our own reading rather than as a finding. Its `> 0` test guards a counter, not the histogram, so nothing is deleted from the sample; we had first read it as a filter. A guard is only a deletion if what it guards is the sample, and that distinction is now what the classifier encodes.

Third, `wrk2`'s disposal was mis-evidenced in an earlier draft of this registry, and the correction is worth spelling out because the wrong attribution is the natural one. `wrk.c` detects the negative interval, prints a block that begins "We are about to crash and die", and records the value anyway with the boolean return unchecked. We first attributed the resulting drop to upstream HdrHistogram_c's value guard (`if (value < 0 ...) return false`) [S58]. `wrk2` does not build against upstream: it vendors its own `hdr_histogram.c`, which has no value guard at all — it bounds-checks the *derived* counts index (lines 219 and 234) and returns an unchecked `false`, and its index computation carries `assert()`s that are live in the shipped build (`-O2`, no `-DNDEBUG`), so a negative may abort the run before the refusal is reached. Whether the assertion trips or the bounds check refuses, the sample is never recorded and nothing counts the loss — and the panic block's own prediction turns out to be the accurate part. The registry row now cites the vendored file, and the classifier's refusal pattern matches both spellings of the same disposal.

Two patterns had to be added to the classifier before it could see these tools, and both additions are recorded here because they bound what the survey can claim. The original patterns were written against JVM sources and could not match Erlang's guard, which is a short-circuit rather than an `if`, nor Erlang's clock read, which carries its unit as an argument. The survey therefore reports what the classifier can currently recognize, which is a lower bound on the field rather than a census of it. The complementary lineage is behavioral: Mittal evaluates six open-loop load generators for rate fidelity and measurement skew without reading their source [S59], and the two methods find disjoint defect classes, which is the argument for doing both.

The pattern has since reached the audited broker itself. In November 2025, Kafka's coordinator metrics were found throwing on negative durations when a host clock stepped backwards, and the merged fix clamps the negative to zero with no counter and no log line [S60] — silent substitution, the class this table marks *substitute*, adopted in the very system whose benchmark opened this audit. Section V-A names the case and its citation in one sentence and sends the reader here for the rest, because its mechanism is genuine wall-clock non-monotonicity, and setting it out beside the scheduling failure would invite exactly the clock-versus-stall conflation the paper separates. The same project makes the opposite choice a few directories away. Kafka Streams' end-to-end latency metric records the wall clock less the record timestamp with no filter at all [S61], so a record

TABLE S16
PUBLISHED RESULTS RARELY CARRY THE DELETION'S SIGNATURE, AND NONE STATES WHAT IT KEPT. THE OUTCOMES REGISTERED IN `FREEZES/29-PUBLISHED-RESULTS-AUDIT/`, COUNTED FROM THE CODING BY THE CODE THAT WRITES THIS DOCUMENT'S NUMBERS.

Outcome	Count
Reports included	43
Configurations they print	1,100
of which print an end-to-end median	868
Primary: median of exactly 1 ms, configurations	8
in reports	2
Strong: p99 of exactly 1 ms as well, configurations	0
Reports stating a retained fraction or a discard count	none
Reports printing a median below 1 ms (not registered)	9

timestamped in the future is recorded as a negative latency. One project clamps the negative in one metric and keeps it in another: the disposal is a choice, not a necessity.

S25.1. Published results, audited against a signature fixed in advance

Few published results carry the deletion's signature, and none says how many samples it kept. The audit behind that sentence was registered in `freezes/29-published-results-audit/` and committed, fingerprinted and public before any report was read. It fixed the signature, the places to search, the rule for inclusion, the coding and the outcomes; nothing in it changed afterwards.

The signature: Every end-to-end sample the benchmark records is a whole number of milliseconds, so grid membership alone identifies nothing. The value does. The primary signature is a printed end-to-end median of exactly 1 ms: the smallest value the benchmark can print, and the value every retained sample takes when the true end-to-end latency is shorter than the resolution. The strong signature is a p99 of exactly 1 ms beside it, a distribution collapsed onto the grid as in our own uninstrumented runs. Neither says a published number is wrong. Each says the number came from a fraction of the samples that the report does not state.

What was searched: Google Scholar and Semantic Scholar, the web, the engineering blogs of the vendors whose brokers the benchmark has drivers for, and the benchmark's GitHub repository and its public copies, for reports dated up to 28 September 2026. A report was included if it named the benchmark or a fork of it as its tool and printed at least one numeric end-to-end percentile; figures read off a chart did not count. Each configuration a report printed is one row of `docs/results/external/published_results_audit/configurations.csv`, with the quote its numbers were copied from, and every quote was checked mechanically against a saved copy of its source.

What was found: Table S16 gives the registered outcomes as the coding counts them. 43 reports were included, printing 1,100 configurations, of which 868 print a median. The primary signature occurs in eight configurations in two reports, both public repository text: five Redis rates in a notebook behind a 2023 journal paper that shows its medians only in a chart, and three maximum-rate runs in three result files of a 2021 fork. None of them also prints a p99 of exactly 1 ms, so the strong signature occurs nowhere. No included report states a retained fraction or a discard count, which is what the paper's reading of the source predicts.

What the rules did not foresee: Several copies of the benchmark print medians below a millisecond, which the upstream clock cannot produce: 9 of the included reports do, among them vendors' copies whose maintainers changed how the span is timed. One fork prints an average just above 1 ms with p95, p99 and p999 all at exactly 1 ms and no median column, so by the registered rule it does not count. Four configurations print a median of zero, from empty histograms. Excluding the raw result files that carry no prose, the primary count falls to five configurations in one report; reading "fork" narrowly, as a GitHub fork of the upstream repository, gives the same. The audit's summary, the search log and every excluded report with its reason are beside the coding in the artifact.

S25.2. Every tool read at source, note by note

The supplementary material's S5 classifies every tool we read at source in its registry table, and keeps notes only on the verdicts a reader may question. This subsection keeps the note on every tool as that section gave it on 15 October 2026, before it was shortened to fit the editor's page budget; the source line behind each verdict is in `docs/results/external/harness_registry.csv`.

- The OpenMessaging Benchmark appears twice because it treats its two spans differently. It times its end-to-end latency on a millisecond clock and filters it, and times its publish latency in nanoseconds and records every sample (Section V-C).
- librdkafka's `rdkafka_performance` also appears twice, and its two spans differ in the same place. Its consumer subtracts the timestamp the producer wrote into the message from its own wall clock, in microseconds, and keeps the

result only if it is above zero and under five minutes. For each value it drops it prints a line saying the value was ignored, but it keeps no count. Its producer times each message from that timestamp to the delivery report on its own clock and records every value.

- The `EndToEndLatency` tool bundled with Kafka cannot produce a value below zero, because it uses one thread, one monotonic clock and a synchronous publish call. It loses the measurement anyway, in two ways. Each sample prints in floating point, but the `long[]` its percentiles come from is filled by the integer division `elapsed / 1000 / 1000`, so on a path faster than a millisecond its p50, p99 and p999 all read zero. And it polls for the message only once its synchronous publish call has returned the acknowledgment, so a message that reached the consumer first is timed no earlier than its acknowledgment. That message's receipt-minus-acknowledgment difference, which this paper measures, is then held at zero by the order of two calls, a later time standing in for the arrival, and nothing counts how often. The registry classes that as a substitution, as it classes Trogdor's floor and Xinfra Monitor's zero, and the test K1 below measured how often. A poll that finds nothing stops the run rather than dropping the message. The companion `ProducerPerformance` takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.
- In `fio` the guard appears in three timing functions, each returning zero for a negative interval under the comment "time warp bug on some kernels?". Nothing records that it fired.
- In `btt`, `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page.
- The `wrk2` load generator detects the negative interval, prints a block that begins "We are about to crash and die", and records the value anyway into its own copy of `hdr_histogram.c`. That copy's `assert()`s are live in the shipped build, and it returns `false` for an out-of-range index, a refusal the tool never checks. Either the assertion aborts the run or the refusal drops the sample, and nothing counts the loss.
- In `emqtt-bench` the `> 0` guard protects a counter and leaves the histogram alone, so nothing is deleted from the histogram; a guard that once deleted from it was removed in January 2025. Its printed average keeps a value at or below zero in its denominator and leaves it out of its numerator, as if it were zero.
- In RabbitMQ PerfTest the method that computes the difference, `getDifference`, returns its absolute value, so a negative span enters its metrics as a positive one of the same size, uncounted. The metrics then record only values at or above zero. From version 2.0.0 of 2017 until version 2.25.0 of June 2026 they recorded only values above zero, so in the tool's millisecond mode every trip shorter than a millisecond was deleted, uncounted.
- The NATS CLI appends every difference, negatives included, to the list its minimum, median and maximum are read from, but records its percentile table into `hdrhistogram-go`, whose bound check refuses the index a negative value maps to. The CLI ignores the error, so its percentile table disagrees with its minimum, median and maximum, and nothing counts the difference.
- Rezolus counts the samples its scheduler sampler discards, those whose two timestamps arrived out of order across CPUs, in a metric that names the cause. The counter was added on 12 August 2026, and its other samplers have none.
- Xinfra Monitor records its end-to-end latency, with no test of its sign, into the metrics library its build pins, `kafka-clients` 2.8.2, whose percentile statistic has a floor of zero and pins a value below it to the floor: a negative latency enters its 99th and higher percentiles as zero, logged only at debug level, while its average keeps it. Its producer and consumer are two services of one process by default and can run as separate processes.
- Strimzi Canary observes each duration into a Prometheus histogram whose lowest bucket is 5 ms, so a negative counts with the fastest messages and shows only as a lower sum. The script the repository ships to read its latency log matches durations with `[0-9]+`, which cannot read a minus sign, so it skips every negative.
- In Kafka's Trogdor the end-to-end latency is the producing agent's wall clock, written into each record, subtracted from the consuming agent's, and recorded as `Long.max(0L, ...)`, so every negative becomes the histogram's floor. Trogdor's histogram throws on a negative: a negative publish latency is lost inside ProduceBench's acknowledgment callback, and in ConsumeBench the exception aborts the task.
- RabbitMQ Stream PerfTest has kept only latencies at or above zero since 2 June 2026, release 1.7.0, the fix for a user's report of a negative latency during a rolling restart. Before that the library it records into refused the value itself and said so once, in a warning.
- The Yahoo Streaming Benchmark's figure is per window: the processing job's clock at a window's last write minus the window's start, which comes from the generator's clock, on another host in the cluster the benchmark describes. The tool writes every value, but the notebook it ships keeps only windows whose figure reaches the window length, so a window whose latency comes out negative is dropped. A user reported values such as $-70{,}646$ ms in 2018, and the report is still open.
- PerfKit Benchmarker's messaging benchmark subtracts a publisher process's `time.time_ns()` from a receiver process's on one machine and floor-divides the difference to whole milliseconds, so a trip under a millisecond reads zero; its mean and percentiles keep every value, sign and all. It measures managed services only: Google Pub/Sub, AWS SQS and Azure Service Bus.
- SBK cuts a value below zero but counts it, the second tool we read to do so. Such a value is kept out of its minimum,

maximum, mean and percentiles, and every report prints how many as invalid latencies; a zero is kept. Its default timestamps are wall-clock milliseconds, and its multi-host mode lets one host subtract another's timestamp, which no document mentions.

- Redpanda's `kgo-repeater` subtracts the timestamping worker's wall clock from that of whichever member of its consumer group receives the record, another process or host in its documented group mode, in whole microseconds. A value at or below zero is kept, unflagged, in the sample its 50th, 90th and 99th percentiles are read from, which are the only figures it reports.
- The Google Cloud Pub/Sub load test framework subtracts one virtual machine's wall clock from another's, in milliseconds, and counts the result into buckets by its logarithm, each bucket half again as wide as the one below. Its Java client puts a zero or a negative into the bucket of the fastest messages. Its controller, the process that gathers the clients' buckets and prints the percentiles, throws when that bucket holds half the values or more. Its Go client, on the build its startup script installs, ends with a panic at the first negative. Its Python client raises before it acknowledges the message, and its Node client files the value under a key named `NaN`, where nothing counts it.
- The Pravega benchmark indexes an array by each latency in whole milliseconds, so the place it records the value refuses a negative, as a library does in the tools above. The exception ends the thread that keeps its statistics: no later window is printed, nor any final percentile, and the only trace is a stack trace at shutdown.
- Apache ActiveMQ Artemis's `artemis perf` runs its producer and its consumer as two processes when, as its documentation suggests, they are on two machines, and records the difference of their wall clocks in microseconds into HdrHistogram 2.2.2, which throws on a negative. The client logs a warning and rolls the session back, and the run's summary still reads `success`.
- Quiver has held its two timestamps as unsigned 64-bit integers since December 2021, so a difference below zero wraps to some 10^{19} ms and enters its average and its top percentiles. Its release of 2019 kept the sign, and its live average still does.
- RocketMQ's benchmark consumer keeps a value below zero in its average, but its maximum starts at zero and only rises, so the value can never appear there.
- Apache BookKeeper's benchmark keeps a publish latency only if it is above zero, because a zero also marks an add that never completed; a latency of zero is dropped with the failures, uncounted. Its clock is monotonic, so no negative can arise.
- IBM Sarama's `kafka-producer-performance` times requests, not messages, on Go's monotonic clock, and truncates each to whole milliseconds, so every request under a millisecond enters its mean and percentiles as zero. `mqtt-benchmark` times each publish to its acknowledgment on the same kind of clock and keeps every value; it has no subscriber.
- Four further tools we read are not in the count, because they do not make the measurement. SPECjms2007 defines Delivery Time and never reports it, Redpanda's `rpk` ships no latency benchmark, Pepper-Box times only the call that hands a message to Kafka's client, and xk6-kafka times whole calls of the script that runs it, four of whose latency figures are always zero.

S25.3. Every tool run, test by test

The supplementary material's S5 gives each tool run one row of a table, with prose for the six tools whose results need explaining. This subsection keeps its account of every run as that section gave it on 15 October 2026, before it was shortened to fit the editor's page budget. The tables it names, of the tools run and of the tests T3, K1 and S1, are in that section, which still gives whole its account of K1, its figures for T3 and the clock's check after the steps of S1; those passages are not repeated here.

Reading a tool tells what it does with a value below zero; running it tells whether it ever meets one. The registered law campaign ran 32 benchmarking tools, four rounds each, in nine blocks, under predictions fixed from their source before any run. The supplementary material's table of the tools run names each tool's block by the freeze and plan version that registered it, and gives the day the block began and the x86 pair it ran on. Each block's plan was public before its runs, the two pairs are of one machine type and processor, and every amendment that touched a block is listed in `freezes/README.md`. Five tools the plan had registered with the first eleven were left unrun by its version 15, and ran in October under version 33. Two timed a managed service: PerfKit Benchmarker's messaging benchmark an Azure Service Bus queue, and the Pub/Sub load test Google's own Pub/Sub, from a Google Cloud project. The queue and the project were made for their runs and deleted once the runs were home. Apache Artemis's perf client, RocketMQ's benchmark and BookKeeper's benchmark each ran against a broker of its own, started alone on the second pair's broker machine, and Quiver ran against that Artemis broker. Quiver and the Pravega benchmark, which ran against Pravega's own server, were built from source on older toolchains. The same table gives what their runs found.

In 39 of the Pub/Sub load test's 48 runs the first attempt never began, because its controller's first call to Pub/Sub timed out. Each such attempt measured nothing, and was set aside and made again a minute later, as version 41 provides. The first

test, T1, adds a delay to the path in steps, which cannot reach a managed service, so for PerfKit and the Pub/Sub load test it was the zero step alone. Of the others, every tool's figure but two followed the delay at a slope inside the band the plan fixed in advance. The Yahoo Streaming Benchmark's did not, as version 36 predicted: its slopes ran from −0.2 to 236 ms per millisecond added. The prediction's reason, read again after the runs, was wrong. It took a window's figure, the time of the window's last write to Redis less the window's end, to be set by the once-a-second flush alone. But the flush writes the hundred campaigns' windows one after another, and each write waits for four replies from Redis, each of which crossed the delay. Within a flush, the writes of one window for successive campaigns were 3 ms apart with no delay added and 10 to 11 ms apart with 2 ms added. In each round the gap grew by 3.1 to 3.7 ms per millisecond added, close to the four replies. (The gap is read in the 27 of its 40 staircase runs whose collected file holds the same windows for every campaign, by `scripts/ysb_post_hoc.py`.) The later a campaign's place in the flush, the later its window's figure, so a millisecond added could move the figure by hundreds of milliseconds.

RocketMQ's benchmark did not follow the delay in any round, against version 39's prediction. Read after the runs (`scripts/rocketmq_post_hoc.py`), two things had happened. Its producer has no rate of its own and published its 3,000 messages in about three seconds, while its consumer is given the run's new queue only when its client next balances its queues, every 20 s. In six of its 76 runs the producer finished first and the broker held the messages until then: five staircase runs averaged 19.4 to 20.3 s, where no run that did not wait averaged more than 5.2 ms. The tool also prints the time from the broker's store of each message to the consumer, and in those runs it equaled the whole, so the wait came after the broker had the messages. The other steps still rose by 1.78 to 2.04 ms per millisecond added, and the store-to-consumer time by 1.63 to 1.97, which leaves the producer out. The consumer asks the broker for messages and asks again when the answer comes. A message stored just after an answer has left therefore waits for the next request, which follows that answer across the delay, and then crosses it itself. The prediction counted one crossing where the consumer makes about two. The fourth round's reference run, from which T2 takes its offsets, waited too, 139 ms, so that round's offsets were longer than the others' by about as much.

A second test, T2, moved back the clock of the process that subtracts, at two offsets longer than the interval the tool times. A program registered with the plan, the judge, then chose for each run among five behaviors: keeps every value, drops the negatives, drops zero and below, replaces the negatives with zero, and times against one clock. The second block exposed two faults in the judge. First, a tool that kept fewer values than it was asked to publish was ruled out of the behaviors that keep every value, but not out of timing against one clock. Yet a tool on one clock keeps every value too, so a run of Pulsar's client that kept 7 of 3,000 latencies was judged to time against one clock. Second, the judge predicts each behavior's figures from the trips of a reference, our own client for the same server, not from the tool's. A tool with a slow tail of its own can therefore be ruled out of the behavior it shows. Version 34 (`freezes/31-experiment-plan`), public before either block was judged again, corrects the first fault. It also decides how many clocks a tool reads from the offset alone, without the reference. A tool reads two if it lost values, or if its average, median or minimum moved by more than half the offset. It reads one if it kept every value and none of those figures moved by a quarter of the offset, and the rule leaves it undecided otherwise. Every T2 run of both blocks was judged again under both rules, beside the verdict it was given. The corrected rule changes two verdicts, both for runs of Pulsar's client. The run above becomes undecided, and one undecided run becomes "drops the negatives".

That table gives each tool's answer in its Clocks column. Its Runs column counts the tool's runs with the clock moved back in which version 34's rule gave the same answer. A tool is counted below as reading one clock, or two, only if the rule decided some of its runs and every run it decided agrees with its Clocks column. Three tools are therefore in neither count, for reasons given below: the Pravega benchmark, Apache Artemis's perf client and the Pub/Sub load test. Of the tools run, 19 read both timestamps from one clock, and moving the clock left their figures within a quarter of the offset in 151 of their 152 runs. In the other, SBK's minimum, which it prints in whole milliseconds, moved by one, more than a quarter of that run's offset. Xinfra Monitor and Strimzi Canary are among them: each runs its producer and its consumer in one process. All but PerfKit Benchmarker take both timestamps in one process; PerfKit's publisher and receiver are two processes of one program on one machine, and T2's offset moved the clock of both. Freeze 21's judge left all of PerfKit's clock-moved runs undecided, because its allowance for noise comes from a staircase, which PerfKit has none of; version 34's answer from the offset alone is one clock in every one of them, as version 37 predicted. BookKeeper's benchmark, which times each add on a monotonic clock, is among them, and its own rate limiter enters every latency it keeps. At 50 adds a second each add waits about a fiftieth of a second for its permit, inside the call the benchmark times, and its median was 25.4 to 25.9 ms in every round, above the 15 ms version 39 predicted.

The Pravega benchmark takes both timestamps in one process too, but it is not among the 19 tools counted as reading one clock. Nothing in its figures moved with the clock, yet version 34's rule did not find the one clock version 40 predicted. Its T2 moved the machine's own clock rather than use libfaketime, because version 40's pilots had found that libfaketime slows the benchmark, a Java program, at an offset other than zero, whether or not the monotonic clock moved with the wall clock. Under it the benchmark's median rose from 1 to 2 ms, and in some runs its reader waited out its one-second timeout. With the machine's clock moved, its median stayed where its control's was in every run, and its average moved by at most 0.3 ms. The count is what misled the rule. The benchmark runs for a fixed minute, set-up included, and counts only what arrives in it, so in all but one run its count fell 43 to 44 short of the 3,000 asked for, and its controls fell as short. Version 34's rule

reads a count shorter than its control's as values dropped. In five of the benchmark's eight runs the count came out one below its control's, and the rule answered two. It was undecided in the two whose counts matched their controls', and answered one only in the one run whose count was whole, where after a stall its writer had published faster than its pace to catch up. Freeze 21's judge, which does not hold a short count against one clock, found the benchmark timing against one clock in seven of its eight runs.

The ten tools counted as reading two clocks timestamp a message in one process and subtract in another, `kgo-repeater` on another machine. The judge read librdkafka's `rdkafka_performance` as keeping every value in three of its four rounds, but its consumer drops any value at or below zero and prints a line for each. No such line appears in any of its runs, and none of its own values came near zero, so that verdict tested nothing. `kgo-repeater` keeps every value: its median fell by the offset in every run, below zero once the offset passed its trip. RocketMQ's benchmark adds every value to the average it prints, which fell by the offset in every round as version 39 predicted, while its maximum, which starts at zero and rises only past it, cannot show a value below zero. Pulsar's client drops a value below zero and keeps a zero. Under the larger offset it kept 3 to 7 of 3,000 latencies, yet its summary printed 3,000 records received beside latency figures computed from those few. The OpenMessaging Benchmark drops every value at or below zero (the supplementary material's S4.5). `emqtt-bench`'s histogram kept every value, its sum going below zero with the clock. Its console printed no latency in any of its offset runs, because it prints its once-a-second average only when a counter that adds the latencies above zero changes. Stream PerfTest drops a value below zero: under the larger offset it kept 3 to 4 of 3,000 latencies. Kafka's Trogdor replaces it with zero, so a moved clock can lower its figures only as far as zero, that is by its own latency. That is more than half the smaller offset and less than half the larger, and so its runs at the larger offset are undecided.

Apache Artemis's perf client reads two clocks as well, and its consumer refuses a value below zero. Its histogram throws before it counts such a value, the client logs AMQ132001 and rolls the message back, and the broker delivers it again, up to ten times, until it comes out at or above zero. Under the offsets it received 5,992 to 8,972 messages for 3,000 published, logged 2,992 to 5,972 such warnings and reported success, in every round, as version 39 predicted. What its histogram kept were the deliveries that came out above zero, many a little above, so its median moved by less than half the offset. Version 34's rule therefore answered one or undecided in all eight of its runs, never two. Version 39 had declined to predict that answer, for this reason, and the table lists the tool as two clocks with none of its runs so answered.

Quiver reads two clocks, because its receiver subtracts the timestamp its publisher wrote into the message from a reading of its own clock. It writes both timestamps of every message, which it calls a transfer, to a file of its own, and its summary subtracts them in unsigned 64-bit integers. A value below zero therefore becomes 2^{64} less its size, about 1.8×10^{19} ms: it wraps. Under the offsets Quiver's maximum wrapped in every run. At the larger offset 2,995 to 2,999 of its 3,000 transfers fell below zero, so its median and average wrapped too. Version 40 predicted that both its maximum and its average would pass 10^{18} ms at both offsets (T2-q). The prediction failed in one round, in which only 1 transfer fell below zero at the smaller offset, so its average stayed far under 10^{18} ms. In that round libfaketime, which moved the receiver's clock, also slowed the receiver by about a millisecond, as in the pilots. Its usual trip became as long as the offset, so moving the clock back by the offset left its median at zero rather than below it. In the other three rounds 2,308 to 2,992 of its transfers fell below zero at the smaller offset, and T2-q held. Freeze 21's judge left all of its runs undecided, and so did version 34's judge, because neither has a behavior for a value that wraps. Version 34's answer from the offset alone needs none, and it was two in every run.

The Pub/Sub load test reads two clocks, because its subscriber subtracts the time its publisher wrote into the message from a reading of its own clock. It counts each value into a bucket by the logarithm of its milliseconds, so every value at or below a millisecond, a value below zero included, falls into the first bucket, with the fastest. Its controller prints each percentile as the bounds of a bucket one below the values in it. The trip T2 took its offsets from was therefore the top of the bucket above the median its reference run printed (D41-3). Under the offsets, 26.6 to 89.5 ms, the first bucket held 87 to 120 of the 120 values its subscriber counted in each run. Its search for the median fails once that bucket holds half the values. In every run its controller then threw an exception reading "Programming error: negative values appear in buckets.", printed no figure for its subscriber and wrote no results file, and the runner stopped it at its time limit, as version 41 predicted (T2-l). Each control printed its figures and exited normally. Both judges therefore left all its runs undecided, and the table lists it as two clocks with no run answered either way.

Its T4 run moved its subscriber's clock a second ahead instead, which raised every value by 1,000 ms. Yet in every round it printed its median's range as 656.8 to 985.3 ms, wholly below a second, because it prints the bucket below the values, as version 41 predicted (T4-l).

The Yahoo Streaming Benchmark is the one tool whose notebook drops values with no clock moved: 20–26% of its windows, those whose last write came before the window's end, as version 36 foresaw (D36-6). Both judges take their base from a reference that drops nothing, and both failed on it. Freeze 21's judge read five of its eight clock-moved runs as timing against one clock, because its allowance for a change is three times the scatter of the tool's figure across the T1 staircase, and the flush had spread YSB's figure there by hundreds of milliseconds. Version 34's judge left all eight undecided, while its answer from the offset alone is two. Read directly against YSB's own reference instead, "drops the negatives" fits every run. It predicts that 49–50% of windows are kept, against 46–58% kept, and averages within 38 ms of those printed. Every run's average fell by 179 ms or more from its round's control, where one clock predicts no fall. This reading was made after the runs, and the

verdicts stand as given (`docs/results/tools/ysb_t2_fit.csv`).

The figures of the third test, T3, a background load with and without real-time priority, leave out three tools, because a wait for a core cannot be read in theirs. The Yahoo Streaming Benchmark's figure is set by its flush, as above, and not by the trip a waiting thread lengthens. PerfKit Benchmarker's trip goes to a managed queue in a data center, and its 99th percentile of 100 messages, set by that queue's own tail, swung between its idle rounds by far more than a wait for a core adds. The Pub/Sub load test prints each percentile as a bucket's range, and the range it printed for its idle 99th percentile, the median over the four rounds, was some 13 ms wide, wider than the most any tool's 99th percentile rose under the load.

The step test: A tool whose two timestamps come from one clock meets a value below zero only if that clock steps back between them, as it did for a user of Stream PerfTest during a cluster's rolling restart. T2's constant offset cannot make one, so version 37 (`freezes/34-experiment-plan`) added S1. Each of five tools ran once as it is and once while the driver's wall clock stepped back by 20 ms, or by 50 ms for PerfKit, whose trip is longer, at gaps drawn around a fifth of a second, with chrony stopped. A message still on its way when the clock steps back is timed short by the step, and so comes out below zero if the step is longer than its trip. Every prediction version 37 fixed from the tools' sources held in all four rounds. PerfKit printed 8 to 18 samples below zero in each stepped run and counted them in its mean and percentiles. Strimzi Canary logged 10 to 20 durations below zero. Stream PerfTest, run as one instance, kept 16 to 25 fewer latencies than the messages it consumed, and said nothing. Xinfra Monitor's metrics library pinned 40 to 84 values to the floor of its percentiles, which only its debug line shows: its printed 99th percentile was the control's in every round. None of them showed such a value in its control, and Kafka's end-to-end tool, which times with a monotonic clock, did not see the steps. T2 had already shown Stream PerfTest's handling, when it ran as two instances. The other three could not have been seen without the steps. Versions 38 and 39 added four tools to S1. SBK counted 26 to 58 of its latencies as invalid in each stepped run, and none in its controls. `mqtt-benchmark`, on a monotonic clock, kept its minimum above zero and its mean within a millisecond of its control's. Sarama's producer tool, also on a monotonic clock, kept its median, but its stepped 99th percentile fell 1.3 to 3.8 ms below its control's, so S1-i, which allowed a millisecond either way, failed in every round. The steps cannot reach its clock, and the figure fell rather than rose: the millisecond allowed was narrower than that figure's spread from one run to the next. Artemis's producer, which reads two clocks with its consumer, published 2 to 10 of its 3,000 messages in each stepped run, stopped, and reported success, as version 39 predicted after its pilot. A step inside one of its own timed publishes makes a value below zero that its histogram refuses, in a thread that catches only the messaging library's own exceptions. Version 40 added two more. In each stepped run 9 to 25 of Quiver's transfers came out below zero and its maximum wrapped, while its controls had none. The Pravega benchmark printed no total in any stepped run. At the first value below zero its one statistics task threw an `ArrayIndexOutOfBoundsException` naming that value, and every later statistic was lost. Its code then exits with status 0, as it does after printing any exception, so a run that lost its statistics ends as one that succeeded; the runs did not record that status. Each control printed its total. Version 41 added the Pub/Sub load test, on two clocks, stepped back by 50 ms, as PerfKit was, because its trip is longer than 20 ms. Nothing it printed mentioned a value below zero, which it puts in its first bucket, with the fastest. Its clients time how long they have run on the wall clock, from a start given on that clock. The steps held that clock back, so the clients reported their first burst of messages with a running time below zero, -7.7 to -6.8 s. Its controller counts only what arrives after the first minute of running time, and on the held-back clock that minute lasted longer, so it counted 80 values in each stepped run where its controls counted 120. Version 41 predicted that the median a stepped run printed would lie in its control's bucket or one either side (S1-l). That held in three rounds and failed in one, where the stepped median lay two buckets below its control's. The steps put 17 of that run's 80 values into the first bucket, but without them its median still lies two buckets below. Read after the runs, the counts show that Pub/Sub delivered faster in that stepped run than in its control minutes before, and the one bucket allowed was narrower than that drift.

The per-run verdicts and figures are in `docs/results/tools/tools_t1.csv`, `tools_t2.csv` and `tools_t3.csv`, folded by `scripts/tools_block.py` from the runs. K1's are in `docs/results/tools/k1.csv`, judged by `scripts/kafka_clamp.py`, and S1's in `docs/results/tools/s1.csv`, `s1_v38.csv`, `s1_v39.csv`, `s1_v40.csv` and `s1_v41.csv`, judged by `scripts/s1_block.py`. The predictions of versions 39, 40 and 41 about their own tools are in `docs/results/tools/v39.csv`, `v40.csv` and `v41.csv`, judged by `scripts/block_v39.py`, `scripts/block_v40.py` and `scripts/block_v41.py`. The readings after the runs are in `docs/results/tools/ysb_flush.csv` and `ysb_t2_fit.csv` for the Yahoo Streaming Benchmark, `rocketmq_waits.csv` and `rocketmq_t1.csv` for RocketMQ's benchmark, and `pubsub_runs.csv`, written by `scripts/pubsub_post_hoc.py`, for the Pub/Sub load test. The write-ups are `docs/results/tools/t1-t4-eleven-tools.md`, with its five recorded faults, `docs/results/tools/t1-t4-five-tools.md`, `docs/results/tools/t1-t4-october-tools.md`, `docs/results/tools/t1-t4-perfkit-and-s1.md`, `docs/results/tools/t1-t4-freezes-35-36.md`, `docs/results/tools/t1-t4-freeze-37.md` and `docs/results/tools/t1-t4-freeze-38.md`.

S26. TABLES

Table S17 is the per-configuration ledger behind Section V-D's grid claim: one row per publish rate, with the arithmetic denominator q, the replicate count, and the spread those replicates actually showed. Section V-D keeps the inference drawn

TABLE S17

REPLICATE SPREAD AGAINST THE PHASE DENOMINATOR q (MOVED FROM SECTION V-D), AT FIXED PAYLOAD, LOAD, DURATION AND HOST. *Pos.* IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST GRID POINT, AGAINST THE CELL HALF-WIDTH: 0 SITS ON A GRID POINT, 1 MIDWAY BETWEEN TWO. POSITION IS SCALE-FREE, SO EVERY CALL IS UNCHANGED WHETHER THE CONTINUOUS VALUE IS ESTIMATED POOLED OR PER RATE. *Predicts* IS *full* WHERE THE POSITION EXCEEDS A HALF; *shows* IS *full* WHERE THE MEASURED SPREAD EXCEEDS HALF THE STEP WIDTH. BOTH ARE RECOMPUTED FROM THE GEOMETRY AT BUILD TIME, SO THE LAST TWO COLUMNS CAN BE READ AGAINST EACH OTHER: 10 OF THE 12 COMMENSURATE ROWS AGREE, AND THE TWO THAT DO NOT ARE 700 AND 1250 MSG/S, DAGGERED AND SET IN BOLD HERE AND DISCUSSED IN SECTION S10. BOTH ARE RESOLVED IN THE MODEL'S FAVOR BY THE REPLACEMENT STATISTIC OF TABLE S7, WHICH DOES NOT USE SPREAD. INCOMMENSURATE CONFIGURATIONS HAVE NO CELL AND THEREFORE NO CLASS: THEIR *predicts* ENTRY IS THE NOISE FLOOR RATHER THAN A PREDICTION, AND THEIR MEASURED SPREADS — ABOUT THREE POINTS — ARE THAT FLOOR, NOT A DISAGREEMENT. ROWS ARE ORDERED BY q, WHICH IS WHAT DECIDES THE PREDICTION. EVERY INCOMMENSURATE RATIO SHOWN IS ALREADY IN LOWEST TERMS.

rate	T_{pub}/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s†	4/5	5	5	20.0	0.91	full	7.6	**flat**
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s†	10/7	7	5	14.3	0.87	full	6.5	**flat**
900/s	10/9	9	5	11.1	0.84	full	6.9	full
incommensurate — phase set effectively continuous								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

TABLE S18

RETENTION AGAINST THE PRODUCER'S PUBLISH INTERVAL, WITH MESSAGE SIZE, BACKGROUND LOAD, DURATION AND HOST HELD FIXED. ONLY THE COMMENSURATE RATE IS UNSTABLE, AND THE TWO INCOMMENSURATE RATES AGREE WITH EACH OTHER DESPITE INTERVALS DIFFERING BY 19%. PREDICTIONS WERE RECORDED BEFORE THE RUNS.

Rate	Interval	Multiple of 1 ms?	Retention per replicate	Spread
500/s	2.000 ms	**exact**	0.47, 1.51, 1.55, 18.02, 99.99	99.5
457/s	2.188 ms	no	48.77, 49.50, 49.69, 50.51, 50.87	2.1
383/s	2.611 ms	no	50.28, 51.09, 51.45, 51.77, 53.13	2.8

from it and the set view of the same configurations; this is the record they are drawn from, and every cell in it is recomputed from the committed campaign index by the test suite rather than proofread. Table S18 keeps the individual replicate retentions for three configurations, which the ledger reduces to a single spread: it is the same evidence at finer grain, and it is where the 0.47-to-99.99 range behind the $q = 1$ corner can actually be seen.

Two further tables hold evidence Sections IV and V spend a line on each. Table S19 sweeps payload at an incommensurate rate, where samples stay dephased and implied latency can be read off directly. Table S20 is the load sweep behind the queueing form we withdrew; it is kept because a fit we abandoned should stay inspectable by anyone who wants to disagree with the abandonment. A third bounds the selection effect unequal audit loss could have introduced into E1's broker equivalence: Table S21 imputes every rejected Redis run at the largest value its own cell allows, against a 1 ms margin. The equivalence that Section VI-D of the main text reports rests on the powered campaigns instead, and S18.1 bounds the selection there.

S26.1. The mechanism campaigns' tables

Table S22 is the mixture across nine load levels; Table S23 the waiting probability manipulation; Table S24 the load-geometry comparison with its replication; Table S25 the payload sweep; and Table S26 the kernel-trace comparison, including the withheld configuration.

S26.2. The audit, timestamping-mode and injected-delay tables

Table S27 is the consistency audit of every run; Table S28 the two timestamping modes; Table S29 the injected one-way delay.

TABLE S19
RETENTION AGAINST PAYLOAD AT 457 MSG/S, INCOMMENSURATE WITH THE MILLISECOND GRID SO SAMPLES STAY DEPHASED. IMPLIED T_{true} INVERTS EQUATION 6. THE FINAL COLUMN IS THE INCREMENT IN IMPLIED LATENCY PER BIT OF ADDED PAYLOAD, IN NANOSECONDS, RELATIVE TO THE 200 B BASELINE; THE TWO SMALLEST STEPS ARE NOISE-DOMINATED AND MARKED.

Payload	Retention (3 reps)	Median	Implied T_{true}	ΔT/bit (ns)
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms	baseline
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms	$-2.600^{\dagger}$
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms	$1.752^{\dagger}$
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms	$0.686^{\dagger}$
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms	$0.499^{\dagger}$
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms	**0.630**
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms	**0.638**

† added serialization under 0.07 ms, comparable to the 0.02–0.04 ms replicate noise, so the ratio is scatter over a small denominator. The two unmarked levels have a signal-to-noise ratio near 20 and carry the result.

TABLE S20
H2: NEGATIVE-SPAN RATE AGAINST ACHIEVED CPU UTILIZATION, NO CORE PINNING, BACKGROUND LOAD SWEPT FROM IDLE TO OVERSUBSCRIPTION. EACH ROW POOLS A CONDITION'S KAFKA AND REDIS RUNS (30 RUNS, 3,582 EVENTS); THE MIXTURE OF TABLE S22 AND THE KNEE SWEEP COUNT KAFKA'S ALONE. THE WITHDRAWN QUEUEING FORM FITS AT $R^2 = 0.945$ AGAINST 0.640 FOR A STRAIGHT LINE.

Utilization ρ	Negative span rate
0.003	0.007
0.253	0.009
0.504	0.022
0.628	0.047
0.753	0.132
0.878	0.207
1.000	0.208
1.000	0.221
1.000	0.264

Magnitudes of this size cannot be a per-event cost: The account we test is a round-trip-bound consumption pattern: our Redis consumer acknowledges each entry with `XACK` and waits — the idiomatic tutorial pattern — so a read of two hundred entries is followed by two hundred sequential round trips, free at 0.22 ms per trip and hopeless at 20 ms, where service rate falls below arrival rate and delay is set by backlog [S62]; Kafka's consumer serves from a prefetched buffer and commits on a timer, no per-record round trip to expose.

The falsification criterion was stated in advance: if amortizing the acknowledgments does not help, the account is wrong. At one feed under true real-time replay, batching them in groups of two hundred takes median event-time latency from 4,138 ms to 103 ms — a factor of **40.2**, replicated four times — and read-loop instrumentation corrects the shape: each `XREADGROUP` returns every waiting entry (median 106 messages in ∼32 ms), so the constraint is entirely in the acknowledgment path.

PART IV. THE LITERATURE, AND WHAT IT ALREADY REQUIRES

This part places the result. Sections S27 and S28 locate our irreproducibility in the variability literature, and the deletion failure against a resolution literature that knew the problem without reaching this. Sections S29 to S32 cover the second clock, where other runtimes write the timestamp, the rules neighboring communities already impose, and what standardized benchmarks require about resolution. Section S33 states where our reading parts from concurrent work.

S27. WHERE OUR IRREPRODUCIBILITY SITS IN THE VARIABILITY LITERATURE

Run-to-run instability has a supporting literature, and it is on our side: The irreproducibility we measure — three-replicate medians reproducing at one load level in five, and the distribution *family* itself moving between passes of an identical configuration (S1.15 and S10) — sits in a line of work showing that benchmark variability is structural, not noise. Mytkowicz et al. [S63] showed that factors as innocuous as link order and environment-variable size flip published conclusions; Georges et al. [S64] that JVM results without confidence intervals routinely mislead; Barrett et al. [S65] that virtual machines frequently never reach the steady state their benchmarks assume, so the distribution being sampled is itself non-stationary — the closest published analogue of our regime mobility. Maricq et al. [S66] found hundreds of trials necessary for stable comparisons on real hardware, and Kalibera and Jones [S67] supply the replication framework. Against this stands common practice rather than any contrary finding: the surveyed norm of a handful of repetitions [S68], [S69] presumes a stable distribution that our phase-locked configurations demonstrably lack. We add the mechanism: at these configurations the instability is not variance around a value but a two-point support whose weights move, so no replicate count converges (S10).

TABLE S21
BOUNDING THE UNEQUAL-RETENTION SELECTION EFFECT IN E1. "WORST CASE" IMPUTES EVERY REJECTED REDIS RUN AT THE LARGEST VALUE OBSERVED IN THAT CELL. MARGIN 1 MS.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	$+0.081$	$+0.081$	yes
9	20/27	$+0.021$	-0.051	yes
10	17/30	$+0.116$	-0.485	yes
12	19/36	$+0.053$	-0.227	yes

TABLE S22
MIXTURE, MEASURED ACROSS NINE LOAD LEVELS (25 RUNS EACH, PRE-REGISTERED). THE CORE WIDTH $\sigma_{\text{core}} = \text{IQR}/1.349$ GROWS $5\times$ FROM IDLE TO THE KNEE; THE NEGATIVE-SPAN RATE — THE TAIL — GROWS $61\times$. NEGATIVE SPANS ARE TEMPORALLY CLUSTERED AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION COORDINATE CANNOT SEPARATE.

Utilization ρ	Core σ_{core} (ms)	Negative span rate	Clustering z
0.003	0.126	0.004	-6.8
0.253	0.138	0.004	-6.8
0.504	0.180	0.006	-6.8
0.628	0.181	0.024	-6.9
0.753	0.238	0.076	-5.7
0.878	0.631	0.224	-4.9
1.000	1.224	0.371	-4.3
1.000	1.293	0.317	-4.7
1.000	1.710	0.261	-4.4

Result by result: For each headline result, the nearest support and the nearest constraint we can find:

- *Deletion law and grid structure* (Sections V and V-D). Supported mathematically by equidistribution and dither theory [S70]–[S72], and practiced in profiling since randomized sampling clocks were introduced to break exactly this kind of synchronization [S73]; no empirical study reports a benchmark's retained fraction, so the nearest contradiction is implicit — every published OMB-derived latency figure [S1] assumes retention near 100%, an assumption our 0.36%–100% range refutes for sub-tick paths.
- *Summary insensitivity* (Section V). Supported in spirit by Ousterhout's injunction to measure beneath the summary [S74] and by reporting standards that require sample counts [S69]; contradicted by no study, but silently contradicted by every report of a latency percentile without the count behind it.
- *Irreproducibility and regime mobility* (S1.15 and S10). Supported by [S63]–[S66]; constrained by the same literature's remedy — more replicates — which our two-point supports show cannot converge here.
- *Scheduling-stall negative spans* (Section IV). Supported by the tail-latency and scheduler literature [S75]–[S77] and by the concurrent negative-span reports [S18]; qualified by Sharma et al.'s skew-threshold reading, which our zero-skew negative spans show is unsafe under load.
- *The sign audit* (Section V-F). Ten million discarded samples, not one negative. Nothing in the literature contradicts a count; what it refuted was our own prior claim, which is the point of publishing it.

Li et al. [S75] decompose the tail latency of three Linux servers into hardware, operating-system and application-level sources, and report the finding that matters most for us: measured tails are *substantially worse* than a queueing model alone predicts, with the excess traced to interference from background processes, request re-ordering under constrained concurrency, interrupt routing, power management and NUMA effects. A single-parameter queueing description is therefore a leading-order account, not a complete one — which is precisely what Section IV-C measures directly on the harness rather than on the system under test. Their background-interference mechanism also predicts the temporal signature we observe: a core blocked by another task delays not one event but every event arriving in that interval, so the resulting failures should appear in consecutive bursts. Section VIII tests that prediction and finds exactly this clustering.

Our contribution here is therefore not the observation that scheduling delay is heavy-tailed, which is established, but that this shape propagates into the *measurement* of an unrelated quantity, and that load moves the tail's weight far faster than the core's width (Section IV-B). The practical consequence is specific: a benchmark can sit at a utilization where its measurements look tight — the core is narrow — while the tail that actually corrupts them is already growing.

The presumption that a ready thread runs is not ours: It is documented: Lozi et al. [S77] document Linux scheduler bugs under which cores stay idle *for seconds* while runnable threads queue elsewhere, invisible to conventional testing because throughput stays healthy. Under exact work conservation a timestamping thread would never wait while a core was free, and the geometry contrast of the table in Section S26 would have to be flat; it is not, and it converges only where no core is free to migrate to.

TABLE S23

WAITING PROBABILITY, NOT UTILIZATION. TIMESTAMPING PROCESSES AT NORMAL PRIORITY VERSUS `SCHED_FIFO`, WITH THE BACKGROUND LOAD UNCHANGED. UTILIZATION IS MEASURED IN BOTH CONFIGURATIONS, NOT ASSUMED: IT MATCHES TO 0.001, SO THE MANIPULATION MOVED WAITING PROBABILITY ALONE. THE NEGATIVE-SPAN RATE FALLS BY A FACTOR OF 39–54 TO THE RUNNING-STATE FLOOR. UTILIZATION-ONLY MODELS PREDICT NO CHANGE.

	Utilization ρ		Negative span rate		
Load	normal	real-time	normal	real-time	factor
75%	0.7531	0.7525	0.1320	0.0034	39×
88%	0.8809	0.8812	0.3049	0.0057	54×

TABLE S24

SAME UTILIZATION, DIFFERENT RATE. BOTH LOAD GEOMETRIES IN ONE CAMPAIGN. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. AT $k = 6$ THE TWO CONFIGURATIONS SIT AT THE SAME ρ TO FOUR DECIMALS — IN *both* CAMPAIGNS — AND THE RATES DIFFER TWOFOLD IN BOTH. EVERY CELL IS 2,497 TO 2,985 MATCHED EVENTS OVER 25 RUNS; z IS A TWO-PROPORTION TEST BETWEEN THE CONFIGURATIONS OF THAT ROW. THE $k = 7$ ROW IS WHERE THE TWO CAMPAIGNS DISAGREE: THE ORIGINAL CANNOT SEPARATE THE GEOMETRIES THERE, THE REPLICATION CAN, AND WE WITHDRAW THE STRONGER READING (TEXT).

	E-A6 (original)			E-A6b (replication)			
k/C	concentrated	spread	z	concentrated	spread	z	factors
5/8	0.0201	0.0379	4.09	0.0128	0.0584	8.83	1.88×, 4.55×
6/8	0.0824	0.1709	10.27	0.0706	0.1429	8.34	2.07×, 2.02×
7/8	0.2281	0.2415	1.22	0.2359	0.2752	3.20	1.06×, 1.17×

Achieved ρ, $k = 6$: 0.7531 in all four configurations. $k = 5$: 0.6284/0.6250 and 0.6284/0.6259. $k = 7$: 0.8778/0.8809 and 0.8778/0.8822.

S28. THE RESOLUTION LITERATURE KNEW THE PROBLEM; THE DELETION FAILURE IS NEW

The problem is older than every tool in the audit: Danzig and Melvin were building microsecond timers for workstations in 1990, and said why in their title: the available clock was too coarse for the interval being timed [S78]. Nothing in the deletion failure is a new observation about clocks. What is new is that the tools which inherited the problem stopped counting what it costs them, and that the cost is computable in advance from two numbers an operator already knows.

No defence of the admission condition has been found, and we looked for one: The audit of Section V-A finds the practice without ever finding a stated reason for it, which is a weaker position than finding a reason and rebutting it, so we record the search. Benchmark-framework papers that do treat validity systematically stop short of this particular threat: ComBench, a framework for communication benchmarking, devotes a threats section to validity but treats latency measurement only in passing [S79], and the same is true of the harness documentation surveyed above. We therefore claim only that we could not find a defence, not that none exists.

Coordinated omission is the closest neighbor, and is the mirror image: In Tene's *coordinated omission* [S80] the worst samples are never taken, so percentiles are optimistic by orders of magnitude; here the samples *are* taken, the latency *is* computed, and a downstream admission condition rejects the result — one loses the slow tail, the other the fast bulk. Both end with a confident summary over a biased subset, and a correction for one does nothing about the other.

Not hypothetical: the benchmark we instrument records into an `HdrHistogram` [S81], which ships explicit coordinated-omission machinery and none for this failure — its sound rejection of negative values is precisely what motivates the admission condition whose `else` branch silently deletes the sub-tick samples: a protection against one measurement artifact created the conditions for another.

The mathematics is a century old; the interaction is unreported: Weyl's equidistribution theorem [S70] governs the irrational regime of Δ/τ in the long run, and the three-distance theorem [S71] the phases a finite run of publishes visits; Schuchman's condition [S72], generalized by Wannamaker et al. [S82], is the condition behind the remedy that Section VI states as randomizing the publish instants. The timing literature treats a coarse tick as staleness, not deletion. What we claim as new, narrowly: not quantization — elementary, and Cloudprofiler [S83] exists because milliseconds are known to be too coarse — but the *interaction* of quantized timestamp, few-phase producer and admission condition, producing a retention fraction bimodal, irreproducible between identical runs, invisible in the reported summary, its damage set by the denominator of the interval-to-resolution ratio in lowest terms rather than by "round" rates.

Run-to-run instability has a supporting literature, and it is on our side: Mytkowicz et al. [S63], Georges et al. [S64], Barrett et al. [S65] and Maricq et al. [S66] establish that benchmark variability is structural, that factors as innocuous as link order flip published conclusions, and that hundreds of trials can be needed — against the surveyed norm of a handful of repetitions [S68], [S69]. Our phase-locked configurations supply the mechanism those studies lack: instability that is not variance around a value but a two-point support whose weights move, so no replicate count converges (S10).

TABLE S25

LONGER END-TO-END LATENCIES ARE MORE RELIABLE MEASUREMENTS, TWICE. PAYLOAD PADDING LENGTHENS THE TRUE TRANSPORT WITHOUT TOUCHING THE SCHEDULER. UTILIZATION IS HELD TO A 0.012 SPREAD; THE SERIALIZATION COST PUSHES IT SLIGHTLY UP, AGAINST THE PREDICTED DIRECTION. BOTH QUANTITIES MOVE MONOTONICALLY, IN OPPOSITE DIRECTIONS, IN BOTH CAMPAIGNS: THE MEDIAN OF S SPANS $74.3\times$ AND $77.3\times$, THE RATE FALLS $3.85\times$ AND $4.04\times$. EACH CELL IS 2,408 TO 2,980 EVENTS.

	E-A10		E-A10b		
Padding (B)	Median S (ms)	Rate	Median S (ms)	Rate	ρ span
0	0.697	0.2627	0.670	0.2749	0.881–0.882
4,096	2.378	0.1909	2.199	0.1944	0.882–0.882
65,536	14.469	0.0901	13.973	0.0928	0.885–0.885
262,144	51.768	0.0683	51.795	0.0681	0.893–0.895

TABLE S26

KERNEL TRACING PREDICTS THE NEGATIVE-SPAN RATE, UNFITTED, IN EVERY CONFIGURATION THAT HAS ONE. $\Pr[\text{STALL} > T_{\text{true}}]$ FROM `SCHED_WAKEUP`/`SCHED_SWITCH`, AGAINST THE NEGATIVE-SPAN RATE MEASURED FROM APPLICATION TIMESTAMPS IN THE *same* RUNS: BOTH BROKERS POOLED, AS THE TRACE COUNTS BOTH, AND KAFKA'S ALONE; THE TWO SHARE NO PROBE AND NO ESTIMATOR. EACH BROKER CONTRIBUTES 2,257 TO 2,985 EVENTS A CONFIGURATION. THE THREE NORMAL-PRIORITY CONFIGURATIONS AGREE WITH THEIR TRACED TAILS TO WITHIN A FIFTH ON THE POOLED RATE, WITH NO CONSISTENT SIGN. EVERY REAL-TIME CONFIGURATION READS EXACTLY ZERO ON KAFKA WHERE THE UNTRACED CONTROL OF THE E-A9 PAIR READS 15 IN 2,985; THAT IS A TRACING ARTIFACT, NOT A FLOOR (TEXT).

						Both brokers		Kafka alone	
Campaign	Priority	Load	ρ	Traced wakeups	$\Pr[\text{wait} > 0.5\,\text{ms}]$	Rate	Ratio	Rate	Ratio
E-A9	normal	88%	0.8822	551,956	0.181	0.214	0.84	0.272	0.66
E-A9	real-time	88%	0.8819	570,591	0.018	0.011	1.63‡	0.000	—
E-A9b	normal	75%	0.7543	709,819	0.168	0.144	1.17	0.164	1.02
E-A9b	real-time	75%	0.7531	641,308	0.016	0.008	1.95‡	0.000	—
E-A9b	normal	88%	0.8825	687,986	0.207	0.205	1.01	0.259	0.80
E-A9b	real-time	88%	0.8822	649,788	0.016	0.009	1.71‡	0.000	—
Untraced control: E-A9's first attempt, probe never attached									
—	normal	88%	0.8812	—	—	0.214	—	0.273	—
—	real-time	88%	0.8809	—	—	0.013	—	0.005	—

‡ Kafka's half of the pooled rate reads zero under the tracer; shown, not used.

S28.1. The payload sweep's underpowered rows, and the comprehensive campaign's duration and plateau paragraphs (moved from the main text)

The sweep discriminates only where the manipulation exceeds the noise, and we say which levels those are. Replicate noise in the implied latency is 0.02–0.04 ms; at 32 and 64 KB the added serialization (0.26, 0.52 ms) exceeds it near twentyfold and the per-byte estimates agree, while below 8 KB it is comparable to the noise and the ratios scatter (-2.600, 1.752, 0.686, 0.499) — rows reported rather than dropped, because omitting them would make the table look more decisive than the experiment was. The law rests on the two largest levels; our first attempt sat entirely inside the underpowered regime and settled nothing, so the sweep was extended rather than reported.

Duration does nothing, which kills the natural alternative: A cumulative drift walk would scale with run length — registered: intermediates rise with duration. Measured over one, three and ten minutes at 500 msg/s: intermediate counts 1, 1, 0, branch values pinned to the vertices throughout (0.43, 1.11, 100.00 at ten minutes), probability at most $(1/3)^3 \approx 0.04$ under the walk's own reading. The walk is falsified; what stands is per-publish jitter, applied at each publish, not accumulated.

The continuous value plateaus, and one miss stands: Incommensurate configurations at 1053 and 1219 msg/s measured $\theta = 47.2\%$ and 48.2% — the downward trend below 889 msg/s does not continue, refuting the linear extrapolation one registered prediction relied on — and the 1250 msg/s configuration, predicted mid-step and full, instead pinned on the $2/5$ vertex (40.66–42.00, one at 48.26): probability ≈ 0.08 under the branch law, recorded as a miss — yet that configuration sits six to seven points below the continuous value beside it, which no continuous account produces: the grid is present even where the branch-probability law under-delivers. The payload sweep succeeded where co-location failed because padding adds serialization without contending for the scheduler.

S29. TWO NEIGHBORING LITERATURES, AND WHAT THEY DO ABOUT THE SECOND CLOCK

Two items belong here rather than in the main text, which sits at the journal's twelve-page limit and at its cap of forty-five references. Both are relevant to Section VI-C, and neither changes a result.

TABLE S27
CONSISTENCY AUDIT OF EVERY RUN BEHIND A REPORTED RESULT. EVERY COUNT IS DERIVED AT BUILD TIME FROM `INTEGRITY_WINDOWS/CLOCK_INTEGRITY_BY_CONDITION.CSV` AND `INTEGRITY_BY_CONDITION.CSV`, THE AUDIT'S OWN OUTPUTS. A CONDITION IS USABLE ONLY IF ALL OF ITS RUNS SURVIVE. THIS CORPUS IS NOT THE SPAN RECOUNT OF THE MAIN TEXT'S TABLE I, WHICH COVERS EVERY CLOUD RUN THAT PRODUCED MATCHED EVENTS RATHER THAN EVERY RUN BEHIND A RESULT.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

TABLE S28
H3: MEDIAN BROKER TRANSPORT UNDER THE TWO TIMESTAMPING MODES, BOTH CONFIGURATIONS AT ONE IN-FLIGHT REQUEST UNDER MATCHED LOAD, IN THE CAMPAIGN (E-C3) AND ITS REPLICATION (E-C4). `CALLBACK` IS THE DEFAULT ASYMMETRIC TIMESTAMPING MODE; `INLINE` TIMESTAMPS KAFKA ON THE CALLING THREAD AS REDIS ALREADY DOES. EACH CELL IS THE MEDIAN OVER RUNS OF A PER-RUN MEDIAN. EACH CAMPAIGN CLOSES WITH THE CHANGE FROM `CALLBACK` TO `INLINE` AND A 95% PERCENTILE-BOOTSTRAP INTERVAL, RESAMPLING RUNS WITHIN EACH CELL. THE BETWEEN-SYSTEM GAP SHRINKS BY 0.071 MS (0.039–0.113) IN E-C3 AND BY 0.039 MS (0.008–0.059) IN E-C4; KAFKA'S CHANGE CARRIES 98% AND 107% OF IT, MORE THAN ALL OF IT IN E-C4 BECAUSE REDIS'S CHANGE THERE RUNS THE OTHER WAY, AND REDIS'S INTERVAL SPANS ZERO IN BOTH. CELLS ARE ROUNDED INDEPENDENTLY, SO A CHANGE CAN DIFFER IN ITS LAST DIGIT FROM THE DIFFERENCE OF THE PRINTED CELLS.

Timestamp	Kafka (ms)	Redis (ms)	Difference (ms)
E-C3, 10 *runs per cell*			
`callback` (asymmetric)	0.392	0.106	+0.286
`inline` (symmetric)	0.322	0.107	+0.215
Change	−0.069	+0.001	−0.071
95% bootstrap	[−0.105, −0.041]	[−0.009, +0.020]	[−0.113, −0.039]
E-C4, replication, 30 *runs per cell*			
`callback` (asymmetric)	0.386	0.110	+0.276
`inline` (symmetric)	0.344	0.108	+0.237
Change	−0.042	−0.003	−0.039
95% bootstrap	[−0.059, −0.013]	[−0.015, +0.008]	[−0.059, −0.008]

S29.1. "One-way latency" that is half a round trip

The most common way to avoid the second clock is to use only the first, by measuring a round trip and halving it. Three widely used benchmarks do this, and we checked each against its own source and documentation rather than against custom:

- **OSU Micro-Benchmarks** [S84]. `osu_latency.c` computes `(t_total * 1e6) / (2.0 * iterations)`, and the documentation reports "average one-way latency". The halving is visible only in the source.
- **NetPIPE** [S85]. The documentation is explicit — "[l]atencies are calculated by dividing the round trip time in half for small messages" — and the source comment asserts that the halved value "is now the 1-directional transmission time".
- **Intel MPI Benchmarks** [S86]. The user guide states "[r]eported timings — time $= \Delta t/2$" for `PingPong`, against `PingPing`, which reports the unhalved time. The guide does not call the result a one-way latency.

The three differ in what they disclose — one claims one-way without saying it halves, one says both, one says it halves without claiming one-way — but none of them states the assumption the halving requires, which is that the two directions are equal. That assumption is a strong one on a cloud path, and it is the assumption our own transport proxy declines to make (Section IV-A). We raise it not as a defect in those tools, whose paths are typically symmetric by construction, but because halving is the standard alternative to the subtraction this paper is about, and its cost is unstated wherever we looked.

S29.2. Drift-aware synchronization in MPI benchmarking

Hunold and Carpen-Amarie give the most careful treatment of clocks in the benchmarking literature we found [S87]. They criticize earlier MPI benchmarks for correcting the clock offset only, when node clocks also drift, and adopt a drift-aware synchronization that fits a linear model per process; the duration of a collective is then the difference between the maximum finishing time and the minimum starting time over all processes, taken against that corrected global clock.

The contrast with our setting is the useful part. They remove the second clock by building a better one, and then subtract global timestamps as though a single clock had produced them. That is the right move for their measurement, and it is unavailable for ours in two ways. Their correction is fitted offline over a whole run, whereas a benchmark reporting a latency per message has no such luxury; and a fitted global clock still leaves a residual offset on any individual interval, for which they give no error model, because they never consider the case where such a difference comes out negative. The word does not appear in their manuscript. Our reading is that this is not an oversight but a consequence of scale: at the millisecond durations of a collective over hundreds of processes, a residual of a few microseconds cannot invert the sign, and the question does not arise. It arises here because the interval is smaller than the timestamp resolution.

TABLE S29
INJECTED ONE-WAY BROKER DELAY, APPLIED IDENTICALLY TO BOTH SYSTEMS, FIVE FEEDS. MEDIANS. THE ZERO-DELAY ROW IS REJECTED IN BOTH CONFIGURATIONS AND MARKS THE ABSENCE OF A USABLE BASELINE.

Delay	Kafka event-time	Kafka transp.	Redis event-time	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

S30. WHERE THE TIMESTAMP IS WRITTEN, IN OTHER RUNTIMES

Where it is written instead, and by whom: The mechanism of Section IV needs a timestamp taken by a thread that can be preempted, which is why Section VIII makes no claim about designs that pin a core or timestamp in hardware. A third design is now in production and is worth naming because it is the closest live work on host latency: *netstacklat* timestamps at kernel hook points and reads those timestamps from eBPF, measuring how long a packet waits in the Linux network stack before an application reads it [S88]. Nothing it subtracts is taken by a user-space thread waiting for a core, so the failure this paper reports cannot reach it — corroboration for the scope rather than a counterexample to the mechanism.

Two of its habits are the rules of Section VI, arrived at independently and in a different community. It states the resolution error its histogram imposes rather than leaving a reader to infer it from the bucket edges, and prices the finer alternative it did not take. And it bounds its own observer effect, where this paper measures its probe cost once (Section IV). A monitoring tool that reports both without being asked is the better evidence that the rules cost little, and it is not ours.

And a proof that placement is the thing that decides: Where the timestamp is written is a design choice in this section and an empirical finding in Section VI-D — our own two clients differ by a payload parse, worth 20.8 µs. A 2026 result from concurrency theory says it is neither: it is a limit. Rodríguez, Castañeda and Piña formalize the *Causal Observability Problem* — assign timestamps to the boundary events of a concurrent execution so they reflect real-time order — and prove that no solution at the observable boundary is both complete, never missing a genuine precedence, and sound, never reporting a spurious one [S89]. Which of the two a monitor gets is decided by where the instrumentation sits relative to the operation boundary: inside yields completeness, outside yields soundness. They state that the dichotomy holds "independently of the underlying timestamp mechanism".

The connection is structural rather than evidential, and it is worth saying which. Their setting is shared-memory concurrent objects and their instrument is a monitor object; they report neither failure mode studied here, and nothing in their proof is about resolution, preemption or brokers. What it supplies is the reason Section VI-C gives for ordering the remedies as it does. A better clock does not help because the question was never the clock, and here that is a theorem in another setting rather than an observation in ours.

Section IV argues that an acknowledgment timestamp is late because the thread that writes it must first be scheduled. That is not peculiar to a JVM benchmark, and three widely used runtimes document the same structure in their own terms. None of them frames it as a hazard, which is the point: the property is public, and the consequence for a cross-process subtraction is not drawn anywhere we could find.

The timestamp is taken when the application next asks for it: `librdkafka`, the C client beneath Confluent's Python, Go and .NET Kafka clients (not the benchmark's Java driver, nor our Python harness), serves its delivery-report callback from `rd_kafka_poll()` and the other queue-serving calls [S90]. The callback therefore runs on the application's own thread, at whatever moment that thread next polls, rather than when the broker's acknowledgment arrives. In a harness built on it, that poll gap plays the part our δ_{ack} plays in ours.

The definition itself names the reap, not the completion: `fio` defines completion latency, for asynchronous ioengines, as running to "the time . . . when the I/O's completion was reaped by fio" [S91]. The definition is honest and the scope matters: for synchronous I/O the same documentation says only "to when it was completed". For the asynchronous case — the one used for latency work — a reaping timestamp is written into the definition of the metric.

Vendors document the device-side timestamp; one draws the consequence: AMD states, under `hipEventElapsedTime()`, that events "record their timestamp when they reach the head of the specified stream" and that "the time that the event recorded [sic] may be significantly after the host calls `hipEventRecord()`" [S92]. NVIDIA documents the same mechanism — "[t]he device will record a timestamp for the event when it reaches that event in the stream" [S93, Sec. 9.1.2] — without drawing AMD's conclusion. We note the asymmetry rather than flatten it: both vendors describe a timestamp written at a moment the host does not choose, and only one says out loud how far from the host call it can be.

A stream processor's own latency metric sits in this regime: Flink's latency tracking, an opt-in metric its documentation recommends for debugging only, subtracts two millisecond clock reads: `LatencyStats` records `System.currentTimeMillis() - marker.getMarkedTime()` with no filter [S94], where the marked time was read when the source emitted the marker, often on another machine. Its documentation says the feature is disabled by default, states the precondition, "Flink assumes that the clocks of all machines in the cluster are in sync", and adds that the markers

bypass operators and so miss the time records spend in them [S95]. Once enabled, the metric sits in this paper's regime twice, at one-millisecond resolution and across two clocks, and it keeps its negatives rather than dropping them, the disposition Section V-A finds in two of the ten tools it reads.

S31. Neighboring rules, and what they already require

Two of this paper's recommendations have partial precedents, and the honest thing is to say where. Neither displaces the contribution, because neither is applied where a streaming benchmark reports a latency; but a reader is entitled to see them named.

Counting what was not measured: In the same summary block as its latency percentiles, `sockperf` prints counts of the messages it sent, received and skipped, together with a line counting dropped, duplicated and out-of-order messages, derived from sequence numbers [S96]. The counters are error counters rather than a retention rate, but the reader of that summary can see the denominator the percentiles were computed over, which is what Section VI of the main text asks for.

The OpenTelemetry exponential-histogram format goes one step further and reserves a field for it. `zero_count` holds observations at or below a zero threshold, described in the specification as including "values that have been rounded to zero", and the total `count` is defined to include that bucket [S97]. We do not claim it as a general sub-resolution indicator, because it is not one: it captures unrepresentable mass only at the bottom of the scale. What it establishes is narrower and still useful — that a mainstream telemetry format already treats rounded-to-zero observations as data to be carried beside the total rather than dropped, which is precisely what the admission condition of Section V of the main text does not do.

A prior sighting in a benchmark, blamed on clocks: Klenik and Pataricza report negative derived durations in a Hyperledger Fabric benchmark, call them a serious violation of event causality, and trace them to a peer clock drifting under a lightweight synchronization service [S98]. Their case is genuinely two-clock — the durations oscillate smoothly around zero, a drift signature, the same channel as our Redis $-1000\,\mu$s finding and not the scheduling failure — and their check is post-hoc data validation with no per-run gate and no campaign rate. It is the nearest prior sighting of the observable inside a benchmark, and the attribution went where the field's default sends it: to the clock.

The strongest precedent for our own recommendation, in network measurement: Our rule says to keep the unusable sample and label it rather than delete it, and one system already does exactly that. TimeWeaver infers one-way delays from passively observed NTP traffic, and where earlier methods discarded a host outright for missing timestamps or negative latencies, it keeps those measurements and assigns them to lower accuracy tiers [S99]. That is a fourth disposition beyond the three Section V of the main text finds in benchmark harnesses — not deletion, not substitution, not passing the value on unmarked, but retention with a quality label — and it is the one we would have asked for. It does not compete with this paper: it is network measurement rather than a benchmark harness, it derives no retention arithmetic, and it reads no harness source. We record it because a reader entitled to ask whether anyone has done the sensible thing deserves the answer, which is yes, in a neighboring field.

It is a precedent for our *first* rule as well as our third, and we had missed that. Making the case for tiers, the paper states what the filter it replaces cost: of the $8{,}804$ clients the prior method considered, about $3{,}631$ — two in five — "were rejected due to missing timestamps, negative latency values, and other reasons". That is a reported rejection rate, which is what Section VI of the main text asks a benchmark to report, arrived at in 2018 and for the same reason: a filter's cost is not visible in what survives it. Two qualifications, so the precedent is not claimed wider than it is. The denominator is clients rather than samples, and the rate is TimeWeaver's characterization of the prior method rather than of its own tiering, which it reports instead as counts per tier.

S32. What standardized benchmarks require about resolution

Section VI of the main text asks that a harness state its timestamp resolution and count what it discards. Standardized benchmarking already contains rules of that form, and one of them is almost exactly the rule we propose. Setting them beside the flagship standardized messaging benchmark is the sharpest way to show the gap, so we do that here rather than assert it.

A rule of the right form, in embedded benchmarking: EEMBC's CoreMark-PRO run rules require that "[e]ach workload must run for at least 1000 times the minimum timer resolution" [S100]. That is the form of rule this paper argues for: a floor on the measured interval expressed *relative to the timestamp resolution*, not in absolute time. The distinction is easy to lose, and worth keeping — plain CoreMark sets an absolute ten-second floor instead, with no reference to resolution at all, and the two rules are not interchangeable.

A rule about unusable samples, in power measurement: SPEC's Power and Performance Methodology, clause 8.2.2, says the benchmark "should report the number of samples that may have a return value 'Unknown"' and the number whose uncertainty exceeds the maximum allowed [S101]. Two qualifications matter and we state them rather than round them off: the modal verb is *should*, in a methodology document rather than run rules, and the samples in question are unusable readings rather than samples excluded after the fact. Within those limits it is the same instinct — report the count of what the timestamp could not measure.

Rules about the sign itself, in network measurement: Three standards already require some part of what Section VI asks, and they are the closest precedents we found for treating an impossible value as data rather than as debris. OWAMP requires a packet timestamped in the future to be recorded unaltered within its timeout window [S102]; ETSI requires removed measurement cycles to be reported [S103, Clause 5.1]; and RFC 7679 treats negative singletons as usable distribution data [S104, Sec. 3.5]. None of the three is ours and none is applied where a streaming benchmark reports a latency, which is the gap Section VI names rather than fills for the first time.

And none of it, in standardized messaging: SPECjms2007 is the flagship standardized benchmark for message-oriented middleware. Its run rules record end-to-end latency as a per-destination histogram whose default granularity is 100 ms and whose default upper bound is 10 s [S105], [S106]. Searching the run rules, design document and user's guide for a resolution clause, a minimum measurable interval, or a counter of discarded latency samples returns nothing: the only timing-quality requirement anywhere in them is clause 3.2's bound of 100 ms on *inter-node clock agreement*, which is a skew limit and not a resolution limit. The reported `deliveryTime` element carries an average, a maximum, a granularity and a ninetieth percentile, and no field for samples outside the histogram's range.

We are careful about one thing here. The user's guide does document sample discarding elsewhere — `trimInitialSamples` and `trimFinalSamples`, "this many samples are thrown away" — but those apply to throughput histories rather than to end-to-end latency histograms, and they are configured constants rather than reported counts. It would be wrong to say the benchmark discards nothing; what it does not do is count the latency samples its own resolution makes unusable.

Two tools reached the same precondition independently, and both disclose rather than delete. `zrk`, a rewrite of `wrk2` begun in July 2026, states that at "the sub-millisecond latencies a fast service actually produces" a millisecond artifact "is frequently larger than the number being measured", and records the overshoot into the sample rather than discarding it [S107]. `wrk2` itself concedes in its own documentation that "measured latencies are only accurate to a $+/-$ ~1 msec granularity, due to OS sleep time behavior" [S108]. Neither counts a discard, because neither discards: the timestamp resolution is disclosed and the sample is kept. That is the contrast Section V turns on.

The comparison is therefore not that standards bodies have failed to think about this. One of them states our rule in almost our words, for a different domain. The rule simply has not reached the place where brokers are compared on sub-millisecond paths.

The property those rules refer to is itself a measurement: A floor expressed relative to the timestamp resolution presumes the timestamp's resolution is known, and on a general-purpose platform it frequently is not: the granularity a runtime advertises, the granularity it achieves, and the cost of the call that reads it are three different numbers. TimerMeter quantifies exactly those properties for the timers a runtime actually exposes [S109], which is the measurement a harness must already have made before a rule of CoreMark-PRO's form can be applied to it. We name it because the rule of Section VI has the same precondition: a harness cannot state its timestamp resolution honestly without having measured it, and the corpus here is a case where the advertised figure would have been the wrong one to state.

S33. Where our reading parts from the concurrent report

Section VII-B states the disagreement in three sentences. This is the whole of it, because a dispute with concurrent work should be checkable rather than asserted.

Sharma et al. [S18] count negative timing spans in distributed inference systems and propose a `causality_health` check. Injecting skew, they observe no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. Their measurements we do not dispute; the reading we do, on one sentence: "queueing alone cannot produce negative timing spans".

Under the queue they model — backlog at a service stage — the sentence is true, and Section IV refutes that account of our own corpus as well. It is false as written because two different queues are in play and only one of them is theirs. A message waits in a service backlog; the thread that timestamps the clock waits in the kernel's *run queue* for a CPU. The second wait is what displaces a timestamp, and it is unaffected by whether any backlog exists: our rates are highest at high utilization with no backlog at all.

The consequence is practical. If backlog cannot invert a span, then eliminating backlog leaves skew as the only explanation, and a millisecond threshold on skew becomes a sufficient check. On a loaded machine it is not: we see a negative-span rate of 30.5% at 88% utilization on a single host — where the inter-host offset is not merely small but undefined, there being one clock — and move that rate fiftyfold with a scheduling parameter, touching no clock. Where two hosts are used, `chrony` puts the offset at 0.067 ms, well inside the 3 ms at which they see no violations.

The skew reading is not confined to this dispute. The benchmark's maintainer gave it in 2021, attributing publish latency above end-to-end latency to inter-node skew on a report from a single node [S110]. A 2026 guide to monitoring Kafka consumers still tells its readers that a negative median latency is "a skew problem, not a fast consumer" [S111]. Neither asks whether the thread that took the timestamp was running.

Their own formulation is nearer ours than the dichotomy allows. They note that the minimum separation "exhibits variability due to scheduling jitter", with violation probability rising as ϵ exceeds its quantiles. That is the negative-span law of Section IV-A

with skew written where we write the stall — the same law, attributed to the other term. What a threshold in milliseconds of skew cannot do is transfer to a machine where the stall, not the skew, is the term that grows.

S33.1. The survey behind the two related-work sentences

Section VII-B and Section VII-C state what this paper takes from each of these and what it adds. This is the survey behind those sentences.

The terms, written down, because an empty search is evidence about a phrase: A search that returns nothing has proved something only if the phrase it searched is the one the field uses. Ours was not, for a while: an earlier sweep reported that *negative latency* returns almost nothing, which is true and measures only that nobody has coined it — this paper included, since Section VII-B takes *negative span* from Sharma et al. The two failure modes also live in different literatures, so one community's vocabulary cannot test both: an interval that comes out negative is discussed in distributed tracing and in network measurement, and a filter that deletes a population is discussed in benchmarking and in metrology.

`scripts/novelty_sweep.py` holds 11 search terms across five communities and both failure modes, each recorded with the kind of hit that would threaten the claim. A run writes what each returned to `docs/results/external/novelty_sweep.csv`; that file is the record of a run and is absent from the artifact until one succeeds, because the script declines to write a sweep whose terms it could not all read — it declined twice while this section was being written, once because a count had moved between samples and once under a rate limit. Terms that come back in the thousands are recorded as too broad to have tested anything, which is the honest verdict for a query that cannot be read through; the ones that matter return few enough to open every hit.

Two properties of that file are worth stating here because they are easy to mistake for carelessness. The counts are sampled three times and a count that moved between samples is written as a band with the verdict *ask again* — the index is approximate under load, and one phrase returned three in a batch and one on each of three consecutive calls a minute later, listing the same single work every time. And the ledger is not written at all unless every term came back stable, so a partial sweep leaves the previous record standing rather than replacing it with half of a new one.

The sharpest term is the exact phrase the field uses and this paper adopts, *negative timing spans* rather than any coinage of ours. Its hits were opened rather than counted, and the only work among them that measures a span computing negative is Sharma et al. [S18], cited above and disputed at length.

S33.2. Production tracing reached the conclusion without the mechanism

The observable is familiar outside benchmarking. Distributed tracing systems ship clock-skew adjusters, which "modif[y] start time and log timestamps for spans that appear to be 'off' with respect to the parent span" [S112]. An issue titled *Clock Skew Adjuster considered harmful* objected that the result is "wrong data and really hard to diagnose" [S113], and Jaeger has defaulted the adjustment off since 2020.

A 2026 preprint reaches Villain's conclusion again on current hardware, pairing an edge inference runtime's own timing against a logic-analyzer reference and finding that deployment interference corrupts the timing infrastructure as well as the timing it reports [S114]. It reports no negative spans, no quantized deletion and no law relating either to the interval measured; what it confirms is the premise, fourteen years after Villain and in another domain.

The dead end is still open, and the clearest instance of it is recent. In July 2026 a bug was filed against the OpenTelemetry eBPF instrumentation, reporting child spans whose start times precede their parents' *on same-node resources as well as distributed ones*, and stating that the reporter had checked for clock skew and found none [S115]. It was untriaged when we read it. We claim nothing from it about our own mechanism and the reason is in the report itself: the deltas are given as seconds to minutes, which is orders of magnitude beyond any run-queue stall, so whatever produces them is not Section IV's displacement. What it evidences is the *absence of a second hypothesis*. A practitioner who has ruled out the clock, on one node, has nowhere else to look, and that is the gap the rules of Section VI are meant to close.

That is our conclusion, reached by practitioners, from the symptom alone. What the argument lacked was a mechanism and a magnitude: if the displacement is a late timestamp rather than a skewed clock, then an adjuster is not correcting error but moving evidence, and how much evidence it moves is a measurable quantity nobody had measured. Section IV measures it.

S33.3. A framework that does not have the problem

Section VI says the checks cost nothing, and the strongest evidence that they are practical is not ours. In March 2026 Paul et al. published mq-bench, a framework for exactly the comparisons this paper is about, and evaluated eight brokers with it [S116]. Its measurement design makes five of the choices we recommend, and argues for none of them:

- **Native resolution.** "Each message includes a 24-byte binary header containing a 64-bit send timestamp in UNIX nanoseconds." There is no millisecond resolution, so the deletion law of Section V has nothing to bind on.
- **A span that is a causal chain.** It computes $t_{\mathrm{recv}} - t_{\mathrm{pub}}$, which is the span Table I shows never inverts, rather than the acknowledgment-timed proxy that does.

- **One clock.** “Clock synchronization between publisher and subscriber processes is ensured by running both on the same workload generator machine.”
- **No guard.** We looked for a positivity filter, a clamp or a discard rule and found none.
- **A reported loss fraction.** Its reliability experiment injects TCP resets and reports what did not arrive: “all brokers exhibit approximately 6.3–6.6% message loss” at QoS 0, or $1{,}150$–$1{,}200$ of $18{,}200$ messages, against 0% at QoS 1 and 2.

mq-bench then reports medians as low as 0.21 ms — squarely inside the range where both failures reported here bite — and needs no remedy, because it never acquired either defect.

The last of those bullets is the one we did not expect to find. Our rule asks for the retained fraction to be reported beside the latency, and here it is, in a framework that never argues for the practice: the loss is induced rather than spontaneous, but the reporting is the point. Their first two experiments run at QoS 0, which is at-most-once and has no redelivery, so the headline latencies are drawn from a configuration that can silently drop — and the third experiment is what tells a reader how much. That is the shape the rule asks for, arrived at independently.

One thing it does not do is say why its span is safe. A reader cannot tell from the paper whether the publish-timed difference was chosen because it is a causal chain or because it was the convenient one to instrument, and that is exactly what the rule “name the span you are subtracting” asks an author to make explicit. The choice is right either way; the record of why is what would let the next framework inherit it.

The one choice it does explain is the one our own rule warns about. Publisher and subscriber are co-located, and the stated reason is that co-location “ensures” clock synchronization — which it does, and clock synchronization is the one rival co-location removes. It carries them no further. Had the same framework referenced its span to an acknowledgment instead, one host would have left the scheduling failure exactly where it is, and the reason given for the safe configuration would not have been a reason for the measurement being safe. That is the distinction the rule is for, drawn here on a framework that does not need it.

S33.4. How large the literature is that misses this

A 2025 preprint synthesizes 42 peer-reviewed Kafka studies published between 2015 and 2025 and reports evaluation rigor, configuration disclosure and reproducibility inconsistent across them [S22]. It finds neither of the failures reported here. The field’s own taxonomy has no slot for them either: a 2024 survey reviews 27 benchmark efforts across five dimensions — one of which is *tracked metrics* — and over 21 pages says *benchmark* 169 times and *latency* 33, while *timestamp*, *clock* and *resolution* occur never [S117]. It closes on the aspects it judges overlooked, and the instrument is not among them. The size matters for the same reason the audit of Section V-A does: the instruments are few and shared, so a defect in one is a defect in a decade of comparisons rather than in one project.

The adjacent literatures are alive to benchmarking pitfalls without reaching this one. Fruth et al. catalogue the perils of database benchmarking [S118], Zhang et al. attribute tail latency to its sources through precise load testing [S119], and the scheduling-latency literature has compared measurement approaches across kernels for over a decade [S120]. Each takes the tool’s own reading as given. That is not a criticism of any of them — it is the assumption this paper is about, and its being shared is the reason the omission survives.

Three 2026 data points, two of them inside the regime: Section VII-A states the condition under which the deletion law does not bind: a median above the timestamp resolution. That condition is real and the concession stands. The denominator behind it is small and runs the other way: of the four 2026 broker comparisons we placed against the resolution, three report figures at or below it. The third is an enterprise-scale Apache Pulsar benchmark whose 3.88 ms median sits well above the tick, so the arithmetic of Section V never reaches it [S121] — a case for the concession rather than against it, and the record was assembled only from exceptions until it was looked for. It earns its row twice over: across four pages it names that median seven times, and the words *timestamp*, *clock*, *resolution* and *discard* do not occur in it. Each is one row of `docs/results/external/literature_regime.csv`, and the count in this sentence is read from that file rather than typed. A 2026 practitioner comparison of NATS, Redis and Kafka gives Kafka “sub-10ms latency at p99” and gives the other two no percentile at all: Redis has “sub-millisecond response times” and NATS “sub-millisecond latency”, with “microsecond-level message delivery” [S4]. That is the sharper exhibit, and not because the numbers are small. A ranking that names a percentile for one system and not for the other two cannot be read at all in this regime: sub-millisecond *somewhere* in the distribution is consistent with a median on the timestamp resolution and with a median well above it, and those two have retained fractions that differ by orders of magnitude. The comparison reports no retained fraction, no rejection count, no timestamp resolution and no publish rate, so nothing in it distinguishes the two cases. It is the clearest current instance of a published ranking produced inside the regime this paper is about.

A second 2026 comparison closes that gap from the other side. It ranks Kafka, Redis Streams and NATS on end-to-end p99 and gives Redis Streams 0.8 ms — a percentile, named, and below the tick — against 3.2 and 12.5 ms [S5]. The measurement is across a three-node cluster, so two clocks, and the method is one line about the node shape: no tool, no timestamp source, no synchronization statement, no resolution, no discard count. Unlike the ranking above, this one *can* be placed against the timestamp resolution, and it lands under it. We draw no conclusion about the number, which may be perfectly good; the point

is that nothing reported beside it would let a reader tell, and on a millisecond-resolution cross-node timestamp most of that distribution would compute to zero or to a negative an admission condition would drop.

A third comparison inside the regime shows how the numbers travel after the tool has moved on. A March 2026 practitioner guide gives NATS "sub-millisecond latency up to roughly the 99.7th percentile for small payloads", Redis the same and Kafka only to "around the 94th percentile" [S122]. Naming a percentile that high puts not the median but nearly the whole body of the distribution beneath the tick, which is the deepest into the regime any row of this ledger reaches. What makes it a different exhibit from the two above is where its figures come from: its own latency chart is captioned "Approximate values drawn from published benchmarks — Brave New Geek & Synadia (2024)", so the ranking is a redrawing of a four-year-old third-party measurement with none of its method carried across. *Timestamp*, *clock*, *resolution* and *discard* do not occur in the article at all. Section VII-A argues that the instruments are few and shared, so a defect in one is a defect in a decade of comparisons. This row says the same of the numbers: a figure produced once inside the regime is re-quoted years later by readers who cannot check it, because what would let them check it was never reported beside it.

Against that, the rules of Section VI are being adopted independently, and by a benchmark rather than by a metrics agent. A 2026 voice-agent latency methodology records both endpoints *on one clock* — a dual-channel recording, chosen because "a platform measures from where it stands, and the caller is not standing there" — reports its discard counts per platform and per reason so that "the denominator is auditable", and declines to claim differences below its detector's resolution [S123]. That is the one-clock rule, the retention rule and the resolution rule, arrived at in another domain by people who had not read this paper. We cite it as corroboration that the rules cost little, not as evidence for any claim of ours.

The load-generator half has an older precedent, and a closer one. Section II-A says the achieved rate is recorded per run because the self-validation SPEC and TPC ask for applies to the measurement harness as well as to the system under test. Klenik and Kocsis did exactly that in 2021, porting TPC-C to Hyperledger Fabric [S124]: they instrument their own workers for what they call the scheduling precision reserve — the gap between when a transaction is picked and when it must be submitted — show it against terminal count, and report that it "can drop below zero in the case of hundreds of terminals". What they then do with that negative is the part worth copying. They do not clamp it and they do not drop it; they weigh it against the timescale of the thing being measured, observing that the violation "never exceeds half a second" where TPC-C's constraints "are in the order of magnitude of seconds", and conclude it is tolerable *here*. That is this paper's own question — is the timestamp resolution smaller than the end-to-end latency? — asked about a load generator, answered in the open, and four years earlier. They report their invalidated fraction as well.

The sharpest exhibit for the gap is a tutorial on exactly this subject: If the omission were confined to benchmarks under deadline it would be a smaller claim. It is not. A 2026 tutorial on statistical evaluation for computer-systems papers devotes a section to *latency-specific pitfalls*, opens it with "latency data is the most common metric in systems papers and the most consistently mis-reported", and names three traps: the mean as a summary, coordinated omission, and warmup effects [S125]. The harness's own clock is not among them. Over 31 pages the words *timestamp*, *resolution*, *quantization* and *retention* do not occur; *clock* occurs once, about non-determinism under seeding. Its pre-submission checklist asks for "median + tail percentiles + CDF; warmup discarded" and for no denominator, no resolution and no sign check.

The industrial half of that exhibit is a vendor's own fairness guide. A May 2026 Kafka benchmarking methodology, written expressly to say how to run a comparison that is fair, covers workload profile, hardware normalization, the durability contract and failure scenarios, and says nothing about discarded samples, negative values, timestamp resolution or the publish rate's relation to the clock tick [S126]. Its entire treatment of the subject is the phrase "with outlier handling described". A tutorial and a vendor guide, four months apart, omitting the same thing is a stronger statement about the field than either alone.

A third document closes the set, and it is the one that should have caught this. In July 2026 a broker-monitoring vendor published a methodology guide whose central section is *The Seven Performance Benchmarking Errors* [S127] — not advice on running a benchmark, but an enumeration of the ways one goes wrong. The seven are co-locating the load generator with the broker, skipping JVM warmup, measuring only throughput, not running long enough, unrepresentative message sizes, triggering producer flow control, and not matching production configuration. Every one is about experimental design; none reaches the measurement tool. The words *clock*, *timestamp*, *resolution* and *negative* do not occur on the page. Its first error is co-locating the load generator, while Section VI-D finds that a co-located testbed *hides* the condition deciding the outcome: the two are different concerns, resource contention against exposure, and the guide is right about its own.

We record all three without complaint about their authors, who are doing something else and doing it well. The point is narrower and harder to argue with than any count of benchmarks: a document written in 2026 to enumerate the ways latency measurement goes wrong does not contain the ways this paper reports, which is what it means for a gap to be a gap.

The half of this that another field found the same year: The deletion failure has two halves — a coarse grid, and a filter that throws away what lands on it — and the first half is not confined to computing. A 2026 process-mining preprint rounds manufacturing event logs to 1-, 5- and 10-minute grids and reports that the *top-ranked station changes* in 4.5%, 19.0% and 30.25% of paired logs [S128]. That is Section V's consequence, in a field with no contact with this one: a grid coarse against the interval destroys an ordering rather than adding noise to it, which is the property that makes it dangerous to a comparison. We record it as corroboration and not as a precedent, because the differences are the whole of our contribution: those logs are synthetic and ours are measured, the grid there is applied in analysis and here it is inside the benchmark tool, and —

decisively — rounding keeps every observation while the admission condition deletes the ones that round to zero. Li has the arithmetic without the sign. It is the sign that makes the population unrecoverable and the retained fraction worth reporting.

S33.5. Where scheduling delay comes from, and what the queueing account gets right

A thread that is runnable does not always run. Work-conservation violations — a runnable thread left waiting while a core sits idle — are documented for CFS-era schedulers [S77]; ours is an EEVDF-era kernel, and we make no claim that those particular defects are present.

Queueing gives the leading-order account of waiting, and it is right about the thing that matters here: utilization alone does not fix a waiting time, because pooling and geometry matter at fixed ρ. This is why the geometry contrast of Section IV refutes a single-parameter law rather than queueing itself: multi-server queueing is geometry-dependent at fixed ρ whether or not the scheduler conserves work.

Chandrasekar and Kramberger [S129] apply exactly such a single-parameter M/G/1 framing to benchmark bias. We adopt it as a leading-order account and refute it on the load axis. On a second axis they reach half of our mechanism concurrently and independently: their client's own interpreter lock "artificially inflates" the latencies it reports, charging the harness's own scheduling to the server under test. That is the inflation half. What a positive number cannot show — the negative span, and the sign channel that makes the displacement legible — is what remains ours.

The field already has a name for the shape of this, and it is not ours: The scheduling failure is, stated abstractly, that an agent acts on the state as *observed* and the observation lags the event. Control and queueing theory call that *partial observability*, and it is studied at this venue: Tahir et al. [S130] solve compute-cluster load balancing where the dispatcher's view of queue lengths arrives delayed, and carry *partial observability* as one of their three index terms. We record this for two reasons and claim nothing from it. The first is that the framing is not exotic — a reader who finds Section IV strange has read a version of it before, in the venue this manuscript is submitted to. The second is the difference, which is where the contribution sits: that literature treats the lag as a known property of the *system* and designs a policy around it, whereas here the lag is inside the *instrument*, is not known to the people reading its output, and is charged to the system under test. A delayed observation you have modeled is a control problem. A delayed observation you have not noticed is a measurement artifact, and no policy repairs it because nobody is aware there is a state to estimate.

Tail behavior under load [S75] is the phenomenon our stall distribution samples. That work also shows real-time priority collapsing a tail that renicing leaves alone, applied to the system under test; we apply the same lever to the harness, which is the manipulation of Section IV.

PART V. THE PRE-REGISTERED LAW CAMPAIGN

This part reports in full the campaign that tested the scheduling failure's law in advance, on machines the earlier campaigns never ran on. Section S34 states what was registered and how it was run, Section S35 what every prediction returned, Section S36 every run that was aberrant and why it was marked, and Section S37 what the runs showed once the verdicts were in. Sections S16.10 and S16.11 carry two of its findings in depth: the shape of the wait, and two faults in the programs that judged it. The plan names its blocks A1 to A9 and the three wait predictions A9-1, A9-2 and A9-2b; they are written L1 to L9 and L9-1, L9-2 and L9-2b here, so that none shares a name with the E-labeled campaigns of Parts I to IV.

S34. THE CAMPAIGN AS REGISTERED, AND AS IT RAN

What was registered: The plan was first frozen as version 5 on 16 September 2026, before its first run. It fixed the law's predictions, the rule that confirms each, the campaigns, the rule that sets how many runs each takes, and how the runs are checked and read; the repository's `freezes` folder holds it. Every later change is a new, numbered freeze, public before the runs it covers, whose first section lists what changed, why, and whether a result already seen prompted it; the version in force at the end is the thirty-second, freeze 28. Versions 6 to 8 corrected what stage 0's first runs found in the machines, the code and the plan's own checks, before any campaign that tests a prediction began, and version 8 has P1 and P9 judged only on the slices a pair can reach, since no message arrives sooner than the client's own end-to-end latency with nothing added. One prediction changed form: P3 became P3a and P3b in version 9, before any load run was read. Three kinds of change followed results of the campaigns themselves. Version 14 judges P3a only where the plateau reaches 2%, after the first load campaign showed that a shallower curve places no fall. The stopping rule was corrected or widened in versions 8, 16, 17, 18 and 20, each time after it had stopped a campaign, and set to record rather than stop, for L5's last runs in version 26 and for every campaign in version 32. And versions 19, 21 and 30 corrected faults in programs that had already judged, each stating what the verdicts it touches had been. Blocks were added as the campaign ran, each read by a program fixed before its runs were: the Arm pair's load campaign (version 24); a 3.75 ms slice for L1, second bridge days and a third x86 load campaign (version 27); L9 (version 28); and M0's recording (version 29). P6 was given its program in version 30, at a margin no run had been read against. The law under test is Equation S4: a negative-span rate at a plateau $c + p$ while the end-to-end latency is shorter than the base slice s, a fall across one tick h, and a floor c beyond. The settings alone predict where the fall starts and how wide it is; the plateau's height and the floor are measured. The plan's second half, how benchmark tools report the same deliveries, is reported in the repository's tool records and not here.

The machines: Three pairs of cloud machines ran it, each a driver with eight virtual CPUs and a broker with two, all in one region: two pairs with AMD EPYC 9V74 processors and one with Arm Neoverse N2 processors. The drivers ran their provider's stock kernels, 6.8.0-1065 on the first x86 pair after its first 65 runs on 6.8.0-1064, and 6.8.0-1064 on the second and on the Arm pair; for the tick campaign both x86 drivers also ran three kernels built from one source tree that differ in the tick alone, 1, 4 and 10 ms. Each run read back the slice, tick, kernel, processor and online CPUs before it started; six that ended before the integrity rule could judge them have no reading.

How a run is made: Each run replays messages at a steady 50 a second (M0 replays a football match's bursts instead) and measures, after a 30 s warm-up, its end-to-end latency (the publish-to-receive time on one clock), its publish latency (the time to the acknowledgment's timestamp, the paper's publish latency), and its negative-span rate: the share of messages whose acknowledgment timestamp is later than their receive timestamp. The end-to-end latency is set by delaying only the traffic to the receiver's own network address, so the acknowledgment's path is left alone, and each session first calibrates how far each added delay moves the end-to-end latency. The slice, tick, core-count and L9 blocks place eight end-to-end latencies around the predicted fall at each slice: $0.5s$ and $0.9s$ on the plateau, $s + 0.2h$ to $s + 0.8h$ across the fall, and $s + 1.5h$ and $2(s + h)$ beyond it; the load block places six, and the priority and client blocks three, at $0.9s$, $s + 0.5h$ and $s + 1.5h$. An end-to-end latency shorter than the client's own with nothing added cannot be set, and was not run.

How many runs, in what order: Each campaign that tests a prediction took its number of rounds from simulating its own decision rule on data made to follow the law at the pair's measured spread, at least four and at most forty. L2 and L9 ran at the floor of four, L2 because the simulation takes one tick at a time where its predictions compare two, and L9 because it has no rule to simulate; M0 ran at a fixed ten. A round holds every setup once, in an order shuffled with a recorded seed. Every prediction about where the fall sits is read twice, on a free reading, a curve required only never to rise, and on the law's own fitted shape, and it passes only where both agree. Intervals come from resampling whole rounds, each campaign's from its own.

Which runs count: The integrity rule judges every run the moment it ends. A run counts when every condition it was meant to have took: the load within 3 points of its target, the publish rate within 2%, at least 99% of its messages published and delivered, the added delay captured and within $50\,\mu$s of its setting, and the slice and tick as set before and after it. A run whose condition did not take is repeated, up to three attempts. A run whose median publish latency moved too far stopped its campaign: the *stopping rule*, which from version 20 holds a run against its setup's earlier runs in the campaign and allows the largest of a quarter of the added delay, three times the scatter pooled over the campaign's setups, and 0.25 ms. From version 32 the stopping rule records rather than stops, and every campaign it had stopped was finished on its own queue, calibration and kernel. Every prediction a recorded run bears on is judged with and without the runs the stopping rule would have stopped.

What ran: Table S30 counts it block by block. The registry holds 8,108 runs, taken from 16 to 27 September 2026; 8,102 lie in the 156 campaigns the judging read, 60 of them in the blocks that test predictions and 96 calibrations, baselines, spread pilots and a smoke test, and the other six were set aside before any campaign read them (S36). Of the 8,102, 7,996 counted, 74 were repeated, 16 stopped a campaign and 16 ended before the rule could judge them.

TABLE S30

EVERY BLOCK OF THE CAMPAIGN, AND HOW MANY OF ITS RUNS COUNTED. THE CAMPAIGNS AND RUNS THE JUDGING READ, BY BLOCK (C0, THE CALIBRATION, INCLUDES THE EIGHT CALIBRATIONS OF L2'S BRIDGE DAYS). REPEATED: A CONDITION DID NOT TAKE. STOPPED: THE STOPPING RULE. UNJUDGED: THE RUN ENDED BEFORE THE INTEGRITY RULE READ IT. PAUSED: A MESSAGE IN THE RUN ARRIVED MORE THAN 150 MS LATE (S36); EVERY PAUSED RUN COUNTED.

Block	What it moves	Campaigns	Runs	Counted	Repeated	Stopped	Unjudged	Paused
C0	the added delay (calibration)	86	2,879	2,844	25	0	10	91
B0, P0	baseline, spread pilot	9	290	289	0	1	0	6
L1	the slice, first x86	7	457	452	4	1	0	21
L2	the tick	24	1,242	1,226	7	5	4	42
L3	the load	6	1,339	1,320	17	1	1	39
L4	the slice, Arm	6	240	233	7	0	0	1
L5	the core count	3	159	156	1	2	0	6
L7	real-time priority	1	48	48	0	0	0	1
L8	the client's language	7	733	720	8	4	1	5
L9	the waits, one by one	5	620	616	2	2	0	10
M0	the end-to-end latency under bursts	1	93	90	3	0	0	0
smoke test	–	1	2	2	0	0	0	0
All		156	8,102	7,996	74	16	16	222

S35. EVERY PREDICTION, AS ITS FROZEN JUDGE RETURNED IT

Table S31 gives every verdict. Each is the frozen program's answer on every run it counts, and each was read three more ways: without the 14 runs the stopping rule would have stopped, without all 1,110 taken while it only recorded, and without the 222 a pause held. One verdict moves under any of them, P3a on the first x86 pair, and only on its rule's edge (S36).

TABLE S31

ONE PREDICTION CONFIRMED ON EVERY CAMPAIGN, THREE ON ONE BROKER, AND THE REST NOT. EVERY PRE-REGISTERED PREDICTION OF THE LAW CAMPAIGN, THE NUMBER IT WAS JUDGED ON (ALL RUNS; A SLOPE, RATIO OR MOVE WITH ITS 95% INTERVAL, FROM RESAMPLING WHOLE ROUNDS), AND ITS VERDICT, BROKER BY BROKER WHERE THE TWO DIFFER; A PREDICTION JUDGED BROKER BY BROKER IS CONFIRMED ONLY WHERE IT HOLDS ON BOTH. THE LAST COLUMN IS THE ONE VERDICT THAT MOVES WHEN THE FLAGGED, RECORDED OR PAUSED RUNS ARE LEFT OUT.

	Block, pair	The prediction	On all runs	Verdict	Moves
P1	L1, first x86	the fall's halfway point inside $[s, s+h]$ at four or more slices, its slope on s in 0.8–1.2	slope, free and fitted: Kafka 0.758 (0.596–1.023) and 0.230; Redis 1.007 (0.844–1.169) and 0.662	not confirmed	–
P9	L4, Arm	the same, on the Arm pair, at two or more slices	Kafka 0.830 (0.627–1.172) and 0.285; Redis 1.153 (0.753–1.325) and 0.798	not confirmed	–
P2	L2	the fall 2.5 to 5.5 times wider at a 4 ms tick than at 1 ms	ratio, free and fitted: Kafka 1.784 and 1.818; Redis 1.064 and 1.126 (S16.11)	not confirmed	–
P2b	L2	6 to 14 times wider at a 10 ms tick	Kafka 4.786 and 5.358; Redis 3.045 and 3.339 (S16.11)	not confirmed	–
P2c	L2	the width on the tick a line through zero, slope 0.8–1.2	fitted slope: Kafka 0.968 (0.271–1.125); Redis 0.874 (0.807–1.034)	confirmed on Redis only	–
P2d	L2	the fitted start within 0.25 ms of the slice at every tick	within it at one of twelve slice and tick pairs	not confirmed	–
P3a	L3, five campaigns	the halfway point moves less than 0.25 ms from 75% to 88% load	free move: -0.009, -0.188, -0.086, -0.011 and -0.309 ms	not confirmed on any	first x86, 19 Sep
P3b	L3, five campaigns	load raises the plateau	rise, free: 0.048 (0.044–0.052), 0.047, 0.048, 0.041 and 0.050	confirmed on all five	–
P4	L7, first x86	real-time priority cuts the plateau at least five times wherever it is at least 2%	Kafka 0.0405 to zero; Redis's plateau 0.0069 (S16.11)	confirmed on Kafka only	–
P6	L6	a recording of every thread's waits predicts the rate within 0.67–1.5	median ratio 2.49 to 13.08 over four pair and broker parts; no setup inside	not confirmed	–
P7	L5, second x86	the default slice follows the core count, and the fall follows it	slices read back at 2, 4 and 8 CPUs as the rule gives; the fall on Kafka at 8 CPUs, Redis's fitted shape missing at all three	confirmed on Kafka only	–
P8	L8, first x86 and Arm	Java has the same fall and priority effect, and Python's plateau is at most 1.5 times Java's	Python over Java: 3.251 (2.795–3.907), and 4.253 (4.013–4.560) on Arm	not confirmed	–
L9-1	L9, three pairs	at least 90% of negative readings mostly spent waiting for a CPU	79.4 to 92.0%	not confirmed	–
L9-2	L9	the time-weighted waits predict the rate within 0.67–1.5	median ratio 0.104 to 0.165	not confirmed	–
L9-2b	L9	the share of acknowledgments whose own wait outlasted $T - g$ predicts it	inside the rule on four of six parts; median ratio 1.11 to 1.30	not confirmed	–
M0	M0, first x86	four named causes of the end-to-end latency's slope under bursts	slope on the added delay, traced half: Kafka 0.995; Redis 1.181 and 1.307; every cause dropped	–	–

The slice (P1, P9): On the first x86 pair, on the free reading, the fall's halfway point lay inside its band at every slice both brokers reach, four for Kafka and five for Redis; on the fitted shape it did at two of the four and two of the five. The free slope on the slice was 1.007 for Redis, inside the rule, and 0.758 for Kafka, below it; on the fitted shape both slopes fell short, 0.662 and 0.230. On the Arm pair the free reading held on both brokers and the fitted shape failed on both. The free reading follows the slice where the fitted shape does not because the rate has no plateau to fit: it falls from the shortest end-to-end latencies on (S16.10). P1 and P9 are not confirmed, since each needs both readings. Figure S17 draws every run of both blocks.

The tick (P2 to P2d): None of the tick predictions was confirmed but P2c on Redis (Figure S18 draws the runs). The programs judging P2, P2b and P2c read the campaign's two slices as one curve, and in a world where the law holds exactly they could not have confirmed P2 or P2b (S16.11); read one slice at a time after the fact, the width is proportional to the tick at the 1.5 ms slice and less than proportional at 3 ms, where the rest of the slice spreads the fall as the tick does. P2d, a fitted start within 0.25 ms of the slice at every tick, held at one of the twelve slice and tick pairs. At the 3 ms slice Redis's fitted starts sit 1.1 to 1.6 ms before the slice at every tick, where a fall that begins at once places them; at the 1.5 ms slice Kafka's sit after it, because that slice's fall lies below the shortest end-to-end latency Kafka can make.

Load (P3a, P3b): Five campaigns on the three pairs compared 75% with 88% load (Figure S19); at 50% the plateau, 0.12 to 0.16%, was below the 2% at which the fall can be placed, and that load is reported as out of reach. Load raised the plateau in all five, by 0.041 to 0.050, every interval above zero: P3b is confirmed. That the fall stays where it is, P3a, was confirmed in none: every move was toward shorter end-to-end latencies, -0.009 to -0.309 ms on the free reading, and in every campaign at least one reading's 95% interval reached past ± 0.25 ms; in the first x86 campaign only the fitted one did, by 0.0026 ms.

Priority (P4): On Kafka real-time priority on the timestamping threads took the plateau from 0.0405 to zero in every run, and P4 holds there. Redis's plateau was 0.0069, under the 2% at which P4 makes its claim, and its program words that part "not confirmed" rather than out of reach (S16.11). That frozen answer is the one reported, so P4 is confirmed on Kafka only.

The core count (P7): At 2, 4 and 8 online CPUs the kernel's default slice read back as the rule computes it from the per-CPU constant the machine reports, 1.4, 2.1 and 2.8 ms. The fall followed it on Kafka at 8 CPUs, the one core count whose slice Kafka's shortest end-to-end latency reaches, and P7 holds there; on Redis the free reading placed the fall inside its band at all three core counts and the fitted shape at none, so P7 is confirmed on Kafka only. At 2 CPUs the stopping rule stopped the session twice, and its last thirteen runs were taken with it recording.

The client's language (P8): Kafka's own Java client shows the same fall and the same effect of priority: its rate in the middle of the fall lay between its plateau and its floor in all 2,000 resampled rounds on both pairs, and priority cut its plateau 1,227 times on the x86 pair and 184 times on Arm. But the Python client's plateau was 3.25 times Java's on the x86 pair and 4.25 times on Arm, where P8 allows 1.5.

Predicting the rate from recorded waits (P6, L9): L6's recording counts every `python3` thread's waits in one histogram, and read against each run's margin it over-predicted the rate 2.49 to 13.08 times. L9 recorded every scheduling event and named the thread that records each acknowledgment's timestamp. That thread was mostly waiting for a CPU in 79.4 to 92.0% of negative readings, where L9-1 asks 90%. Its waits summed over time under-predicted the rate six to ten times (L9-2). The share of acknowledgments whose own wait outlasted the margin came closest (L9-2b), inside the rule on four of the six pair and broker parts, and S16.10 draws those waits.

Bursts (M0): Replaying a football match's bursts, Kafka's end-to-end latency grew with the added delay, on the traced half of the runs as M0's reading asks, at a slope of 0.995; Redis's grew at 1.181 with one acknowledgment per message and at 1.307 in batches of 200. Each of the four registered causes was dropped for Redis: the added delay reached the receiver as set, a replay of the receive loop's own rules gave a slope of one, and the receiving thread's wake-ups and held acknowledgments added less than 0.01 ms. What does account for it was found afterwards (S37).

S36. Every aberrant run, marked, and why

The plan's rule for the unexpected is that a run whose conditions held is a result, never a re-run, and that a run which fails is logged with its reasons and kept. So nothing is taken out of Figures S17 to S20: every run of the law's blocks that measured an end-to-end latency and a rate, 4,823 of their 4,838, is drawn at the end-to-end latency it had and the rate it measured, and every aberrant run carries a mark saying why. The marks are read from the repository's records by a program, not assigned by eye, and Table S32 counts them. A run may carry more than one.

TABLE S32

Nine reasons a run is marked, and what each did to the verdicts. Runs of the whole campaign carrying each mark, pair by pair, of the registry's 8,108. The first five keep a run out of every judge; the last four leave it counted. The verdicts were read again without the flagged and the paused runs, and without every run taken while the stopping rule only recorded (S35).

Mark	Why a run carries it	first x86	second x86	Arm	In the verdicts
void	set aside before any campaign read it: two calibrations wrote into one folder on 16 September, and one was opened on top of a running one on 21 September	1	5	0	never read
false start	its campaign stopped before measuring anything worth keeping and was replaced at once by one that ran whole; the text below gives each of the six its cause	3	2	13	never read
repeated	a condition did not take: the load off its target in 55, the run's files unreadable in 15, and another check in six: the publish rate once, the calibration in three and, in two void runs, a ping reading version 7 stopped judging	28	20	28	not counted
stopped	its median publish latency moved more than the stopping rule allows; never a message that arrived before it was published	4	8	4	not counted
unjudged	it ended before the integrity rule read it	9	6	2	not counted
flagged	taken while the stopping rule only recorded, and it would have stopped on it	6	7	1	counted; read again without
paused	a message in it arrived more than 150 ms late: the driver's disk stalled the receiving side, or Kafka's broker stalled	112	109	1	counted; read again without
far	its end-to-end latency, publish latency or held delay lies more than five robust deviations from its setup's other repeats, and past that quantity's floor	27	70	23	counted
traced	in L9's or M0's traced half, whose scheduling events were recorded: L9's tracer raised the rate it recorded, M0's recording moved the end-to-end latency	159	59	126	counted; L9 is read on them

Out of every judge: Six runs are void: the queue once named a run's folder by its key alone and two calibrations wrote into one folder, and once a waiter opened a second calibration on top of a running one; the folders were set aside before anything read them. Eighteen runs belong to six false starts, each replaced at once by a campaign of the same design that ran whole. Three come from two campaigns running at once on the Arm pair, twice: two L4s started three minutes apart on 18 September, and an L8 started 30 s before its twin on 21 September. Every run of theirs judged while both ran measured a load of 99.7 to 99.8% against 75%, which is why the Arm false starts sit well above their blocks' other runs in Figure S17; the twin went on alone, and its two runs from those minutes were repeated. The other three failed alone: an L8 on the Arm pair was placed from a calibration with no zero-delay median publish latency, so every run failed its check; the first x86 pair's L3 of 21 September measured its load 7.7 to 12.0 points above target in each of its three runs; and an L2 session on the second x86 pair was stopped by hand after two counted runs and followed five minutes later by a session on the same kernel that ran whole. The integrity rule repeated 76 runs whose conditions did not take, most often a load more than 3 points off its target, and 16 runs stopped their campaigns on the stopping rule, every one of them on a move in publish latency. No run in the whole campaign had a message arrive before it was published.

Counted, with a warning: Fourteen runs taken while the stopping rule only recorded would have stopped it; left out, they move no verdict. In 222 runs, 112 and 109 of them on the two x86 pairs and one on Arm, a message arrived more than 150 ms late. On the receiving side the consumer and an unrelated process on the same machine stopped writing together while the processors waited on the disk; at Kafka's broker, the broker could not reach its own controller. The receiving-side stalls keep the clock's hour and not each machine's boot, and never came on both x86 machines within 5 s of each other; their cause is not identified. Judged without those runs, every verdict stands but P3a on the first x86 pair, whose fitted interval ends 0.0026 ms outside its bar with them and 0.0037 ms inside without them: a verdict on its rule's edge, and the one on all the runs is the one reported. A further 120 runs lie far from their setup's other repeats in end-to-end latency, publish latency or held delay, by the quality report's own rule, and they are kept, as the plan says. In L9's traced half the event tracer raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, and the gray rings of Figure S20 sit above the unringed runs for that reason; L9-2b's prediction and the rate it is held against come from the same recorded runs.

Extremes that are not aberrations: Some unmarked points lie far from their neighbors because the law puts them there. The highest in Figure S18, an L5 run at 4 CPUs on Redis with a rate of 0.640, had an end-to-end latency of 0.916 ms and a publish latency of 0.941 ms: its margin $T - g$ was below zero, so any wait at all turned a reading negative. Nothing in it went wrong, and it is counted and unmarked.

S37. What the runs showed after the verdicts

Everything in this section was read after the verdicts were known, and none of it changes one.

Why the end-to-end latency grows faster than the delay under bursts: In the replayed football match 46% of messages arrive right behind another message of the same burst. Redis serves a waiting read only the first of them, so each follower waits out one or more further delayed round trips: its end-to-end latency grows by k times the added delay, with $k = 2$ in batches of 200 and $k = r + 2$ for the r-th follower when each message is acknowledged on its own. Measured, the followers grew 2.02 times in batches and 3.07, 4.14, 5.20 and 6.27 times one at a time. Moving each message of the zero-delay runs by its own k predicts the median's slope with nothing fitted, for Kafka and for Redis one at a time and in batches: 1.000, 1.148 and 1.229 against the measured 0.990, 1.137 and 1.233 over all 30 runs of each. The slopes of S35 are the traced half's, the runs M0's reading is held to, and the recording raised Redis's (Table S32). The median is a step, not a slope, and the replay M-H2 froze gives every burst's first message the same end-to-end latency, so it could not keep this cause.

Why P6 over-predicts, in part: L6's histogram, which P6 reads, holds $33{,}122$ to $36{,}870$ waits a run over every `python3` thread, about as many as L9's records show begun by a wake-up alone, $31{,}159$ to $32{,}500$, over the same four pair and broker parts; the waits L9's records show begun by a preemption are not in it. P6 holds every thread's waits against each run's margin, where the rate follows the one thread that records the acknowledgment's timestamp, and why the other threads' waits outlast the margin more often than that thread's own is not read here.

What the waits and the judges showed: The shape of the wait, the rest of another task's slice, is S16.10, and the two faults in the judges are S16.11. L9-2 and L9-2b differ by an exact product, the waits per millisecond times their mean excess over the margin times the ratio of the share of waits over it to the share of acknowledgments over it; each factor is below one, and their product is the six- to tenfold under-prediction.

What stays open: P3a's small shift of the fall toward shorter end-to-end latencies at 88% load, in every campaign; the size of P8's gap between the clients; Kafka's free slope on the slice, 0.758, below what either shape gives the same design; L9-1's share of time spent running; L9-2b on Kafka on the Arm and second x86 pairs, over-predicting at 75% load; and what stalled the x86 disks on the hour.

References

[S1] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

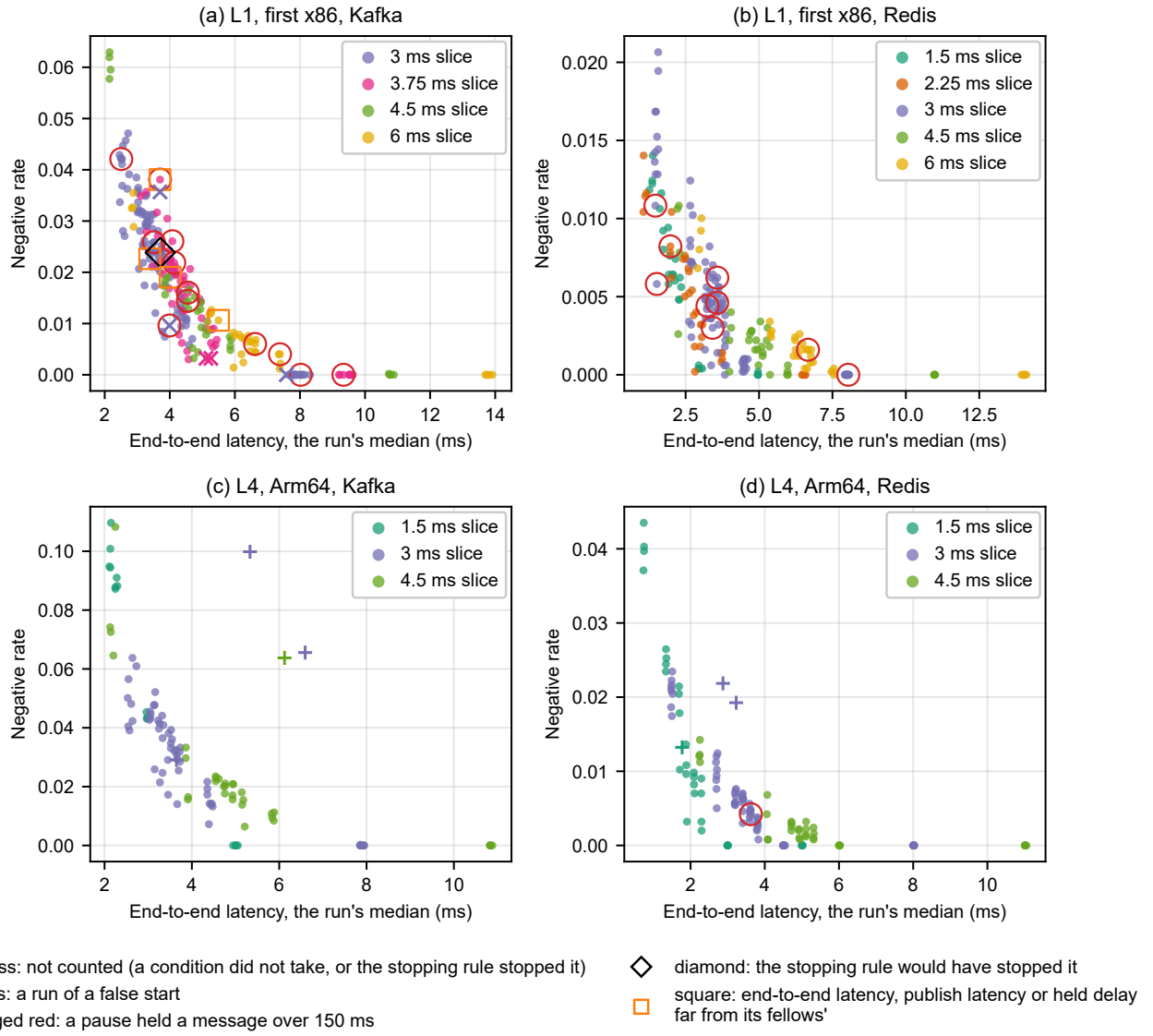


Fig. S17. **Every slice run, aberrant runs marked.** Each run of L1 (first x86 pair) and L4 (Arm pair, Arm64 in the panels) at its median end-to-end latency and its negative-span rate, colored by the slice it ran at. Crosses: runs the integrity rule did not count. Plusses: runs of L4's two false starts. Rings: counted runs with a warning, each reason its own shape (Table S32).

[S2] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, Apr. 2018, pp. 1507–1518.

[S3] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. de Oliveira, A. G. Yaglikci, and O. Mutlu, "Cleaning up the mess: Re-evaluating the real-system modeling accuracy of Ramulator 2.0," arXiv preprint arXiv:2510.15744v4, May 2026, accessed: Sep. 3, 2026.

[S4] Index.dev, "NATS vs Redis vs Kafka: message broker comparison," Practitioner comparison, 2026, https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka. Accessed: Sep. 3, 2026.

[S5] Y. Gao, "Real-time event streaming: Kafka vs Redis Streams vs NATS in 2026," Practitioner comparison, 2026, https://dev.to/young_gao/real-time-event-streaming-kafka-vs-redis-streams-vs-nats-in-2026-34o1. Accessed: Sep. 4, 2026.

[S6] Apache Kafka, "KIP-526: Reduce producer metadata lookups for large number of topics," Apache Software Foundation, Kafka Improvement Proposal, 2020.

[S7] Joint Committee for Guides in Metrology, "Evaluation of measurement data — guide to the expression of uncertainty in measurement (JCGM 100:2008)," https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf, 2008, accessed: Aug. 20, 2026.

[S8] J. W. Pratt, "Remarks on zeros and ties in the Wilcoxon signed rank procedures," *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.

[S9] IEEE, "IEEE standard for terminology and test methods for analog-to-digital converters," IEEE Std 1241-2010, 2011.

[S10] Apache Kafka, "KIP-489: Kafka consumer record latency metric," Apache Software Foundation, Kafka Improvement Proposal, 2019.

[S11] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, accessed: Aug. 21, 2026.

[S12] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel source, commit 2ae891b82695, first released in v6.15, 2025, https://github.com/torvalds/linux/commit/2ae891b826958b60919ea21c727f77bcd6ffcc2c. Accessed: Oct. 15, 2026.

[S13] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

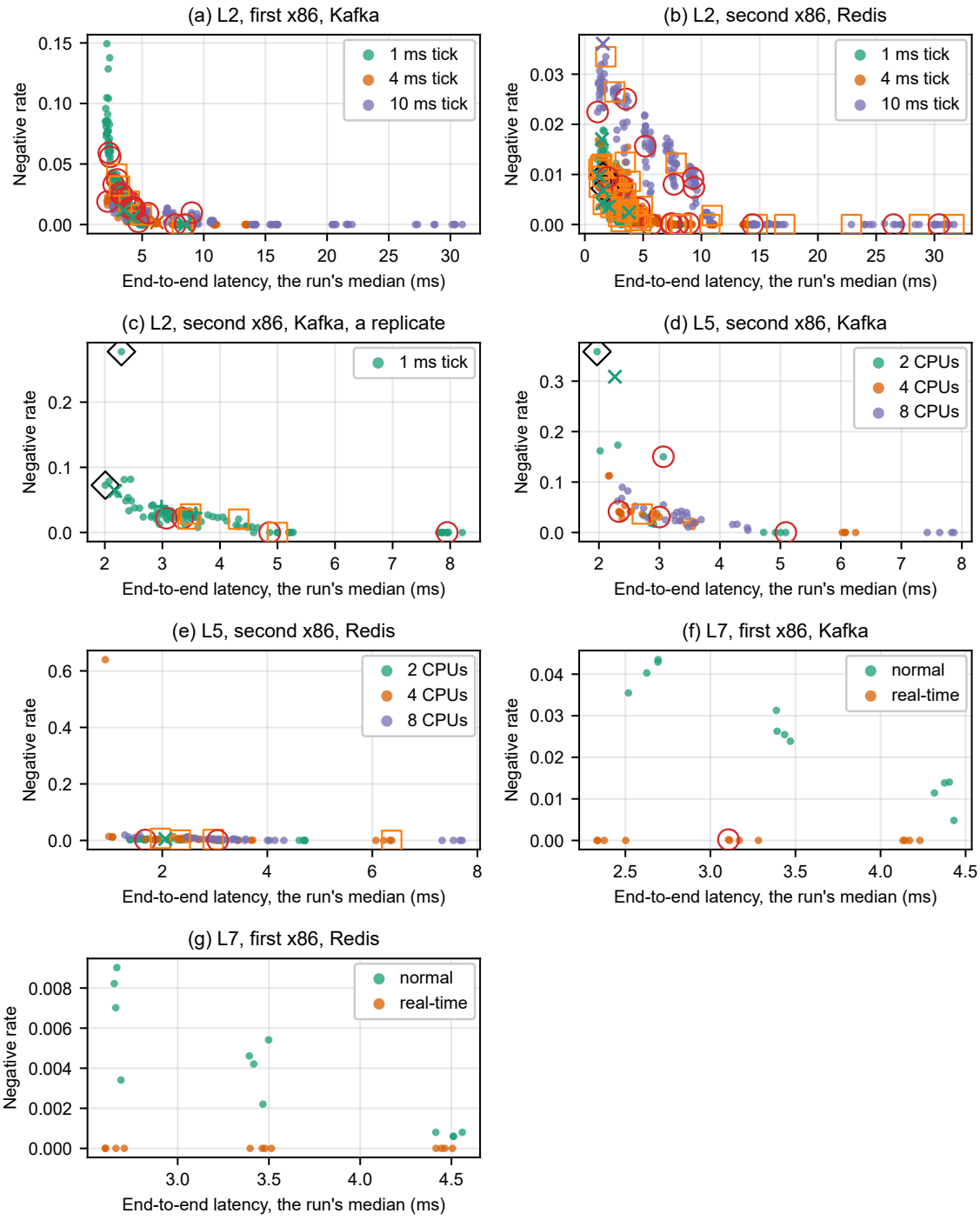


Fig. S18. **Every tick, core-count and priority run, aberrant runs marked.** L2 on the first x86 pair (Kafka) and the second (Redis), colored by the tick, the bridge days on the stock kernel with the 1 ms builds; in (c), the Kafka sessions the second pair ran on the 1 ms build before L2's Kafka half moved to the first pair, a replicate that no judgment pools with the first pair's runs; L5 on the second x86 pair, colored by the online CPUs; L7 on the first, normal against real-time priority. Marks as in Figure S17.

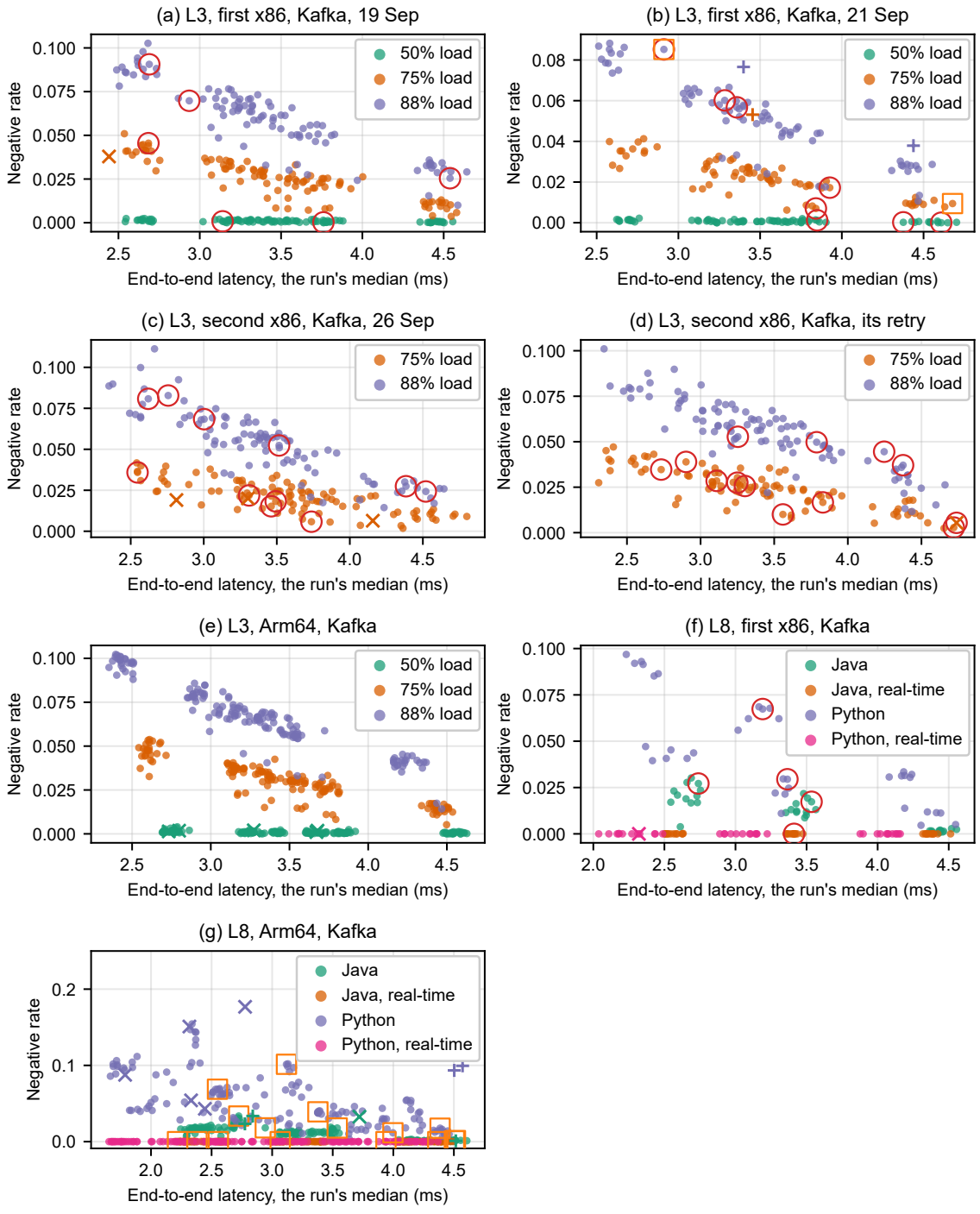


Fig. S19. **Every load and language run, aberrant runs marked.** L3's five campaigns, colored by load, and L8's two, colored by client and priority. Marks as in Figure S17.

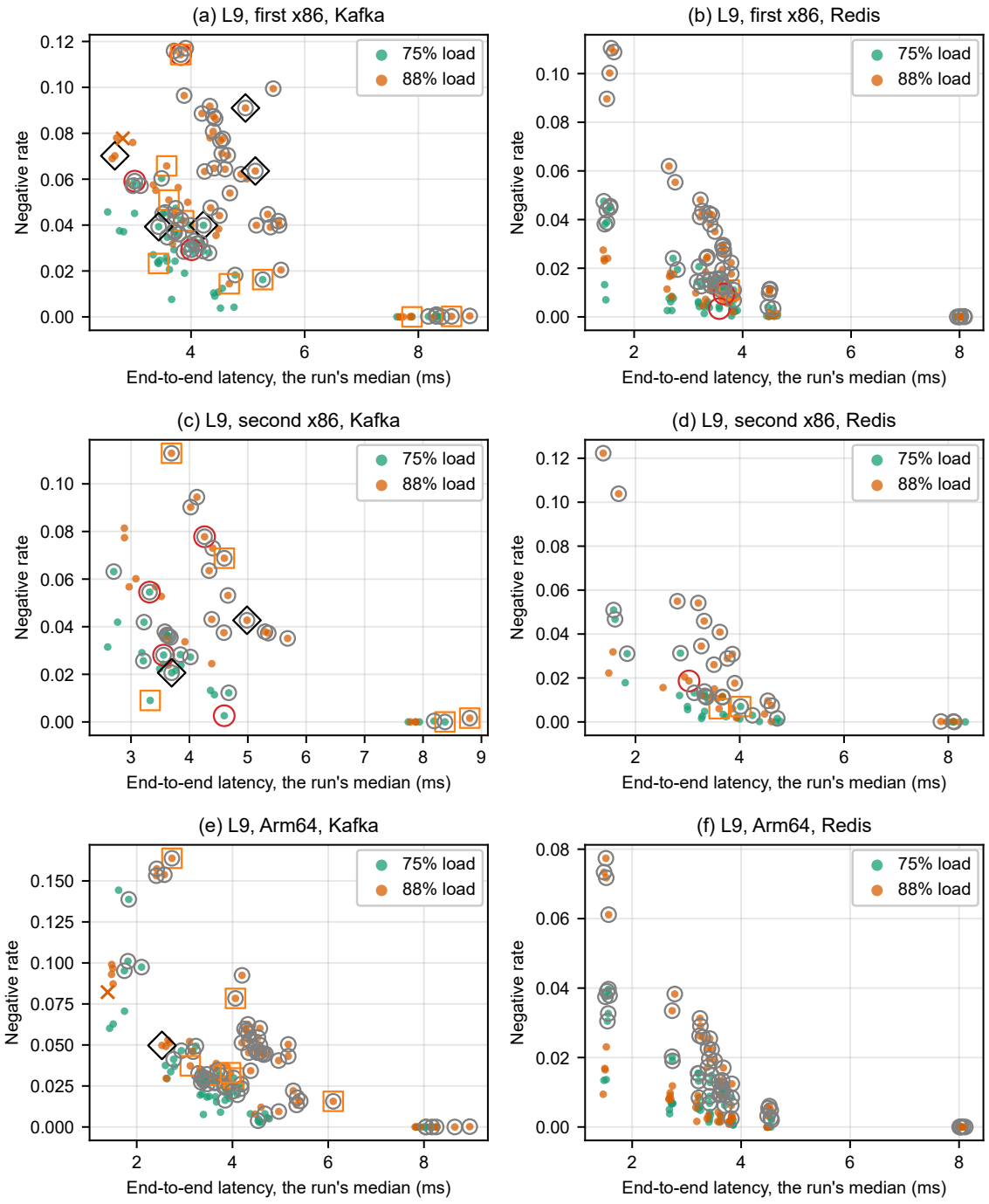


Fig. S20. **Every L9 run, aberrant runs marked.** Both brokers on all three pairs, colored by load; gray rings mark the traced half, whose event tracer raised the rate it recorded. Other marks as in Figure S17.

[S14] J. H. Gliever and D. C. Chu, “Measurement averaging counting apparatus employing a randomly phase modulated time base to improve counting resolution,” U.S. Patent 3,938,042, Feb., 1976, assigned to the Hewlett-Packard Company. Accessed: Sep. 11, 2026.
[S15] J. Rapp, R. M. A. Dawson, and V. K. Goyal, “Estimation from quantized Gaussian measurements: When and how to use dither,” *IEEE Transactions on Signal Processing*, vol. 67, no. 13, pp. 3424–3438, 2019.
[S16] V. Raisanen, G. Grotefeld, and A. Morton, “Network performance measurement with periodic streams,” Internet Engineering Task Force, RFC 3432, Nov. 2002.
[S17] S. Henning, A. Vogel, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, “When should I run my application benchmark?: Studying cloud performance variability for the case of stream processing applications,” in *Proc. 33rd ACM Int. Conf. Foundations of Software Engineering (FSE), Industry Track*, 2025, arXiv:2504.11826.
[S18] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, “Time, causality, and observability failures in distributed AI inference systems,” *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.
[S19] StatsBomb, “Statsbomb open data,” *GitHub Repository*, 2023, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/statsbomb/open-data
[S20] H. Liu, W. Hopkins, A. M. Gomez, and J. S. Molinuevo, “Inter-operator reliability of live football match statistics from opta sportsdata,” *International Journal of Performance Analysis in Sport*, vol. 13, no. 3, pp. 803–821, 2013.
[S21] L. Pappalardo, P. Cintia, A. Rossi, E. Massucco, P. Ferragina, D. Pedreschi, and F. Giannotti, “A public data set of spatio-temporal match events in soccer competitions,” *Scientific Data*, vol. 6, no. 1, p. 236, 2019.
[S22] M. Mohammad, “Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems,” *arXiv preprint arXiv:2512.16146*, 2025.
[S23] T. Downs, “Grow buffer if histogram can’t fit (pull request #398),” OpenMessaging Benchmark, commit 4eb2327c, Dec. 2023, https://github.com/openmessaging/benchmark/pull/398. Accessed: Aug. 19, 2026.
[S24] J. Vanlightly, “Kafka versus Redpanda performance — do the claims add up?” https://jack-vanlightly.com/blog/2023/5/15/kafka-vs-redpanda-performance-do-the-claims-add-up, May 2023.
[S25] S. Guo, “Avoid adding negative metric values (pull request #56),” OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56. Accessed: Aug. 18, 2026.
[S26] HdrHistogram contributors, “Recorder — HdrHistogram API documentation,” Project documentation, 2026, https://hdrhistogram.github.io/HdrHistogram/JavaDoc/org/HdrHistogram/Recorder.html. Accessed: Sep. 10, 2026.
[S27] ——, “hdrhistogram-go: package documentation,” Go package documentation, 2026, https://pkg.go.dev/github.com/HdrHistogram/hdrhistogram-go. Accessed: Sep. 13, 2026.
[S28] ——, “hdrhistogram: crate documentation,” Rust crate documentation, 2026, https://docs.rs/hdrhistogram/. Accessed: Sep. 13, 2026.
[S29] franz1981, “The load generation need to fix coordinated omission (issue #247),” OpenMessaging Benchmark issue tracker, 2022, issue #247, opened 2022-03-01. https://github.com/openmessaging/benchmark/issues/247. Accessed: Sep. 10, 2026.
[S30] SamBarker, “Export serialized HDR histogram to enable statistical significance testing of results (issue #452),” OpenMessaging Benchmark issue tracker, 2026, issue #452, opened 2026-03-26. https://github.com/openmessaging/benchmark/issues/452. Accessed: Sep. 10, 2026.
[S31] Y. Virkar and A. Clauset, “Power-law distributions in binned empirical data,” *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.
[S32] Apache Software Foundation, “Apache ActiveMQ artemis — `artemis perf`,” Source repository, 2026, https://github.com/apache/artemis. Accessed: Oct. 11, 2026.
[S33] ——, “Apache BookKeeper — `BenchThroughputLatency.java`,” Source repository, 2026, https://github.com/apache/bookkeeper. Accessed: Oct. 11, 2026.
[S34] ——, “Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`,” Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.
[S35] Apache Pulsar contributors, “`PerformanceConsumer.java`,” Source repository, 2026, https://github.com/apache/pulsar. Accessed: Aug. 21, 2026.
[S36] Apache Software Foundation, “Apache RocketMQ — `example/.../benchmark/Consumer.java`,” Source repository, 2026, https://github.com/apache/rocketmq. Accessed: Oct. 11, 2026.
[S37] Google, “Cloud pub/sub load test framework — `MetricsHandler.java`,” Source repository, 2026, https://github.com/GoogleCloudPlatform/pubsub. Accessed: Oct. 11, 2026.
[S38] IBM, “Sarama — `broker.go` and `tools/kafka-producer-performance`,” Source repository, 2026, https://github.com/IBM/sarama. Accessed: Oct. 11, 2026.
[S39] Apache Software Foundation, “Apache Kafka trogdor — `TimestampRecordProcessor.java`,” Source repository, 2026, https://github.com/apache/kafka. Accessed: Oct. 11, 2026.
[S40] Synadia, “NATS cli — `cli/latency_command.go`,” Source repository, 2026, https://github.com/nats-io/natscli. Accessed: Oct. 2, 2026.
[S41] Google, “Perfkit benchmarker — `messaging_service_scripts`,” Source repository, 2026, https://github.com/GoogleCloudPlatform/PerfKitBenchmarker. Accessed: Oct. 11, 2026.
[S42] Pravega, “Pravega benchmark — `PerfStats.java`,” Source repository, 2026, https://github.com/pravega/pravega-benchmark. Accessed: Oct. 11, 2026.
[S43] J. Ross, “Quiver — `python/quiver/arrow.py`,” Source repository, 2026, https://github.com/ssorj/quiver. Accessed: Oct. 11, 2026.
[S44] Broadcom, “RabbitMQ PerfTest — `TimestampProvider.java` and `DefaultPerformanceMetrics.java`,” Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test. Accessed: Oct. 2, 2026.
[S45] ——, “Rabbitmq stream perftest — `DefaultPerformanceMetrics.java`,” Source repository, 2026, https://github.com/rabbitmq/rabbitmq-stream-perf-test. Accessed: Oct. 11, 2026.
[S46] Redpanda Data, “kgo-verifier — `pkg/worker/repeater/repeater_worker.go`,” Source repository, 2026, https://github.com/redpanda-data/kgo-verifier. Accessed: Oct. 11, 2026.
[S47] IOP Systems, “Rezolus: systems performance telemetry — `scheduler_discarded_samples`,” Source repository, 2026, https://github.com/iopsystems/rezolus. Accessed: Aug. 20, 2026.
[S48] K. Munegowda, “SBK: Storage benchmark kit — `LatencyRecorder.java`,” Source repository, 2026, https://github.com/kmgowda/SBK. Accessed: Oct. 11, 2026.
[S49] Strimzi, “Strimzi canary — `internal/services/consumer.go`,” Source repository, 2026, https://github.com/strimzi/strimzi-canary. Accessed: Oct. 11, 2026.
[S50] LinkedIn, “Xinfra monitor — `ConsumeService.java`,” Source repository, 2026, https://github.com/linkedin/kafka-monitor. Accessed: Oct. 11, 2026.
[S51] Yahoo, “Yahoo streaming benchmark — `data/src/setup/core.clj`,” Source repository, 2026, https://github.com/yahoo/streaming-benchmarks. Accessed: Oct. 11, 2026.
[S52] A. D. Brunelle, “`blktrace`/`btt` — `btt/inlines.h`,” Source repository, 2026, accessed: Aug. 20, 2026.
[S53] EMQ Technologies, “`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`,” Source repository, 2026, https://github.com/emqx/emqtt-bench. Accessed: Aug. 21, 2026.
[S54] J. Axboe, “fio: Flexible I/O tester — `gettime.c`,” Source repository, 2026, https://github.com/axboe/fio. Accessed: Aug. 20, 2026.
[S55] Confluent, “`librdkafka` — `examples/rdkafka_performance.c`,” Source repository, 2026, https://github.com/confluentinc/librdkafka. Accessed: Oct. 11, 2026.

[S56] A. Krylovsky, "mqtt-benchmark — `client.go`," Source repository, 2026, https://github.com/krylovsk/mqtt-benchmark. Accessed: Oct. 11, 2026.

[S57] G. Tene, "`wrk2` — `src/wrk.c`," Source repository, 2026, https://github.com/giltene/wrk2. Accessed: Aug. 21, 2026.

[S58] HdrHistogram contributors, "`HdrHistogram_c` — `src/hdr_histogram.c`," Source repository, 2026, https://github.com/HdrHistogram/HdrHistogram_c. Accessed: Aug. 21, 2026.

[S59] Y. Mittal, "An empirical study of open-loop load generators for microservices and web workloads," Master's thesis, University of Illinois Urbana-Champaign, 2025, https://hdl.handle.net/2142/129222. Accessed: Aug. 21, 2026.

[S60] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, https://issues.apache.org/jira/browse/KAFKA-19888. Accessed: Aug. 21, 2026.

[S61] ——, "Kafka Streams `StreamTask.java`: end-to-end latency recording (KIP-613)," Source repository, 2026, https://github.com/apache/kafka/blob/trunk/streams/src/main/java/org/apache/kafka/streams/processor/internals/StreamTask.java. Accessed: Sep. 13, 2026.

[S62] L. Kleinrock, *Queueing Systems: Volume I - Theory*. Wiley-Interscience, 1975.

[S63] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Mar. 2009, pp. 265–276.

[S64] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, Oct. 2007, pp. 57–76.

[S65] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.

[S66] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, "Taming performance variability," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 409–425.

[S67] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*. ACM, 2013, pp. 63–74.

[S68] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *Proc. 4th IEEE EuroS&P*. IEEE, 2019, pp. 310–325.

[S69] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, Nov. 2015, pp. 73:1–73:12.

[S70] H. Weyl, "Über die gleichverteilung von zahlen mod. eins," *Mathematische Annalen*, vol. 77, pp. 313–352, 1916.

[S71] V. T. Sós, "On the distribution mod 1 of the sequence $n\alpha$," *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, vol. 1, pp. 127–134, 1958.

[S72] L. Schuchman, "Dither signals and their effect on quantization noise," *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

[S73] S. McCanne and C. Torek, "A randomized sampling clock for CPU utilization estimation and code profiling," in *Proc. USENIX Winter Technical Conf.*, 1993, pp. 387–394.

[S74] J. Ousterhout, "Always measure one level deeper," *Communications of the ACM*, vol. 61, no. 7, pp. 74–83, 2018.

[S75] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, Nov. 2014, pp. 1–14.

[S76] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[S77] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.

[S78] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[S79] S. Herrnleben, M. Leidinger, V. Lesch, T. Prantl, J. Grohmann, C. Krupitzer, and S. Kounev, "ComBench: A benchmarking framework for publish/subscribe communication protocols under network limitations," in *Performance Evaluation Methodologies and Tools (VALUETOOLS 2021)*. Springer, 2021.

[S80] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop 2015, St. Louis, MO, USA. https://www.youtube.com/watch?v=lJ8ydIuPFeU.

[S81] ——, "HdrHistogram: A high dynamic range histogram," 2015, available at hdrhistogram.org.

[S82] R. A. Wannamaker, S. P. Lipshitz, J. Vanderkooy, and J. N. Wright, "A theory of nonsubtractive dither," *IEEE Transactions on Signal Processing*, vol. 48, no. 2, pp. 499–516, 2000.

[S83] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[S84] Network-Based Computing Laboratory, Ohio State University, "OSU micro-benchmarks 7.5 — `osu_latency`," https://mvapich.cse.ohio-state.edu/benchmarks/, 2025, accessed: Aug. 20, 2026.

[S85] D. Turner and X. Chen, "NetPIPE: A network protocol independent performance evaluator," https://netpipe.cs.ksu.edu/, 2025, accessed: Aug. 20, 2026.

[S86] Intel Corporation, "Intel MPI benchmarks — `PingPong`," Intel MPI Library User Guide, 2021, accessed: Aug. 20, 2026.

[S87] S. Hunold and A. Carpen-Amarie, "Reproducible MPI benchmarking is still not as easy as you think," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 12, pp. 3617–3630, 2016.

[S88] S. Sundberg, A. Brunstrom, S. Ferlin-Reiter, J. D. Brouer, and T. Høiland-Jørgensen, "Waiting at the front door: Continuous monitoring of latency in the host network stack," in *Proc. ACM Internet Measurement Conference (IMC)*, 2026.

[S89] G. V. Rodríguez, A. Castañeda, and M. Piña, "On the limits of causal observation in shared-memory systems," arXiv preprint arXiv:2606.14093, 2026, accessed: Sep. 10, 2026.

[S90] Confluent, "`librdkafka` introduction — delivery reports," https://github.com/confluentinc/librdkafka, 2026, accessed: Aug. 20, 2026.

[S91] J. Axboe, "`fio` HOWTO — interpreting the output," https://fio.readthedocs.io/, 2026, accessed: Aug. 20, 2026.

[S92] Advanced Micro Devices, "HIP runtime API: Event management," https://rocm.docs.amd.com/projects/HIP/en/latest/, 2026, accessed: Aug. 20, 2026.

[S93] NVIDIA Corporation, "CUDA C++ best practices guide, using CUDA GPU timers," https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/, 2026, accessed: Aug. 20, 2026.

[S94] Apache Flink contributors, "`LatencyStats.java`," Source repository, 2026, https://github.com/apache/flink/blob/master/flink-runtime/src/main/java/org/apache/flink/streaming/util/LatencyStats.java. Accessed: Sep. 14, 2026.

[S95] ——, "Metrics: latency tracking," Apache Flink documentation, 2026, https://nightlies.apache.org/flink/flink-docs-master/docs/ops/metrics/. Accessed: Sep. 14, 2026.

[S96] Mellanox Technologies, "`sockperf`: network benchmarking utility," https://github.com/Mellanox/sockperf, 2026, accessed: Aug. 20, 2026.

[S97] OpenTelemetry Authors, "OpenTelemetry metrics data model — exponential histogram: Zero count and zero threshold," https://opentelemetry.io/docs/specs/otel/metrics/data-model/, 2026, accessed: Aug. 20, 2026.

[S98] A. Klenik and A. Pataricza, "Adding semantics to measurements: Ontology-guided, systematic performance analysis," *Acta Cybernetica*, 2022, arXiv:2112.11270.

[S99] R. Durairajan, S. K. Mani, P. Barford, R. Nowak, and J. Sommers, "TimeWeaver: Opportunistic one way delay measurement via NTP," in *Proc. 30th International Teletraffic Congress (ITC 30)*, 2018, pp. 185–193.

[S100] EEMBC, "CoreMark-PRO run rules," https://github.com/eembc/coremark-pro, 2026, accessed: Aug. 20, 2026.

[S101] Standard Performance Evaluation Corporation, "SPEC power and performance benchmark methodology, version 2.3," https://www.spec.org/power/docs/SPEC-Power_and_Performance_Methodology.pdf, 2025, accessed: Aug. 20, 2026.
[S102] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
[S103] ETSI, "Speech and multimedia transmission quality (STQ); QoS aspects for popular services in mobile networks; part 4: Requirements for quality of service measurement equipment," European Telecommunications Standards Institute, Technical Specification TS 102 250-4 V2.3.1, Nov. 2019.
[S104] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, Ed., "A one-way delay metric for IP performance metrics (IPPM)," Internet Engineering Task Force, RFC 7679, Jan. 2016, STD 81.
[S105] Standard Performance Evaluation Corporation, "SPECjms2007 run and reporting rules, version 1.0," https://www.spec.org/jms2007/docs/RunRules.html, 2014, accessed: Aug. 20, 2026.
[S106] K. Sachs, S. Kounev, J. Bacon, and A. P. Buchmann, "Performance evaluation of message-oriented middleware using the SPECjms2007 benchmark," *Performance Evaluation*, vol. 66, no. 8, pp. 410–434, 2009.
[S107] zoxy-io, "`zrk`: a Zig rewrite of `wrk2`," https://github.com/zoxy-io/zrk, 2026, accessed: Aug. 21, 2026.
[S108] G. Tene, "`wrk2`: a constant-throughput, correct-latency variant of `wrk`," https://github.com/giltene/wrk2, 2019, accessed: Aug. 21, 2026.
[S109] M. Kuperberg, M. Krogmann, and R. H. Reussner, "TimerMeter: Quantifying properties of software timers for system analysis," in *Proc. 6th Int. Conf. Quantitative Evaluation of Systems (QEST)*, 2009, pp. 85–94.
[S110] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, accessed: Oct. 15, 2026.
[S111] C. Hasija, "Kafka consumer lag is lying to you – how to measure and fix it properly," Blog post, https://chiraghasija.cc/posts/kafka-consumer-group-lag-monitoring-production-2026/, 2026, accessed: Sep. 14, 2026.
[S112] Jaeger contributors, "`model/adjuster/clockskew.go`," https://github.com/jaegertracing/jaeger, 2026, version 1.57.0. Accessed: Aug. 21, 2026.
[S113] ——, "Clock skew adjuster considered harmful (issue #1459)," Issue tracker, 2019, https://github.com/jaegertracing/jaeger/issues/1459. Accessed: Aug. 21, 2026.
[S114] A. Swami and N. Chougule, "Architecture dependent temporal observability under deployment interference in edge inference systems," *arXiv preprint arXiv:2605.17701*, 2026.
[S115] OpenTelemetry eBPF Instrumentation contributors, "Child span start time precedes parent span start time (issue #2568)," Issue tracker, 2026, https://github.com/open-telemetry/opentelemetry-ebpf-instrumentation/issues/2568. Accessed: Sep. 13, 2026.
[S116] T. C. Paul, P. Lertpongrujikorn, H. D. Nguyen, and M. Amini Salehi, "Benchmarking message brokers for IoT edge computing: A comprehensive performance study," arXiv preprint arXiv:2603.21600, 2026, accessed: Aug. 25, 2026.
[S117] W. Yue, M. Boissier, and T. Rabl, "A survey of stream processing system benchmarks," in *Performance Evaluation and Benchmarking (TPCTC)*, ser. Lecture Notes in Computer Science, Hasso Plattner Institut. Springer, 2025, pp. 24–43.
[S118] M. Fruth, S. Scherzinger, W. Mauerer, and R. Ramsauer, "Tell-tale tail latencies: Pitfalls and perils in database benchmarking," in *Performance Evaluation and Benchmarking (TPCTC)*, ser. Lecture Notes in Computer Science. Springer, 2022, pp. 119–134.
[S119] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, Jun. 2016, pp. 456–468.
[S120] F. Cerqueira and B. B. Brandenburg, "A comparison of scheduling latency in Linux, PREEMPT_RT, and LITMUS^RT," in *Proc. 9th Annual Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPERT)*, 2013, pp. 19–29.
[S121] M. R. C. Mukkolakkal, "1.5 million messages per second on 3 machines: Benchmarking and latency optimization of Apache Pulsar at enterprise scale," arXiv preprint arXiv:2603.29113, 2026, accessed: Sep. 10, 2026.
[S122] Java Code Geeks, "NATS vs. Kafka vs. Redis Streams for Java microservices: when "simpler" actually wins," Practitioner comparison, Mar. 2026, https://www.javacodegeeks.com/2026/03/nats-vs-kafka-vs-redis-streams-for-java-microservices-when-simpler-actually-wins.html. Accessed: Sep. 12, 2026.
[S123] Openbenchmarks, "How voice agent latency is measured," Methodology page, 2026, https://openbenchmarks.com/voice-agent-latency/how-voice-agent-latency-is-measured. Accessed: Aug. 31, 2026.
[S124] A. Klenik and I. Kocsis, "Porting a benchmark with a classic workload to blockchain: TPC-C on Hyperledger Fabric," in *arXiv preprint arXiv:2112.11277*, 2021, accessed: Sep. 4, 2026.
[S125] B. Krishnamachari, "How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results," arXiv preprint arXiv:2605.00428, 2026, accessed: Sep. 3, 2026.
[S126] AutoMQ, "Kafka benchmark methodology: run fair comparisons," Vendor methodology guide, May 2026, https://www.automq.com/blog/kafka-benchmark-methodology-run-fair-comparisons. Accessed: Sep. 3, 2026.
[S127] meshIQ, "ActiveMQ performance benchmarking: the complete methodology guide," Practitioner guide, 2026, https://www.meshiq.com/blog/activemq-performance-benchmarking-methodology/, 21 July 2026. Accessed: Sep. 4, 2026.
[S128] A. Li, "When timestamp rounding changes bottleneck rankings," SSRN preprint, 2026, accessed: Sep. 13, 2026.
[S129] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, May 2026.
[S130] A. Tahir, B. Alt, A. Rizk, and H. Koeppl, "Load balancing in compute clusters with delayed feedback," *IEEE Transactions on Computers*, vol. 72, no. 6, pp. 1610–1622, 2023.